AJL Ferro e Aco LTDA

Documentacao Tecnica

Modulo de Emissao Propria de NF-e

Certificado A1 | SEFAZ-PR | NF-e 4.00 | DANFE PDF

Middleware	api-odoo-rhzf.onrender.com
Odoo	ajlferroeaco.odoo.com (SaaS 19.2)
Ambiente	Homologacao (tpAmb=2)
UF Emitente	PR (cUF=41)

Documento gerado automaticamente a partir do repositorio GitHub.
Contem codigo-fonte atualizado de todos os arquivos do modulo de NF-e.

Sumario

1. Visao Geral do Modulo NF-e

1.1 Fluxo de Emissao
1.2 Arquitetura dos Arquivos
1.3 Variaveis de Ambiente
1.4 Endpoints da API

2. Certificado Digital A1 (nfe-cert.js + pfx-openssl.js)

2.1 services/nfe-cert.js
2.2 services/pfx-openssl.js

3. Gerador de XML NF-e 4.00 (sieg-nfe-xml.js)

4. Assinatura Digital XMLDSig (nfe-signer.js)

5. Comunicacao com a SEFAZ (sefaz-client.js)

6. Orquestracao da Emissao (sieg-api.js)

7. Processamento de Faturas Pendentes (sieg-odoo-emit.js)

8. Cancelamento de NF-e

8.1 services/nfe-cancelamento.js
8.2 services/nfe-cancelar-odoo.js

9. DANFE - PDF (danfe-pdf.js)

10. Rotas da API (routes/)

10.1 routes/nfe-cert.js
10.2 routes/sieg.js

11. Script Odoo - Botao Cancelar NF-e

12. Documentacao de Referencia

1. Visao Geral do Modulo NF-e

O modulo de emissao propria de NF-e do sistema AJL permite a emissao direta de Notas Fiscais Eletronicas (NF-e modelo 55, versao 4.00) com certificado digital A1, sem dependencia da SIEG para a emissao propriamente dita. O XML e gerado no middleware, assinado com o certificado A1 da empresa e enviado diretamente a SEFAZ do Parana via SOAP 1.2 com mTLS. Apos a autorizacao (cStat 100), o DANFE em PDF e gerado localmente usando PDFKit + bwip-js e anexado a fatura no Odoo junto com o XML do nfeProc.

O fluxo e acionado automaticamente: quando uma fatura no Odoo recebe o status "pendente" no campo x_studio_nfe_status, o sistema de polling do middleware detecta a fatura, extrai todos os dados necessarios via XML-RPC (empresa, parceiro, linhas, impostos), monta o XML da NF-e, assina, envia a SEFAZ e atualiza o Odoo com o resultado. Tudo acontece de forma assincrona e o usuario final ve o resultado no chatter da fatura.

1.1 Fluxo de Emissao

O fluxo completo de emissao de uma NF-e no sistema AJL segue as etapas abaixo, desde a marcacao da fatura no Odoo ate a disponibilizacao do DANFE e XML no chatter:

- Odoo: Fatura (account.move) e marcada com x_studio_nfe_status = "pendente"
- sieg-odoo-emit.js: Polling detecta faturas pendentes via XML-RPC search
- sieg-odoo-emit.js: Leitura completa da fatura, empresa, parceiro, linhas e produtos
- sieg-nfe-xml.js: Geracao do XML NF-e 4.00 (NFe + infNFe + det + impostos + pag)
- nfe-signer.js: Assinatura digital XMLDSig RSA-SHA1 do grupo infNFe
- sefaz-client.js: Envia lote via SOAP 1.2 + mTLS para NFeAutorizacao4
- sefaz-client.js: Se cStat 103/105, faz polling via NFeRetAutorizacao4
- sefaz-client.js: cStat 100 = autorizada; monta o nfeProc (NF-e + protNFe)
- danfe-pdf.js: Gera o DANFE em PDF (A4 retrato, code128, layout padrao)
- sieg-odoo-emit.js: Atualiza fatura no Odoo (chave, protocolo, status)
- sieg-odoo-emit.js: Anexa XML + PDF no chatter da fatura

1.2 Arquitetura dos Arquivos

O modulo de NF-e e composto por 11 arquivos de codigo fonte organizados em tres camadas: servicos (logica de negocio), rotas (API REST) e scripts Odoo (automacao). A tabela abaixo resume cada arquivo e sua responsabilidade dentro do sistema:

Arquivo	Camada	Responsabilidade
sieg-nfe-xml.js	Servico	Gerador de XML NF-e 4.00 completo
nfe-signer.js	Servico	Assinatura digital XMLDSig RSA-SHA1
nfe-cert.js	Servico	Cofre do certificado A1 (.pfx/.p12)
pfx-openssl.js	Servico	Fallback de leitura PFX via OpenSSL CLI
sefaz-client.js	Servico	Comunicacao mTLS + SOAP com a SEFAZ
nfe-cancelamento.js	Servico	Evento de cancelamento (evCancNFe)
nfe-cancelar-odoo.js	Servico	Integracao cancelamento + Odoo
danfe-pdf.js	Servico	Geracao do DANFE em PDF (PDFKit + bwip-js)
sieg-api.js	Servico	Orquestracao: modo proprio vs SIEG
sieg-odoo-emit.js	Servico	Polling de faturas pendentes + Odoo integration
routes/nfe-cert.js	Rota	Endpoints de certificado + DANFE
routes/sieg.js	Rota	Endpoints de emissao + cancelamento
criar-botao-cancelar-nfe.py	Script	Cria botao "Cancelar NF-e" no Odoo

1.3 Variaveis de Ambiente

O comportamento do modulo de NF-e e controlado por variaveis de ambiente configuradas no servidor (Render). As variaveis principais determinam o modo de emissao, a UF do emitente, o ambiente (producao/homologacao) e as credenciais do certificado digital:

Variavel	Default	Descricao
NFE_EMISSAO_MODO	proprio	proprio = assina e autoriza na SEFAZ direto; sieg = importa XML autorizado
NFE_UF	PR	UF do emitente; define qual webservice da SEFAZ usar
SIEG_TP_AMB	2	1 = producao, 2 = homologacao
NFE_CERT_PFX_BASE64	-	Conteudo do .pfx em base64 (sobrevive deploys)
NFE_CERT_SENHA	-	Senha do certificado A1
NFE_CERT_DIR	/var/data	Diretorio do certificado em disco
NFE_CERT_KEK	-	Chave para cifrar a senha em disco (AES-256-GCM)
NFE_DANFE_PROVIDER	local	local = PDFKit; sieg = via API SIEG
SIEG_IMPORT_XML	0	1 = importa XML autorizado no cofre SIEG apos emissao
NFE_TLS_INSECURE	0	1 = desativa validacao TLS (debug apenas)

1.4 Endpoints da API

Os endpoints relacionados a NF-e estao divididos em dois grupos: os endpoints de certificado (montados em routes/nfe-cert.js) e os endpoints de emissao/cancelamento (montados em routes/sieg.js). Todos exigem o header x-api-key para autenticacao:

Metodo	Rota	Funcao
POST	/api/v1/nfe/certificado	Upload do .pfx (base64 + senha)
GET	/api/v1/nfe/certificado	Status/validade do certificado carregado
DELETE	/api/v1/nfe/certificado	Remove o certificado do servidor
GET	/api/v1/nfe/sefaz/status	Status do servico da SEFAZ (testa mTLS)
POST	/api/v1/nfe/danfe	Gera DANFE PDF a partir de XML autorizado
POST	/api/v1/sieg/emitir	Emitir NF (auto-detect NF-e ou NFS-e)
POST	/api/v1/sieg/emitir-nfe	Emitir NF-e (forcar produto)
POST	/api/v1/sieg/process-pending	Processar emissoes pendentes (polling)
POST	/api/v1/sieg/cancelar	Cancelar NF-e (chave + protocolo)

2. Certificado Digital A1

O certificado digital A1 e o componente central da emissao propria de NF-e. Ele e utilizado para assinar o XML da nota (XMLDSig RSA-SHA1) e estabelecer a conexao mTLS com os webservices da SEFAZ. O sistema suporta tres fontes de carregamento: variaveis de ambiente (prioridade, sobrevive deploys), disco persistente (fallback), e upload via endpoint REST. A senha do certificado e cifrada em disco usando AES-256-GCM com uma chave derivada do ambiente.

A abertura do arquivo PFX tenta primeiro o node-forge (compativel com algoritmos classicos 3DES/RC2). Se falhar com "Unsupported PKCS12 PFX data" (comum em certificados ICP-Brasil recentes que usam PBES2/AES-256), o sistema faz fallback para o OpenSSL do sistema operacional via CLI, tentando o modo moderno e, se necessario, o modo -legacy para algoritmos antigos. A senha nunca e escrita em logs ou retornada nas respostas da API.

2.1 services/nfe-cert.js

Cofre do certificado digital A1 (.pfx/.p12). Gerencia carregamento, cache, persistencia em disco (com senha cifrada), extracao de chave privada/chain em PEM, e validacao de validade. Tenta node-forge primeiro, depois OpenSSL como fallback.

nfe-cert.js (/home/z/my-project/API-Odoo/services/nfe-cert.js)

```
/** * services/nfe-cert.js — Cofre do certificado digital A1 (.pfx/.p12) *
============================================================== * O certificado eh enviado por endpoint protegido
(routes/nfe-cert.js) ou, * como fallback, lido das variaveis de ambiente NFE_CERT_PFX_BASE64 / NFE_CERT_SENHA. * *
Persistencia: grava em NFE_CERT_DIR (default /var/data, fallback os.tmpdir()). * No Render, configure um Disk
persistente em /var/data para nao precisar * reenviar o certificado a cada deploy. * * Nunca logamos a senha nem o
conteudo do .pfx. */ var fs = require('fs'); var os = require('os'); var path = require('path'); var crypto =
require('crypto'); var forge = require('node-forge'); var openPfxWithOpenssl =
require('./pfx-openssl').openPfxWithOpenssl; var DIR = process.env.NFE_CERT_DIR || '/var/data'; var PFX_FILE =
'nfe-cert.pfx'; var META_FILE = 'nfe-cert.meta.json'; var cache = null; // { pfx: Buffer, senha: string, info: {...}
} function ensureDir() { var dir = DIR; try { fs.mkdirSync(dir, { recursive: true }); fs.accessSync(dir,
fs.constants.W_OK); } catch (e) { dir = path.join(os.tmpdir(), 'nfe-cert'); fs.mkdirSync(dir, { recursive: true }); }
return dir; } /** Chave simetrica derivada de um segredo do ambiente para proteger a senha em disco. */ function
secretKey() { var base = process.env.NFE_CERT_KEK || process.env.API_KEY || process.env.SIEG_CLIENT_SECRET ||
'ajl-nfe-local-kek'; return crypto.createHash('sha256').update(String(base)).digest(); } function encryptSenha(senha)
{ var iv = crypto.randomBytes(12); var c = crypto.createCipheriv('aes-256-gcm', secretKey(), iv); var enc =
Buffer.concat([c.update(String(senha), 'utf8'), c.final()]); return Buffer.concat([iv, c.getAuthTag(),
enc]).toString('base64'); } function decryptSenha(blob) { var buf = Buffer.from(String(blob), 'base64'); var iv =
buf.slice(0, 12); var tag = buf.slice(12, 28); var d = crypto.createDecipheriv('aes-256-gcm', secretKey(), iv);
d.setAuthTag(tag); return Buffer.concat([d.update(buf.slice(28)), d.final()]).toString('utf8'); } function onlyNum(s)
{ return String(s || '').replace(/\D/g, ''); } /** Monta as infos publicas a partir do certificado do titular em PEM.
*/ function infoFromCertPem(certPem) { var leaf = forge.pki.certificateFromPem(certPem); var cn = ''; try { cn =
(leaf.subject.getField('CN') || {}).value || ''; } catch (e) { cn = ''; } var cnpj = onlyNum((cn.split(':')[1] ||
'')); if (cnpj.length !== 14) { var m = cn.match(/(\d{14})/); cnpj = m ? m[1] : ''; } return { titular:
cn.split(':')[0] || cn, cnpj: cnpj, emissor: (function () { try { return (leaf.issuer.getField('CN') || {}).value ||
''; } catch (e) { return ''; } })(), validoDe: leaf.validity.notBefore.toISOString(), validoAte:
leaf.validity.notAfter.toISOString(), diasRestantes: Math.floor((leaf.validity.notAfter.getTime() - Date.now()) /
86400000), expirado: leaf.validity.notAfter.getTime() < Date.now(), serial: leaf.serialNumber,

}; } /** Leitura via node-forge (PFX classicos: 3DES/RC2 com provider disponivel). */ function
openPfxForge(pfxBuffer, senha) { var der = forge.util.createBuffer(pfxBuffer.toString('binary')); var asn1 =
forge.asn1.fromDer(der); var p12 = forge.pkcs12.pkcs12FromAsn1(asn1, false, String(senha)); var keyBags =
p12.getBags({ bagType: forge.pki.oids.pkcs8ShroudedKeyBag })[forge.pki.oids.pkcs8ShroudedKeyBag] || []; if
(!keyBags.length) { keyBags = p12.getBags({ bagType: forge.pki.oids.keyBag })[forge.pki.oids.keyBag] || []; } if
(!keyBags.length || !keyBags[0].key) throw new Error('Chave privada nao encontrada no certificado A1.'); var
privateKeyPem = forge.pki.privateKeyToPem(keyBags[0].key); var certBags = p12.getBags({ bagType:
forge.pki.oids.certBag })[forge.pki.oids.certBag] || []; if (!certBags.length) throw new Error('Certificado nao
encontrado no arquivo .pfx.'); var leaf = null; var chainPem = []; for (var i = 0; i < certBags.length; i++) { var c =
certBags[i].cert; if (!c) continue; chainPem.push(forge.pki.certificateToPem(c)); var isCa = false; try { var bc =
c.getExtension('basicConstraints'); isCa = !!(bc && bc.cA); } catch (e) { isCa = false; } if (!isCa && !leaf) leaf =
c; } if (!leaf) leaf = certBags[0].cert; var certPem = forge.pki.certificateToPem(leaf); return { privateKeyPem:
privateKeyPem, certPem: certPem, chainPem: chainPem, info: infoFromCertPem(certPem) }; } /** * Abre o PKCS#12, valida
a senha e extrai chave privada + cadeia em PEM. * Tenta o node-forge e, quando o PFX usa algoritmos que ele nao
suporta * (PBES2/AES-256 dos certificados ICP-Brasil recentes -> "Unsupported PKCS12 * PFX data"), cai para o OpenSSL
do sistema. */ function openPfx(pfxBuffer, senha) { if (!Buffer.isBuffer(pfxBuffer) || !pfxBuffer.length) throw new
Error('Arquivo de certificado invalido.'); var forgeErr = null; try { return openPfxForge(pfxBuffer, senha); } catch
(e) { forgeErr = e; var msg = String((e && e.message) || e); if (/mac|password|senha/i.test(msg) &&
!/unsupported/i.test(msg)) { throw new Error('Senha do certificado incorreta ou arquivo .pfx invalido.'); }
console.warn('[NFE-CERT] node-forge nao leu o PFX (' + msg + '). Tentando via OpenSSL...'); } var viaSsl; try {
viaSsl = openPfxWithOpenssl(pfxBuffer, senha); } catch (e2) { throw new Error(String((e2 && e2.message) || e2) + '
(node-forge: ' + String((forgeErr && forgeErr.message) || forgeErr) + ')'); } console.log('[NFE-CERT] PFX lido via
OpenSSL' + (viaSsl.legacy ? ' (modo legacy)' : '') + '.'); return { privateKeyPem: viaSsl.privateKeyPem, certPem:
```

```
viaSsl.certPem, chainPem: viaSsl.chainPem, info: infoFromCertPem(viaSsl.certPem), }; } /** Salva o certificado (upload). Retorna as infos publicas do certificado. */ function salvarCertificado(pfxBuffer, senha) { if (!pfxBuffer || !pfxBuffer.length) throw new Error('Arquivo .pfx vazio.'); if (!senha) throw new Error('Senha do certificado obrigatoria.'); var aberto = openPfx(pfxBuffer, senha); // valida senha antes de persistir

var dir = ensureDir(); fs.writeFileSync(path.join(dir, PFX_FILE), pfxBuffer, { mode: 0o600 }); fs.writeFileSync( path.join(dir, META_FILE), JSON.stringify({ senha: encryptSenha(senha), info: aberto.info, uploadEm: new Date().toISOString() }), { mode: 0o600 } ); cache = { pfx: pfxBuffer, senha: String(senha), privateKeyPem: aberto.privateKeyPem, certPem: aberto.certPem, chainPem: aberto.chainPem, info: aberto.info, }; console.log('[NFE-CERT] Certificado A1 armazenado. Titular: ' + aberto.info.titular + ' | CNPJ: ' + aberto.info.cnpj + ' | valido ate ' + aberto.info.validoAte); return aberto.info; } /** Carrega o certificado do cache, do disco ou das variaveis de ambiente. */ function carregarCertificado() { if (cache) return cache; // 1) Variaveis de ambiente (PRIORIDADE — sobrevive a deploys/reinicios no Render) var b64 = process.env.NFE_CERT_PFX_BASE64; var envSenha = process.env.NFE_CERT_SENHA; if (b64 && envSenha) { try { console.log('[NFE-CERT] Carregando certificado das variaveis de ambiente (NFE_CERT_PFX_BASE64=' + b64.slice(0, 20) + '...)'); var buf = Buffer.from(b64, 'base64'); var ab = openPfx(buf, envSenha); cache = { pfx: buf, senha: envSenha, privateKeyPem: ab.privateKeyPem, certPem: ab.certPem, chainPem: ab.chainPem, info: ab.info, }; console.log('[NFE-CERT] Certificado carregado via env vars. Titular: ' + ab.info.titular + ' | CNPJ: ' + ab.info.cnpj); return cache; } catch (e) { console.error('[NFE-CERT] Falha ao abrir certificado das env vars: ' + e.message); } } // 2) Disco (backup — pode ser perdido em reinicios no Render) try { var dir = ensureDir(); var pfxPath = path.join(dir, PFX_FILE); var metaPath = path.join(dir, META_FILE); console.log('[NFE-CERT] Verificando disco: dir=' + dir + ' pfx=' + fs.existsSync(pfxPath) + ' meta=' + fs.existsSync(metaPath)); if (fs.existsSync(pfxPath) && fs.existsSync(metaPath)) { var meta = JSON.parse(fs.readFileSync(metaPath, 'utf8')); var pfx = fs.readFileSync(pfxPath); var senha = decryptSenha(meta.senha); var aberto = openPfx(pfx, senha); cache = { pfx: pfx, senha: senha, privateKeyPem: aberto.privateKeyPem, certPem: aberto.certPem, chainPem: aberto.chainPem, info: aberto.info, }; console.log('[NFE-CERT] Certificado carregado do disco. Titular: ' + aberto.info.titular); return cache; } } catch (e) { console.error('[NFE-CERT] Falha ao ler certificado do disco: ' + e.message); } console.warn('[NFE-CERT] Nenhum certificado encontrado. Defina NFE_CERT_PFX_BASE64 + NFE_CERT_SENHA nas env vars (recomendado) ou envie via POST /api/v1/nfe/certificado.'); return null; } function statusCertificado() { var c; try { c = carregarCertificado(); } catch (e) { return { configurado: false, erro: e.message }; } if (!c) return { configurado: false, mensagem: 'Nenhum certificado A1 carregado. Envie via POST /api/v1/nfe/certificado.' }; return Object.assign({ configurado: true }, c.info); }

function removerCertificado() { var dir = ensureDir(); [PFX_FILE, META_FILE].forEach(function (f) { try { fs.unlinkSync(path.join(dir, f)); } catch (e) { /* ignore */ } }); cache = null; return true; } module.exports = { salvarCertificado, carregarCertificado, statusCertificado, removerCertificado, };
```

2.2 services/pfx-openssl.js

Fallback de leitura de PKCS#12 via OpenSSL CLI. Necessario quando o node-forge nao suporta o algoritmo do PFX (PBES2/AES-256 dos certificados ICP-Brasil recentes). Usa env:VAR para a senha (nunca na linha de comando), apaga o .pfx temporario apos o uso, e tenta modo -legacy se o moderno falhar.

pfx-openssl.js (/home/z/my-project/API-Odoo/services/pfx-openssl.js)

```
/** * services/pfx-openssl.js — Fallback de leitura de PKCS#12 via OpenSSL CLI * ========================================================================== * O node-forge nao suporta PFX gerados com PBES2/AES-256 (padrao das ACs * ICP-Brasil desde o OpenSSL 3) nem os antigos RC2-40 quando o runtime usa * OpenSSL 3 sem provider legacy. Nesses casos ele lanca: * "Unsupported PKCS12 PFX data" / "Unsupported PKCS#12 ..." * * Aqui usamos o binario `openssl` (presente na imagem Docker) para extrair * chave privada e certificados em PEM. Tentamos primeiro o modo moderno e, * se falhar, com -legacy (algoritmos antigos). * * A senha NUNCA vai na linha de comando (usamos env:VAR) e o .pfx temporario * eh gravado com modo 0600 e apagado ao final. */ var fs = require('fs'); var os = require('os'); var path = require('path'); var crypto = require('crypto'); var execFileSync = require('child_process').execFileSync; function run(args, senha) { return execFileSync('openssl', args, { encoding: 'utf8', maxBuffer: 20 * 1024 * 1024, env: Object.assign({}, process.env, { NFE_PFX_PW: String(senha) }), stdio: ['ignore', 'pipe', 'pipe'], }); } function extract(pfxPath, senha, mode, legacy) { var args = ['pkcs12', '-in', pfxPath, '-passin', 'env:NFE_PFX_PW']; if (mode === 'key') args = args.concat(['-nocerts', '-nodes']); if (mode === 'leaf') args = args.concat(['-clcerts', '-nokeys']); if (mode === 'ca') args = args.concat(['-cacerts', '-nokeys']); if (legacy) args.push('-legacy'); return run(args, senha); } function pemBlocks(text, tag) { var re = new RegExp('-----BEGIN ' + tag + '-----[\\s\\S]*?-----END ' + tag + '-----', 'g'); return String(text || '').match(re) || []; } function normalizeKey(pem) { var b = pemBlocks(pem, 'PRIVATE KEY')[0] || pemBlocks(pem, 'RSA PRIVATE KEY')[0] || pemBlocks(pem, 'ENCRYPTED PRIVATE KEY')[0]; return b ? b + '\n' : ''; } /** * Extrai { privateKeyPem, certPem, chainPem[] } do PFX usando o OpenSSL CLI. * Lanca erro descritivo quando a senha esta incorreta. */ function openPfxWithOpenssl(pfxBuffer, senha) { var tmp = path.join(os.tmpdir(), 'nfe-' + crypto.randomBytes(8).toString('hex') + '.pfx'); fs.writeFileSync(tmp, pfxBuffer, { mode: 0o600 }); try { var lastErr = null; var modos = [false, true]; // moderno, depois -legacy for (var i = 0; i < modos.length; i++) { try { var keyPem = normalizeKey(extract(tmp, senha, 'key', modos[i])); var leafOut = extract(tmp, senha, 'leaf', modos[i]); var certPem = pemBlocks(leafOut, 'CERTIFICATE')[0]; var chain = []; try { chain = pemBlocks(extract(tmp, senha, 'ca', modos[i]), 'CERTIFICATE'); } catch (e) { chain = []; } if (!certPem) { // alguns PFX trazem tudo como "cacert"; usa o primeiro disponivel certPem = chain[0]; } if (!keyPem) throw new Error('Chave privada nao encontrada no arquivo .pfx.'); if (!certPem) throw new Error('Certificado nao encontrado no arquivo .pfx.'); var chainPem = [certPem].concat(chain.filter(function (c) { return c !== certPem; })); return { privateKeyPem: keyPem, certPem: certPem + '\n', chainPem: chainPem, legacy: modos[i] }; } catch (e) { lastErr = e; var out = String((e && e.stderr) || (e && e.message) || ''); if (/mac verify failure|invalid password|wrong password/i.test(out)) {

throw new Error('Senha do certificado incorreta.'); } } } var det = String((lastErr && lastErr.stderr) || (lastErr && lastErr.message) || 'desconhecido').trim(); if (/ENOENT/.test(det)) { throw new Error('OpenSSL nao disponivel no
```

```
servidor para ler este certificado.'); } throw new Error('Falha ao abrir o certificado com OpenSSL: ' +
det.split('\n').slice(-3).join(' ')); } finally { try { fs.unlinkSync(tmp); } catch (e) { /* ignore */ } } } function
opensslDisponivel() { try { execFileSync('openssl', ['version'], { stdio: ['ignore', 'pipe', 'pipe'] }); return true;
} catch (e) { return false; } } module.exports = { openPfxWithOpenssl: openPfxWithOpenssl, opensslDisponivel:
opensslDisponivel };
```

3. Gerador de XML NF-e 4.00

O arquivo sieg-nfe-xml.js e o coracao da geracao do XML da NF-e. Ele recebe os dados extraidos do Odoo (empresa, parceiro, linhas de fatura, impostos, pagamentos) e monta o XML completo no formato NF-e 4.00, incluindo os grupos ide (identificacao), emit (emitente), dest (destinatario), det (itens com prod + impostos), total (ICMSTot), transp (transporte), pag (pagamentos - PIX) e infAdic (informacoes complementares). O XML gerado segue o padrao exigido pela SEFAZ e inclui tratamentos especificos como CSOSN 103 (simples nacional sem ICMS), CFOP 6102 (venda fora do estado) e formatacao de enderecos do Odoo (que frequentemente inclui o numero no campo street).

services/sieg-nfe-xml.js

Gerador de XML NF-e v4.00 a partir de dados Odoo. Monta com todos os grupos obrigatorios (ide, emit, dest, det, total, transp, pag, infAdic). Inclui CSOSN 103, CFOP 6102, ICMS-ST com MVA, e pagamentos via PIX (tPag=15).

sieg-nfe-xml.js (/home/z/my-project/API-Odoo/services/sieg-nfe-xml.js)

```
/** * services/sieg-nfe-xml.js — Gerador de XML NF-e v4.00 a partir de dados Odoo * * Gera o <NFe> para envio ao SIEG
via POST /api/v1/send-xml * O SIEG adiciona: XMLDSig (assinatura) + envia à SEFAZ + retorna nfeProc * * Baseado no XML
real emitido pela AJL via SIEG (29/07/2026) */ const NFE_NS = 'http://www.portalfiscal.inf.br/nfe'; /** * Extrai
numero do logradouro quando o campo 'number' esta vazio/S/N. * Odoo frequentemente armazena o endereco completo no
campo 'street': * "Avenida Juscelino Kubitschek de Oliveira, 7525" * "Rua Bom Jesus, 212" * "Rua Augusta, 1200 Sala
53" * Retorna { street, number } */ function parseStreetNumber(street, number) { if (!street) return { street: '',
number: number || 'S/N' }; // Se ja tem numero valido, retorna como esta if (number && number !== 'S/N' &&
String(number).trim() !== '') { return { street: street, number: String(number) }; } // Padrão 1: "Logradouro, NNNN"
(virgula + espaco + numero) var m = street.match(/^(.+?),\s*(\d+[\w]?(?:\s*[A-Za-zÀ-ÿ]+)?)\s*$/); if (m) return {
street: m[1].trim(), number: m[2].trim() }; // Padrao 2: "Logradouro, NNNN complemento" (virgula + numero +
complemento) m = street.match(/^(.+?),\s*(\d+)\s+(.+)$/); if (m) return { street: m[1].trim(), number: m[2].trim() };
// Padrao 3: "Logradouro NNNN" (espaco + numero no final, sem virgula) m = street.match(/^(.+?)\s+(\d+)\s*$/); if (m)
return { street: m[1].trim(), number: m[2].trim() }; // Nenhum padrao encontrado return { street: street, number:
number || 'S/N' }; } /** * Preenche campos de endereço comuns (emit/dest) */ function xmlEndereco(end, tagPrefix) {
if (!end) return ''; // Separar logradouro e numero var parsed = parseStreetNumber(end.street || end.xLgr || '',
end.number || end.nro); const parts = []; parts.push(` <${tagPrefix}>`); parts.push(`
<xLgr>${esc(parsed.street)}</xLgr>`); parts.push(` <nro>${esc(parsed.number)}</nro>`); if (end.street2 ||
end.xBairro) { parts.push(` <xBairro>${esc(end.street2 || end.xBairro || '')}</xBairro>`); } parts.push(`
<cMun>${esc(String(end.city_ibge_code || end.cMun || ''))}</cMun>`); parts.push(` <xMun>${esc(end.city || end.xMun ||
'')}</xMun>`); parts.push(` <UF>${esc(end.state || end.UF || '')}</UF>`); parts.push(` <CEP>${cepFmt(end.zip ||
end.CEP || '')}</CEP>`); parts.push(` <cPais>1058</cPais>`); parts.push(` <xPais>Brasil</xPais>`); if (end.phone ||
end.fone) { parts.push(` <fone>${foneFmt(end.phone || end.fone || '')}</fone>`); } parts.push(` </${tagPrefix}>`);
return parts.join('\n'); } /** * Gera bloco de impostos por item — dinâmico baseado nos dados reais do Odoo * Suporta:
ICMS (Simples Nacional CSOSN / Regime Normal CST), PIS, COFINS */ function xmlImpostoItem(line, crt, ibsInfo) { const
csosn = line.csosn || ''; const cstIcms = line.cst_icms || ''; const orig = line.orig || '0'; const modBC =
line.mod_bc || ''; const vBC = num(line.vbc_icms || line.vbc || '0.00'); const vICMS = num(line.vicms || '0.00');
const pICMS = num(line.picms || '0.00'); const cstPis = line.cst_pis || '01'; const baseItem = line.vbc_icms ||
line.vbc || line.price_subtotal || 0; const vBCPis = num(line.vbc_pis || baseItem || '0.00');

const pPis = num(line.ppis || line.pis_aliquota || '0.00'); const vPIS = num(line.vpis || '0.00'); const cstCof =
line.cst_cofins || '01'; const vBCCof = num(line.vbc_cofins || baseItem || '0.00'); const pCofins = num(line.pcofins
|| line.cofins_aliquota || '0.00'); const vCOFINS = num(line.vcofins || '0.00'); // --- ICMS --- // Mapeamento CSOSN
→ Grupo XSD (NT 2024.001 / NF-e 4.00): // CSOSN 101 → ICMSSN101 // CSOSN 102,103,300,400 → ICMSSN102 // CSOSN 201 →
ICMSSN201 // CSOSN 202,203 → ICMSSN202 // CSOSN 500 → ICMSSN500 // CSOSN 900 → ICMSSN900 let icmsBlock = ''; if
(csosn) { const csosnNum = csosn.replace(/\D/g, ''); let tag = 'ICMSSN102'; // fallback if (csosnNum === '101') { tag
= 'ICMSSN101'; } else if (['102','103','300','400'].includes(csosnNum)) { tag = 'ICMSSN102'; } else if (csosnNum ===
'201') { tag = 'ICMSSN201'; } else if (['202','203'].includes(csosnNum)) { tag = 'ICMSSN202'; } else if (csosnNum ===
'500') { tag = 'ICMSSN500'; } else if (csosnNum === '900') { tag = 'ICMSSN900'; } if (csosnNum === '900') { icmsBlock
= `<ICMS><${tag}><orig>${orig}</orig><CSOSN>${csosn}</CSOSN><vICMS>${vICMS}</vICMS><vBC>${vBC}</vBC><pICMS>${pICMS}</
pICMS></${tag}></ICMS>`; } else { icmsBlock =
`<ICMS><${tag}><orig>${orig}</orig><CSOSN>${csosn}</CSOSN></${tag}></ICMS>`; } } else if (cstIcms) { // Regime
Normal: CST 00-90 const cstNum = String(cstIcms).replace(/\D/g, ''); let tag = 'ICMS' + cstNum; if
(['00','10','20','30','40','41','50','51','60','70','90'].includes(cstNum)) { tag = 'ICMS' + cstNum; } else { tag =
'ICMS00'; } icmsBlock = `<ICMS><${tag}><orig>${orig}</orig><CST>${cstIcms}</CST><modBC>${modBC ||
'0'}</modBC><vBC>${vBC}</vBC><pICMS>${pICMS}</pICMS><vICMS>${vICMS}</vICMS></${tag}></ICMS>`; } else { // Fallback
CSOSN 103 icmsBlock = `<ICMS><ICMSSN102><orig>${orig}</orig><CSOSN>103</CSOSN></ICMSSN102></ICMS>`; } // --- PIS ---
// PISAliq: CST 01,02,03 | PISNT: CST 04,05,06,07,08,09 | PISOutr: CST 49,50,51,52,53,54,55,56,60-75,98,99 let
```

```
pisBlock = ''; const pisCstNum = String(cstPis).replace(/\D/g, ''); if
(['04','05','06','07','08','09'].includes(pisCstNum)) { pisBlock = `<PIS><PISNT><CST>${cstPis}</CST></PISNT></PIS>`;
} else if (['01','02','03'].includes(pisCstNum)) { pisBlock =
`<PIS><PISAliq><CST>${cstPis}</CST><vBC>${vBCPis}</vBC><pPIS>${pPis}</pPIS><vPIS>${vPIS}</vPIS></PISAliq></PIS>`; }
else { pisBlock =
`<PIS><PISOutr><CST>${cstPis}</CST><vBC>${vBCPis}</vBC><pPIS>${pPis}</pPIS><vPIS>${vPIS}</vPIS></PISOutr></PIS>`; }
// --- COFINS --- // COFINSAliq: CST 01,02,03 | COFINSNT: CST 04,05,06,07,08,09 | COFINSOutr: CST
49,50,51,52,53,54,55,56,60-75,98,99 let cofinsBlock = ''; const cofCstNum = String(cstCof).replace(/\D/g, ''); if
(['04','05','06','07','08','09'].includes(cofCstNum)) { cofinsBlock =
`<COFINS><COFINSNT><CST>${cstCof}</CST></COFINSNT></COFINS>`; } else if (['01','02','03'].includes(cofCstNum)) {
cofinsBlock = `<COFINS><COFINSAliq><CST>${cstCof}</CST><vBC>${vBCCof}</vBC><pCOFINS>${pCofins}</pCOFINS><vCOFINS>${vC
OFINS}</vCOFINS></COFINSAliq></COFINS>`; } else { cofinsBlock = `<COFINS><COFINSOutr><CST>${cstCof}</CST><vBC>${vBCCo
f}</vBC><pCOFINS>${pCofins}</pCOFINS><vCOFINS>${vCOFINS}</vCOFINS></COFINSOutr></COFINS>`; } // --- IPI --- // Emitir
IPI somente quando o Odoo realmente fornecer o CST do item. let ipiBlock = '';

if (String(crt || '') === '3' && line.cst_ipi) { const cstIpi = String(line.cst_ipi).replace(/\D/g, '').padStart(2,
'0'); const vBCIpi = num(line.vbc_ipi || '0.00'); const pIpi = num4(line.pipi || '0.00'); const vIpi = num(line.vipi
|| '0.00'); const cEnq = line.cenq || '999'; if (['51','52','53','54','55'].includes(cstIpi)) { ipiBlock =
`<IPI><cEnq>${cEnq}</cEnq><IPINT><CST>${cstIpi}</CST></IPINT></IPI>`; } else if
(['00','49','50','99'].includes(cstIpi)) { ipiBlock = `<IPI><cEnq>${cEnq}</cEnq><IPITrib><CST>${cstIpi}</CST><vBC>${v
BCIpi}</vBC><pIPI>${pIpi}</pIPI><vIPI>${vIpi}</vIPI></IPITrib></IPI>`; } } // --- IBS/CBS (Reforma Tributaria -
obrigatorio no layout vigente 2026) --- let ibsBlock = ''; if (ibsInfo) { ibsBlock =
`<IBSCBS><CST>${ibsInfo.cst}</CST><cClassTrib>${ibsInfo.cClassTrib}</cClassTrib>` +
`<gIBSCBS><vBC>${ibsInfo.vBC}</vBC>` +
`<gIBSUF><pIBSUF>${ibsInfo.pIBSUF}</pIBSUF><vIBSUF>${ibsInfo.vIBSUF}</vIBSUF></gIBSUF>` +
`<gIBSMun><pIBSMun>${ibsInfo.pIBSMun}</pIBSMun><vIBSMun>${ibsInfo.vIBSMun}</vIBSMun></gIBSMun>` +
`<vIBS>${ibsInfo.vIBS}</vIBS>` + `<gCBS><pCBS>${ibsInfo.pCBS}</pCBS><vCBS>${ibsInfo.vCBS}</vCBS></gCBS>` +
`</gIBSCBS></IBSCBS>`; } // NT2024/004 (IBSCBS): o schema PR-v4_9_86 nao aceita <vTrib> como filho de <imposto> //
quando IBSCBS esta presente. A informacao de tributos flui pelo proprio IBS/CBS. // Ordem exigida pelo XSD: ICMS →
IPI → PIS → COFINS → IBSCBS return `<imposto>${icmsBlock}${ipiBlock}${pisBlock}${cofinsBlock}${ibsBlock}</imposto>`;
} /** * Calcula IBS/CBS de um item conforme o padrao do XML real da AJL: * vBC = vProd - ICMS - PIS - COFINS (tributos
"por dentro" excluidos) * vIBSUF = vBC * pIBSUF% | vIBSMun = vBC * pIBSMun% | vCBS = vBC * pCBS% */ function
calcIbsCbsItem(line, vProdNum, cfg) { const pIBSUF = parseFloat(cfg.pIBSUF != null ? cfg.pIBSUF :
(process.env.NFE_P_IBS_UF || '0.10')); const pIBSMun = parseFloat(cfg.pIBSMun != null ? cfg.pIBSMun :
(process.env.NFE_P_IBS_MUN || '0.00')); const pCBS = parseFloat(cfg.pCBS != null ? cfg.pCBS : (process.env.NFE_P_CBS
|| '0.90')); const vICMS = parseFloat(line.vicms || 0) || 0; const vPIS = parseFloat(line.vpis || 0) || 0; const
vCOFINS = parseFloat(line.vcofins || 0) || 0; let base = vProdNum - vICMS - vPIS - vCOFINS; if (!(base > 0)) base =
vProdNum; const vIBSUF = round2(base * pIBSUF / 100); const vIBSMun = round2(base * pIBSMun / 100); const vCBS =
round2(base * pCBS / 100); return { cst: line.cst_ibscbs || cfg.cstIBSCBS || '000', cClassTrib: line.cclass_trib ||
cfg.cClassTrib || '000001', vBCNum: round2(base), vBC: num(base), pIBSUF: num4(pIBSUF), vIBSUF: num(vIBSUF),
vIBSUFNum: vIBSUF, pIBSMun: num4(pIBSMun), vIBSMun: num(vIBSMun), vIBSMunNum: vIBSMun, vIBS: num(vIBSUF + vIBSMun),
vIBSNum: round2(vIBSUF + vIBSMun), pCBS: num4(pCBS), vCBS: num(vCBS), vCBSNum: vCBS, }; } function round2(n) { return
Math.round((parseFloat(n) || 0) * 100) / 100; } /** * Gera XML NF-e completo a partir dos dados extraidos do Odoo * *
@param {Object} data - Dados do Odoo (sale.order + company + partner + lines) * @param {Object} data.company - Dados
da empresa (emitente) * @param {Object} data.partner - Dados do cliente (destinatario) * @param {Object} data.order -
Dados do pedido * @param {Array} data.lines - Linhas do pedido (produtos) * @param {Array} [data.duplicatas] -
Parcelas de cobranca * @param {Array} [data.pagamentos] - Pagamentos recebidos * @param {Object} [data.config] -
Configuracoes adicionais * @returns {string} XML string do <NFe> */ function gerarXmlNFe(data) { const { company,
partner, order, lines, config: cfg = {} } = data; const dup = data.duplicatas || []; const pag = data.pagamentos ||
[];

// === Validacao de campos obrigatorios === const cUF = company.state_ibge || '41'; const cNF = randomCnf(); const
serie = cfg.serie || '100'; const nNF = String(order.number || order.name || '1').replace(/\D/g, ''); const dhEmi =
formatDh(order.date_order || new Date().toISOString()); const tpNF = '1'; // saida const idDest =
calcIdDest(company.state || company.UF, partner.state || partner.UF); const cMunFG = company.city_ibge_code || '';
const tpAmb = cfg.tpAmb || process.env.SIEG_TP_AMB || '2'; // 2=homologacao (padrao seguro), 1=producao — defina
SIEG_TP_AMB=1 para producao const finNFe = cfg.finNFe || '1'; // 1=normal // indFinal: '1' quando consumidor final OU
quando nao contribuinte (indIEDest=9) // Regra SEFAZ cStat 696: operacao com nao contribuinte DEVE ter indFinal=1
const _indIEDestCalc = calcIndIEDest(partner); const indFinal = (partner.is_consumer || partner.indFinal ||
_indIEDestCalc === '9') ? '1' : '0'; const indPres = cfg.indPres || process.env.NFE_IND_PRES || '9'; // 9=operacao
nao presencial, outros // indIntermed obrigatorio quando indPres = 2, 3, 4 ou 9 (NT2015.003 / cStat 434 SEFAZ PR)
const indIntermed = ['2','3','4','9'].includes(String(indPres)) ? (cfg.indIntermed || '0') : null; const ibsCbsAtivo
= String(cfg.ibsCbs != null ? cfg.ibsCbs : (process.env.NFE_IBSCBS || 'true')) !== 'false'; const verProc =
cfg.verProc || 'Odoo19-SIEG-1.0'; // === VALIDACAO DETALHADA POR CAMPO === var xmlErrors = []; var xmlWarnings = [];
console.log('[NFE-XML] ========== VALIDACAO CAMPOS XML =========='); // --- Emitente --- console.log('[NFE-XML]
[EMITENTE]'); logField('cUF', cUF); logField('CNPJ', onlyNum(company.cnpj_cpf), !onlyNum(company.cnpj_cpf));
logField('xNome', company.legal_name || company.xNome, !(company.legal_name || company.xNome)); logField('xFant',
company.name || company.xFant); logField('xLgr', company.street, !company.street); logField('nro', company.number ||
'S/N'); logField('xBairro', company.street2, !company.street2); logField('cMun', company.city_ibge_code,
!company.city_ibge_code); logField('xMun', company.city, !company.city); logField('UF', company.state,
!company.state); logField('CEP', company.zip, !company.zip); logField('fone', company.phone); logField('IE',
company.inscr_est, !company.inscr_est); logField('CRT', company.crt || '1'); if (!onlyNum(company.cnpj_cpf))
xmlErrors.push('CNPJ emitente vazio'); if (!company.inscr_est) xmlErrors.push('IE emitente vazia'); if
(!company.street) xmlErrors.push('Logradouro emitente vazio'); if (!company.city_ibge_code)
xmlErrors.push('cMunFG/cMun emitente vazio'); if (!company.city) xmlWarnings.push('xMun emitente vazio'); if
(!company.state) xmlErrors.push('UF emitente vazia'); // --- Destinatario --- console.log('[NFE-XML]
[DESTINATARIO]'); var docDest2 = onlyNum(partner.cnpj_cpf || ''); logField('CNPJ/CPF', docDest2, !docDest2);
logField('xNome', partner.legal_name || partner.xNome, !(partner.legal_name || partner.xNome)); logField('xLgr',
partner.street, !partner.street); logField('nro', partner.number || 'S/N'); logField('xBairro', partner.street2,
!partner.street2); logField('cMun', partner.city_ibge_code, !partner.city_ibge_code); logField('xMun', partner.city,
!partner.city); logField('UF', partner.state, !partner.state); logField('CEP', partner.zip, !partner.zip);
logField('fone', partner.phone); logField('email', partner.email); if (!docDest2) xmlErrors.push('CNPJ/CPF
destinatario vazio'); if (!(partner.legal_name || partner.xNome)) xmlErrors.push('Nome destinatario vazio'); if
```

```
(!partner.street) xmlErrors.push('Logradouro destinatario vazio'); if (!partner.city_ibge_code) xmlErrors.push('cMun destinatario vazio'); if (!partner.city) xmlWarnings.push('xMun destinatario vazio'); if (!partner.state) xmlErrors.push('UF destinatario vazia'); // --- IDE --- console.log('[NFE-XML] [IDE]'); logField('cMunFG', cMunFG, !cMunFG); logField('natOp', cfg.natOp || 'Venda de Mercadoria'); logField('serie', serie); logField('nNF', nNF); logField('dhEmi', dhEmi);

logField('tpAmb', tpAmb); logField('mod', cfg.mod || '55'); if (!cMunFG) xmlErrors.push('cMunFG vazio (empresa sem codigo IBGE da cidade)'); // --- Itens (linhas) --- lines.forEach(function(l, i) { console.log('[NFE-XML] [ITEM ' + (i+1) + ']'); logField(' cProd', l.cProd || l.default_code, !(l.cProd || l.default_code)); logField(' xProd', l.xProd || l.product_name); logField(' NCM', l.ncm || l.NCM, !(l.ncm || l.NCM)); logField(' CFOP', l.cfop || '5102'); logField(' uCom', l.uom || 'UN'); logField(' qCom', l.qty); logField(' vUnCom', l.price_unit); logField(' vProd', l.price_subtotal || (l.qty * l.price_unit)); logField(' CSOSN', l.csosn || ''); logField(' CST_ICMS', l.cst_icms || '(vazio - usara CSOSN)'); logField(' vICMS', l.vicms || '0.00'); logField(' CST_PIS', l.cst_pis || '01'); logField(' vPIS', l.vpis || '0.00'); logField(' CST_COFINS', l.cst_cofins || '01'); logField(' vCOFINS', l.vcofins || '0.00'); if (!(l.ncm || l.NCM)) xmlErrors.push('NCM vazio no item ' + (i+1) + ' (' + (l.xProd || l.product_name || '?') + ')'); if (!(l.cProd || l.default_code)) xmlWarnings.push('cProd vazio no item ' + (i+1) + ' (codigo interno do produto)'); }); // --- Totais / Pagamento --- // Calcular vProdTotal antes da validacao (usado nos logs) var vProdTotal = 0; for (var vi = 0; vi < lines.length; vi++) { vProdTotal += parseFloat(lines[vi].price_subtotal || (lines[vi].qty * lines[vi].price_unit)); } console.log('[NFE-XML] [TOTAIS]'); logField('vNF', order.amount_total || vProdTotal); logField('vProd', vProdTotal); if (pag.length > 0) { logField('tPag', pag[0].tPag || '15'); logField('vPag', pag[0].vPag); } else { xmlWarnings.push('Nenhum pagamento informado (bloco <pag> vazio)'); } // --- Resumo da validacao --- console.log('[NFE-XML] ========== RESUMO VALIDACAO =========='); if (xmlErrors.length > 0) { console.error('[NFE-XML] *** ERROS (' + xmlErrors.length + ') — XML provavelmente sera REJEITADO ***'); for (var e = 0; e < xmlErrors.length; e++) { console.error('[NFE-XML] ERRO: ' + xmlErrors[e]); } } if (xmlWarnings.length > 0) { console.warn('[NFE-XML] *** AVISOS (' + xmlWarnings.length + ') ***'); for (var w = 0; w < xmlWarnings.length; w++) { console.warn('[NFE-XML] AVISO: ' + xmlWarnings[w]); } } if (xmlErrors.length === 0 && xmlWarnings.length === 0) { console.log('[NFE-XML] Todos os campos validados com sucesso!'); } console.log('[NFE-XML] ========================================'); // Calcular cDV e adicionar Id na infNFe (obrigatorio pelo schema NF-e 4.00) var tpEmis = '1'; var accessKey = calcAccessKey(cUF, dhEmi, company.cnpj_cpf, '55', serie, nNF, tpEmis, cNF); var cDV = accessKey[43]; console.log('[NFE-XML] cDV calculado: ' + cDV + ' (chave sem DV: ' + accessKey.slice(0, 43) + ')'); let xml = `<?xml version="1.0" encoding="UTF-8"?> <NFe xmlns="${NFE_NS}"> <infNFe versao="4.00" Id="NFe${accessKey}"> <ide> <cUF>${cUF}</cUF> <cNF>${cNF}</cNF> <natOp>${esc(cfg.natOp || 'Venda de Mercadoria')}</natOp> <mod>55</mod> <serie>${serie}</serie> <nNF>${nNF}</nNF> <dhEmi>${dhEmi}</dhEmi>

<dhSaiEnt>${dhEmi}</dhSaiEnt> <tpNF>${tpNF}</tpNF> <idDest>${idDest}</idDest> <cMunFG>${cMunFG}</cMunFG>${ibsCbsAtivo ? ` <cMunFGIBS>${cMunFG}</cMunFGIBS>` : ''} <tpImp>1</tpImp> <tpEmis>${tpEmis}</tpEmis> <cDV>${cDV}</cDV> <tpAmb>${tpAmb}</tpAmb> <finNFe>${finNFe}</finNFe> <indFinal>${indFinal}</indFinal> <indPres>${indPres}</indPres>${indIntermed ? ` <indIntermed>${indIntermed}</indIntermed>` : ''} <procEmi>0</procEmi> <verProc>${esc(verProc)}</verProc> </ide>`; // === emit === xml += ` <emit> <CNPJ>${onlyNum(company.cnpj_cpf)}</CNPJ> <xNome>${esc(company.legal_name || company.xNome)}</xNome> <xFant>${esc(company.name || company.xFant || '')}</xFant> ${xmlEndereco(company, 'enderEmit')} <IE>${onlyNum(company.inscr_est || company.IE || '')}</IE> <CRT>${company.crt || '1'}</CRT> </emit>`; // === dest === const docDest = onlyNum(partner.cnpj_cpf || ''); const destTag = docDest.length === 14 ? 'CNPJ' : (docDest.length === 11 ? 'CPF' : ''); const indIEDest = calcIndIEDest(partner); xml += ` <dest> ${destTag ? `<${destTag}>${docDest}</${destTag}>` : '<CPF>00000000000</CPF>'} <xNome>${esc(partner.legal_name || partner.xNome || '')}</xNome> ${xmlEndereco(partner, 'enderDest')} <indIEDest>${indIEDest}</indIEDest>${partner.inscr_est ? ` <IE>${onlyNum(partner.inscr_est)}</IE>` : ''}${partner.email ? ` <email>${esc(partner.email)}</email>` : ''} </dest>`; // === det (items) === const ibsTot = { vBC: 0, vIBSUF: 0, vIBSMun: 0, vIBS: 0, vCBS: 0 }; lines.forEach((line, idx) => { const nItem = String(idx + 1); const vProdNum = round2(line.price_subtotal || (line.qty * line.price_unit)); const vProd = num(vProdNum); // vProdTotal ja foi calculado no bloco de validacao acima const ibsItem = ibsCbsAtivo ? calcIbsCbsItem(line, vProdNum, cfg) : null; if (ibsItem) { ibsTot.vBC += ibsItem.vBCNum; ibsTot.vIBSUF += ibsItem.vIBSUFNum; ibsTot.vIBSMun += ibsItem.vIBSMunNum; ibsTot.vIBS += ibsItem.vIBSNum; ibsTot.vCBS += ibsItem.vCBSNum; } xml += ` <det nItem="${nItem}"> <prod> <cProd>${esc(String(line.default_code || line.cProd || ''))}</cProd> <cEAN>${line.barcode || 'SEM GTIN'}</cEAN> <xProd>${esc(line.product_name || line.xProd || '')}</xProd> <NCM>${esc(String(line.ncm || line.NCM || ''))}</NCM> <CFOP>${esc(String(line.cfop || '5102'))}</CFOP> <uCom>${esc(line.uom || 'UN')}</uCom> <qCom>${num3(line.qty)}</qCom> <vUnCom>${num3(line.price_unit)}</vUnCom> <vProd>${vProd}</vProd> <cEANTrib>SEM GTIN</cEANTrib> <uTrib>${esc(line.uom || 'UN')}</uTrib> <qTrib>${num3(line.qty)}</qTrib> <vUnTrib>${num3(line.price_unit)}</vUnTrib> <indTot>1</indTot> </prod>${xmlImpostoItem(line, company.crt || '1', ibsItem)}${ibsItem ? ` <vItem>${vProd}</vItem>` : ''} </det>`; });

// === total (somar impostos reais das linhas) === let vBC_total = 0, vICMS_total = 0, vPIS_total = 0, vCOFINS_total = 0; lines.forEach(line => { vBC_total += parseFloat(line.vbc_icms || line.vbc || 0); vICMS_total += parseFloat(line.vicms || 0); vPIS_total += parseFloat(line.vpis || 0); vCOFINS_total += parseFloat(line.vcofins || 0); }); // vTotTrib: NT2024/004 com IBSCBS — a SEFAZ PR-v4_9_86 calcula como 0.00 quando nao ha // <vTrib> por item. Zeramos aqui para manter consistencia (0.00 = 0.00, sem cStat 685). var vTotTrib = 0; // (loop removido: vTrib por item nao e permitido no schema com IBSCBS) const vNF = num(order.amount_total || vProdTotal); xml += ` <total> <ICMSTot> <vBC>${num(vBC_total)}</vBC> <vICMS>${num(vICMS_total)}</vICMS> <vICMSDeson>0.00</vICMSDeson> <vFCP>0.00</vFCP> <vBCST>0.00</vBCST> <vST>0.00</vST> <vFCPST>0.00</vFCPST> <vFCPSTRet>0.00</vFCPSTRet> <vProd>${num(vProdTotal)}</vProd> <vFrete>0.00</vFrete> <vSeg>0.00</vSeg> <vDesc>0.00</vDesc> <vII>0.00</vII> <vIPI>0.00</vIPI> <vIPIDevol>0.00</vIPIDevol> <vPIS>${num(vPIS_total)}</vPIS> <vCOFINS>${num(vCOFINS_total)}</vCOFINS> <vOutro>0.00</vOutro> <vNF>${vNF}</vNF> <vTotTrib>${num(vTotTrib)}</vTotTrib> </ICMSTot>${ibsCbsAtivo ? ` <IBSCBSTot> <vBCIBSCBS>${num(ibsTot.vBC)}</vBCIBSCBS> <gIBS> <gIBSUF><vDif>0.00</vDif><vDevTrib>0.00</vDevTrib><vIBSUF>${num(ibsTot.vIBSUF)}</vIBSUF></gIBSUF> <gIBSMun><vDif>0.00</vDif><vDevTrib>0.00</vDevTrib><vIBSMun>${num(ibsTot.vIBSMun)}</vIBSMun></gIBSMun> <vIBS>${num(ibsTot.vIBS)}</vIBS> <vCredPres>0.00</vCredPres> <vCredPresCondSus>0.00</vCredPresCondSus> </gIBS> <gCBS> <vDif>0.00</vDif> <vDevTrib>0.00</vDevTrib> <vCBS>${num(ibsTot.vCBS)}</vCBS> <vCredPres>0.00</vCredPres> <vCredPresCondSus>0.00</vCredPresCondSus> </gCBS> </IBSCBSTot> <vNFTot>${num(round2(parseFloat(vNF) + ibsTot.vIBS + ibsTot.vCBS))}</vNFTot>` : ''} </total>`; // === transp === const modFrete = String(cfg.modFrete || '9'); const hasVolume = cfg.qVol != null && Number(cfg.qVol) > 0; xml += ` <transp> <modFrete>${modFrete}</modFrete>${hasVolume ? ` <vol> <qVol>${Math.trunc(Number(cfg.qVol))}</qVol> </vol>` : ''} </transp>`; // === cobr (duplicatas) === if (dup.length > 0) { xml += ` <cobr> <fat> <nFat>${esc(String(order.name || nNF))}</nFat> <vOrig>${vNF}</vOrig> <vDesc>0.00</vDesc> <vLiq>${vNF}</vLiq>
```

```
</fat>`; dup.forEach((d, i) => { xml += ` <dup> <nDup>${String(i + 1).padStart(3, '0')}</nDup> <dVenc>${d.dVenc ||
''}</dVenc> <vDup>${num(d.vDup)}</vDup> </dup>`; }); xml += `\n </cobr>`; } // === pag === if (pag.length > 0) { xml
+= `\n <pag>`; pag.forEach(p => { xml += ` <detPag> <tPag>${p.tPag || '15'}</tPag> <vPag>${num(p.vPag)}</vPag>
</detPag>`; }); xml += `\n </pag>`; } // === infAdic === var infCplText = stripHtml(order.note || order.infCpl ||
''); if (infCplText) { xml += `\n <infAdic> <infCpl>${esc(infCplText.substring(0, 2000))}</infCpl> </infAdic>`; } //
=== infRespTec (obrigatorio na NF-e 4.00 desde a NT 2018.005) === // O CNPJ informado DEVE estar cadastrado na SEFAZ
como responsavel tecnico do emitente. var respCnpj = onlyNum(cfg.respTecCnpj || process.env.NFE_RESP_TEC_CNPJ || '');
var respNome = cfg.respTecContato || process.env.NFE_RESP_TEC_CONTATO || ''; var respEmail = cfg.respTecEmail ||
process.env.NFE_RESP_TEC_EMAIL || ''; var respFone = onlyNum(cfg.respTecFone || process.env.NFE_RESP_TEC_FONE || '');
// idCSRT/hashCSRT: opcionais. Se NFE_RESP_TEC_TOKEN_CSRT estiver definido, // o hash é calculado dinamicamente:
base64(sha1(chave44 + token_csrt)) var respIdCSRT = cfg.respTecIdCSRT || process.env.NFE_RESP_TEC_ID_CSRT || ''; var
respTokenCSRT = process.env.NFE_RESP_TEC_TOKEN_CSRT || ''; var respHashCSRT = ''; if (respIdCSRT && respTokenCSRT) {
var crypto = require('crypto'); // chave de 44 digitos sem o digito verificador prefixo 'NFe' var chave44 =
String(accessKey).replace(/\D/g, '').slice(0, 44); respHashCSRT = crypto.createHash('sha1').update(chave44 +
respTokenCSRT).digest('base64'); } if (respCnpj && respNome && respEmail && respFone) { console.log('[NFE-XML]
infRespTec CNPJ=' + respCnpj + ' contato=' + respNome + (respIdCSRT ? ' idCSRT=' + respIdCSRT : ' (sem CSRT)')); xml
+= '\n <infRespTec>' + '\n <CNPJ>' + respCnpj + '</CNPJ>' + '\n <xContato>' + esc(respNome) + '</xContato>' + '\n
<email>' + esc(respEmail) + '</email>' + '\n <fone>' + respFone + '</fone>' + (respIdCSRT && respHashCSRT ? '\n
<idCSRT>' + respIdCSRT + '</idCSRT>' : '') + (respIdCSRT && respHashCSRT ? '\n <hashCSRT>' + respHashCSRT +
'</hashCSRT>' : '') + '\n </infRespTec>'; } else { console.error('[NFE-XML] *** infRespTec ausente — configure
NFE_RESP_TEC_CNPJ, NFE_RESP_TEC_CONTATO, NFE_RESP_TEC_EMAIL e NFE_RESP_TEC_FONE ***'); } xml += `\n
</infNFe>\n</NFe>`; return xml; } // === Helper functions === function stripHtml(s) { if (!s) return ''; return
String(s).replace(/<br\s*\/?>/gi, '\n').replace(/<[^>]+>/g, '').replace(/ /gi, ' ').replace(/&amp;/gi,
'&').replace(/&lt;/gi, '<').replace(/&gt;/gi, '>').replace(/&quot;/gi, '"').trim(); } function esc(s) { if (!s)
return ''; return String(s) .replace(/&/g, '&amp;') .replace(/</g, '&lt;')
```

```
.replace(/>/g, '&gt;') .replace(/"/g, '&quot;') .replace(/'/g, '&apos;'); } function num(v, decimals) { if (v ===
null || v === undefined) return '0.00'; const n = parseFloat(String(v).replace(',', '.')); if (isNaN(n)) return
decimals === 3 ? '0.000' : '0.00'; return n.toFixed(decimals || 2); } function num4(v) { const n =
parseFloat(String(v == null ? 0 : v).replace(',', '.')); return (isNaN(n) ? 0 : n).toFixed(4); } function num3(v) {
return num(v, 3); } function onlyNum(s) { return String(s || '').replace(/\D/g, ''); } function cepFmt(s) { return
onlyNum(s); } function foneFmt(s) { return onlyNum(s); } function randomCnf() { return String(Math.floor(10000000 +
Math.random() * 90000000)); } function formatDh(iso) { // 2026-07-29T07:56:51-03:00 if (!iso) { const now = new
Date(); const offset = -now.getTimezoneOffset(); const offH = String(Math.floor(Math.abs(offset) / 60)).padStart(2,
'0'); const offM = String(Math.abs(offset) % 60).padStart(2, '0'); const sign = offset >= 0 ? '+' : '-'; return `${no
w.getFullYear()}-${String(now.getMonth()+1).padStart(2,'0')}-${String(now.getDate()).padStart(2,'0')}T${String(now.ge
tHours()).padStart(2,'0')}:${String(now.getMinutes()).padStart(2,'0')}:${String(now.getSeconds()).padStart(2,'0')}${s
ign}${offH}:${offM}`; } var str = String(iso).trim(); // Odoo (XML-RPC) devolve "YYYY-MM-DD HH:MM:SS" em UTC, sem 'T'
e sem offset. // Converter para horario de Brasilia (-03:00) no formato exigido pelo XSD. var mOdoo =
str.match(/^(\d{4})-(\d{2})-(\d{2})[ T](\d{2}):(\d{2})(?::(\d{2}))?$/); if (mOdoo) { var utc = Date.UTC(+mOdoo[1],
+mOdoo[2] - 1, +mOdoo[3], +mOdoo[4], +mOdoo[5], +(mOdoo[6] || 0)); var d = new Date(utc - 3 * 3600 * 1000); return
d.getUTCFullYear() + '-' + pad2(d.getUTCMonth() + 1) + '-' + pad2(d.getUTCDate()) + 'T' + pad2(d.getUTCHours()) + ':'
+ pad2(d.getUTCMinutes()) + ':' + pad2(d.getUTCSeconds()) + '-03:00'; } // Ja veio com data + hora + offset (ISO
completo) if (/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:[.,]\d+)?(?:Z|[+-]\d{2}:?\d{2})$/.test(str)) { var dIso = new
Date(str); if (!isNaN(dIso.getTime())) { var b = new Date(dIso.getTime() - 3 * 3600 * 1000); return
b.getUTCFullYear() + '-' + pad2(b.getUTCMonth() + 1) + '-' + pad2(b.getUTCDate()) + 'T' + pad2(b.getUTCHours()) + ':'
+ pad2(b.getUTCMinutes()) + ':' + pad2(b.getUTCSeconds()) + '-03:00'; } } // Apenas data "YYYY-MM-DD" if
(/^\d{4}-\d{2}-\d{2}$/.test(str)) return str + 'T00:00:00-03:00'; // Fallback: nao arriscar um valor invalido no XML
— usar agora console.warn('[NFE-XML] dhEmi em formato desconhecido ("' + str + '"), usando data/hora atual'); return
formatDh(null); } function pad2(n) { return String(n).padStart(2, '0'); } function calcIdDest(ufEmit, ufDest) {
```

```
if (!ufEmit || !ufDest) return '1'; if (ufEmit === ufDest) return '1'; // mesma UF if (ufDest === 'EX') return '3'; //
exterior return '2'; // fora UF } function calcIndIEDest(partner) { const ie = partner.inscr_est || ''; const uf =
partner.state || partner.UF || ''; if (!ie || onlyNum(ie) === 'ISENTO' || onlyNum(ie) === '') return '9'; return '1';
// contribuinte ICMS } function logField(fieldName, value, isError) { var displayVal = value === undefined || value
=== null ? '(undefined)' : JSON.stringify(value); if (isError) { console.error('[NFE-XML] ' + fieldName + ': ' +
displayVal + ' *** VAZIO/INVALIDO ***'); } else { console.log('[NFE-XML] ' + fieldName + ': ' + displayVal); } } /** *
Calcula a chave de acesso da NF-e (44 digitos) com DV. * Formato: cUF(2) + AAMM(4) + CNPJ(14) + mod(2) + serie(3) +
nNF(9) + tpEmis(1) + cNF(8) + cDV(1) = 44 */ function calcAccessKey(cUF, dhEmi, cnpj, mod, serie, nNF, tpEmis, cNF) {
var aamm = dhEmi.slice(2, 4) + dhEmi.slice(5, 7); var key = String(cUF).padStart(2, '0') + aamm +
onlyNum(cnpj).padStart(14, '0') + String(mod).padStart(2, '0') + String(serie).padStart(3, '0') +
String(nNF).padStart(9, '0') + String(tpEmis) + String(cNF).padStart(8, '0'); // DV = modulo 11 dos 43 digitos, pesos
2-9 ciclicos do DIREITA para ESQUERDA // Para iterar da esquerda, o peso do digito i (0=esquerda, 42=direita) e: //
pesos_ciclicos[(42 - i) % 8] var cycle = [2,3,4,5,6,7,8,9]; var sum = 0; for (var i = 0; i < 43; i++) { sum +=
parseInt(key[i]) * cycle[(42 - i) % 8]; } var remainder = sum % 11; var cDV = 11 - remainder; if (cDV === 10 || cDV
=== 11) cDV = 0; return key + String(cDV); } module.exports = { gerarXmlNFe };
```

4. Assinatura Digital XMLDSig

A assinatura digital da NF-e segue o padrao XMLDSig definido no Manual de Orientacao do Contribuinte da SEFAZ. O grupo infNFe e assinado com RSA-SHA1, utilizando canonicalizacao C14N (REC-xml-c14n-20010315) e transformacoes enveloped-signature + C14N. A referencia (Reference URI) aponta para o atributo Id="NFe" do infNFe. O elemento Signature e inserido imediatamente apos o fechamento de infNFe, dentro de NFe, conforme exigido pela SEFAZ.

services/nfe-signer.js

Assinatura digital XMLDSig da NF-e (A1). Usa xml-crypto para assinar o grupo infNFe com RSA-SHA1 + C14N. Valida a presenca de X509Certificate no resultado (rejeicao 297 da SEFAZ).

nfe-signer.js (/home/z/my-project/API-Odoo/services/nfe-signer.js)

```
/** * services/nfe-signer.js — Assinatura digital XMLDSig da NF-e (A1) *
============================================================ * Assina o grupo <infNFe> conforme o Manual de
Orientacao do Contribuinte: * - Canonicalizacao: C14N (REC-xml-c14n-20010315) * - Transformacoes: enveloped-signature
+ C14N * - Digest: SHA-1 * - Assinatura: RSA-SHA1 * - Reference URI: #NFe<chave de 44 digitos> * - <Signature> fica
logo apos </infNFe>, dentro de <NFe> */ var { SignedXml } = require('xml-crypto'); var { carregarCertificado } =
require('./nfe-cert'); var C14N = 'http://www.w3.org/TR/2001/REC-xml-c14n-20010315'; var ENVELOPED =
'http://www.w3.org/2000/09/xmldsig#enveloped-signature'; var SHA1 = 'http://www.w3.org/2000/09/xmldsig#sha1'; var
RSA_SHA1 = 'http://www.w3.org/2000/09/xmldsig#rsa-sha1'; /** Remove indentacao entre tags — a NF-e deve trafegar
minificada. */ function minify(xml) { return String(xml).replace(/\r?\n\s*/g, '').replace(/>\s+</g, '><').trim(); }
/** * Assina um XML de NF-e (elemento raiz <NFe>). * @param {string} xmlNFe XML da NF-e nao assinado * @returns {{
xml: string, chave: string, certInfo: object }} */ function assinarNFe(xmlNFe) { var cert = carregarCertificado(); if
(!cert) { var err = new Error('Certificado digital A1 nao configurado. No Render, defina as variaveis de ambiente
NFE_CERT_PFX_BASE64 (conteudo do .pfx em base64) e NFE_CERT_SENHA. Alternativamente, envie via POST
/api/v1/nfe/certificado (mas o arquivo sera perdido em reinicios).'); err.code = 'CERT_AUSENTE'; throw err; } if
(cert.info && cert.info.expirado) { var e2 = new Error('Certificado digital A1 expirado em ' + cert.info.validoAte +
'. Renove antes de emitir.'); e2.code = 'CERT_EXPIRADO'; throw e2; } var xml = minify(xmlNFe); var m =
xml.match(/Id="(NFe\d{44})"/); if (!m) throw new Error('infNFe sem atributo Id="NFe<chave>" — nao eh possivel
assinar.'); var idAttr = m[1]; var chave = idAttr.slice(3); var sig = new SignedXml({ privateKey: cert.privateKeyPem,
publicCert: cert.certPem, signatureAlgorithm: RSA_SHA1, canonicalizationAlgorithm: C14N, }); sig.addReference({
xpath: "//*[local-name(.)='infNFe']", transforms: [ENVELOPED, C14N], digestAlgorithm: SHA1, uri: '#' + idAttr, });
sig.computeSignature(xml, { location: { reference: "//*[local-name(.)='infNFe']", action: 'after' }, }); var assinado
= sig.getSignedXml(); // A SEFAZ nao aceita prefixo de namespace na Signature nem KeyInfo ausente if
(assinado.indexOf('<Signature') === -1) { throw new Error('Falha ao gerar a assinatura digital (Signature
ausente).'); } if (assinado.indexOf('<X509Certificate>') === -1) { throw new Error('Assinatura gerada sem
X509Certificate — a SEFAZ rejeitaria (rejeicao 297).'); } console.log('[NFE-SIGN] NF-e ' + chave + ' assinada com o
certificado de ' + (cert.info && cert.info.titular)); return { xml: assinado, chave: chave, certInfo: cert.info }; }

module.exports = { assinarNFe, minify };
```

5. Comunicacao com a SEFAZ

O sefaz-client.js implementa toda a comunicacao direta com os webservices da SEFAZ para NF-e 4.00. Ele cria um agente HTTPS com mTLS (mutual TLS) usando o certificado A1, monta envelopes SOAP 1.2, envia lotes via NFeAutorizacao4 com indSinc=1 (sincrono), e trata os cenarios de processamento assincrono (cStat 103/105) com polling via NFeRetAutorizacao4. Suporta os autorizadores PR, SP e SVRS (fallback para UFs nao mapeadas). O bundle de CAs da ICP-Brasil esta embutido no codigo para resolver "self-signed certificate in certificate chain" em containers Linux sem o bundle instalado.

services/sefaz-client.js

Comunicacao direta com a SEFAZ (NF-e 4.00) via mTLS + SOAP 1.2. Inclui: criacao do agente mTLS com PEM, envelope SOAP, NFeAutorizacao4, NFeRetAutorizacao4 (polling), NFeConsultaProtocolo4, NFeStatusServico4, e montagem do nfeProc. Webservices mapeados: PR, SP, SVRS (fallback).

sefaz-client.js (/home/z/my-project/API-Odoo/services/sefaz-client.js)

```
/** * services/sefaz-client.js — Comunicacao direta com a SEFAZ (NF-e 4.00) *
============================================================================ * Autorizacao propria com certificado A1 (mTLS
+ SOAP 1.2). * * Fluxo: * 1. assinarNFe() -> XML assinado * 2. NFeAutorizacao4 -> envia lote (indSinc=1, sincrono) *
3. cStat 100 -> autorizada; monta o nfeProc * cStat 103/105 (lote em processamento) -> NFeRetAutorizacao4 (polling) *
4. Retorna { autorizada, chave, protocolo, nfeProc, cStat, xMotivo } * * UF suportadas por autorizador: PR usa
webservice proprio; demais UFs caem * no SVRS quando nao mapeadas explicitamente. */ var https = require('https');
var axios = require('axios'); var { carregarCertificado } = require('./nfe-cert'); var { assinarNFe, minify } =
require('./nfe-signer'); // ============================================================ // Bundle de CAs da
ICP-Brasil para validacao TLS dos webservices da SEFAZ. // Inclui: AC SOLUTI SSL EV G4 + AC Raiz Brasileira v10 //
Validos ate: 01/07/2032. Atualizar se a SEFAZ trocar a cadeia. //
============================================================ var ICP_BRASIL_CA_BUNDLE = [ // AC SOLUTI SSL EV G4
(emitida pela AC Raiz Brasileira v10) '-----BEGIN CERTIFICATE-----\nMIIHtDCCBZygAwIBAgIJANjGl6F55VD+MA0GCSqGSIb3DQEBD
QUAMIGYMQswCQYD\nVQQGEwJCUjETMBEGA1UECgwKSUNQLUJyYXNpbDE9MDsGA1UECww0SW5zdGl0dXRv\nIE5hY2lvbmFsIGRlIFRlY25vbG9naWEgZG
EgSW5mb3JtYWNhbyAtIElUSTE1MDMG\nA1UEAwwsQXV0b3JpZGFkZSBDZXJ0aWZpY2Fkb3JhIFJhaXogQnJhc2lsZWlyYSB2\nMTAwHhcNMjMwMzIyMTg
wOTExWhcNMzIwNzAxMTIwMDU5WjB3MQswCQYDVQQGEwJC\nUjETMBEGA1UEChMKSUNQLUJyYXNpbDE1MDMGA1UECxMsQXV0b3JpZGFkZSBDZXJ0\naWZp
Y2Fkb3JhIFJhaXogQnJhc2lsZWlyYSB2MTAxHDAaBgNVBAMTE0FDIFNPTFVU\nSSBTU0wgRVYgRzQwggIiMA0GCSqGSIb3DQEBAQUAA4ICDwAwggIKAoI
```

```
CAQDHv3Kv\noEPrNrzImIPn17GI5vdoVxghsm6EVLMUjnM4JdCpDED+0BqZF0kycyZaiWt7jqSR\nvcGm66RKzSGcHJlUgahp9qcXAmSwMn00pwvgBKb+
4htp48vQc1/5MWpaBzQW4Di/\ntWvNkh9URtMyhtltf2u3s9r5vgF12ff7mCu3oj0bDBIaGs/a9EtMKoCfw/ziKUp7\n11JYu1fbIWVOgbW9iHE24oiE3
3LGLm+uToCWpjGL3n9D+q+ryfIYFoes6gPCYYSt\nudDUB9lfpe83IOVcVslL3DmYd2oEncGCogO3qzaMSH3OVLMO4Rg5edERpMw5U0tA\nMeyO0k5/tm
nFfUM476lZl+ce2Ol56p7R2yjKxHJizeCOSmwDE5FXz7ll+Zq9C7QW\nUzoPQtyT739UGEeBRTAz4KsO77frCtdifGRvX3lMfI8qeMnfvf08BK9e2dRkC
HwD\niv23Aw7QIixDS9PiSsMxObgjHwroEqAAN2Mwz1B1zAuzZVUH7k6MyQQ/II/GDUpT\njT4VKnhjdIfz5aEFHx7By2XjMkx1hyeONLS/2SoDnKitE9
yY/PASqWDCPCpSoJ+x\nfEdyZvoawEbJfL+CMhU5I7IXgf9f7gibghIc2CG4bf6dfVAdPcGkYkcjw21dtq/G\n1V2dHpOX67BbihThAVr8Z7NTgVAv4nC
6MPpAywIDAQABo4ICHzCCAhswggEHBgNV\nHSAEgf8wgfwwQwYFYEwBAQAwOjA4BggrBgEFBQcCARYsaHR0cDovL2FjcmFpei5p\nY3BicmFzaWwuZ292
LmJyL0RQQ2FjcmFpei5wZGYwUAYGYEwBAYECMEYwRAYIKwYB\nBQUHAgEWOGh0dHA6Ly9jY2QuYWNzb2x1dGkuY29tLmJyL2RvY3MvZHBjLWFjLXNv\nb
HV0aS1zc2wtZXYucGRmMFAGBmBMAQIBcDBGMEQGCCsGAQUFBwIBFjhodHRwOi8v\nY2NkLmFjc29sdXRpLmNvbS5ici9kb2NzL2RwYy1hYy1zb2x1dGkt
c3NsLWV2LnBk\nZjAHBgVngQwBATAIBgZngQwBAgIwQAYDVR0fBDkwNzA1oDOgMYYvaHR0cDovL2Fj\ncmFpei5pY3BicmFzaWwuZ292LmJyL0xDUmFjc
mFpenYxMC5jcmwwHwYDVR0jBBgw\nFoAUdPN+//yfU3rxfOurPqSm2hi6RWMwHQYDVR0OBBYEFP4GuSyVfi/m0Lio8S+3\n8i6F1dfAMA8GA1UdEwEB/w
QFMAMBAf8wDgYDVR0PAQH/BAQDAgGGMB0GA1UdJQQW\nMBQGCCsGAQUFBwMBBggrBgEFBQcDAjBMBggrBgEFBQcBAQRAMD4wPAYIKwYBBQUH\nMAKGMGh
0dHA6Ly9hY3JhaXouaWNwYnJhc2lsLmdvdi5ici9JQ1AtQnJhc2lsdjEw\nLmNydDANBgkqhkiG9w0BAQ0FAAOCAgEAABxayeHitwL18QeXSQvRZ2eiNb
82IlYT\nuvER4JRMzZDWoamKOqmD7KXSSj+sdBThYiRkkNiVFiMn2qoYAdylI2I4w1npbxyr\nukfXQ7tadTEiMCFva0uHHw9lpBx+oyy9rcLM7qC5qqu
ksyhC222Yt3WbqC6Fla+L\no3GlTOpogqexeyc9hgvAQxeMmq+xyDcjSLzKmmRmMKQ9y3w7wpufXTO/0K5uOLLZ\nsfyXZTw+MYYeIk2+GNv1qQBbWo3g
mwlD1W0pJEHe+/KxiCRkDHpJY7Lk2Rm4bSDZ\nRr4Bn8bk/XJWpiu7Fm9b8piPKjTtstDYTzu40ccPRh9UCWDUz4nKF97dXjIgYf+a\nTA0vnKdlnpPUD
eBVpfyXavhGf/akFh5AO7/v6xkzWOUlawn5g614mWhOQ6ITwmua\ny1spnpBO684d0bynFQfMoZGS5fdKoYKKDzp29xhBm3s9WD1f/oP79Ie0eDribpOv
\nj3Xsjz72MTG4+UVxuv0OIYuXDc8x1foMzVOco6DxuLel6KG5RH+m0tWmX4ouCgBK\nTNUQC70AWHBa4PCF5YA7H8qVnH2EUBPo3rxOY0wN6GzyMbg9+
D9l5e2Xcg7/ytqY\nBIBnZKLPjzS3OqUsM9UgUKGwcEnaHnmRxH8vyVEMGnoK1cZNf9uDM9sMGgQUzKwV\nwnixwyINOM8U=\n-----END
CERTIFICATE-----', // Autoridade Certificadora Raiz Brasileira v10 (self-signed, ITI) '-----BEGIN CERTIFICATE-----\nM
IIGrDCCBJSgAwIBAgIJANLVi0S/gZNCMA0GCSqGSIb3DQEBDQUAMIGYMQswCQYD\nVQQGEwJCUjETMBEGA1UECgwKSUNQLUJyYXNpbDE9MDsGA1UECww0
SW5zdGl0dXRv\nIE5hY2lvbmFsIGRlIFRlY25vbG9naWEgZGEgSW5mb3JtYWNhbyAtIElUSTE1MDMG\nA1UEAwwsQXV0b3JpZGFkZSBDZXJ0aWZpY2Fkb
3JhIFJhaXogQnJhc2lsZWlyYSB2\nMTAwHhcNMTkwNzAxMTkxNTU5WhcNMzIwNzAxMTIwMDU5WjCBmDELMAkGA1UEBhMC\nQlIxEzARBgNVBAoMCklDUC
1CcmFzaWwxPTA7BgNVBAsMNEluc3RpdHV0byBOYWNp\nb25hbCBkZSBUZWNub2xvZ2lhIGRhIEluZm9ybWFjYW8gLSBJVEkxNTAzBgNVBAMM\nLEF1dG9
yaWRhZGUgQ2VydGlmaWNhZG9yYSBSYWl6IEJyYXNpbGVpcmEgdjEwMIIC\nIjANBgkqhkiG9w0BAQEFAAOCAg8AMIICCgKCAgEAk3AxKl1ZtP0pNyjChq
O7qNkn\n+/sClZeqiV/Kd7KnnbkDbI2y3VWcUG7feCE/deIxot6GH6JXncRG794UZl+4doD0\nD0/cEwBd4DvrDSZm0RT40xhmYYOTxZDJxv+coTHdmsT
5aNmSkktfjzYX4HQHh/7M\nem+kTOpT/3E4K6B7KVs9HkOT7nXx5yU1qYbVWqI0qpJM9mOTSFx8C9HiKcHvLCvt\n1ioXKPAmFuHPkayOcXP2MXeb+VRN
jWKU4E+L2t5uZPKVx1M/9i1DztlLb4K8OfYg\nGaPDUSF1sxnoGk5qZHLleO6KjCpmuQepmgsBvxi2YNO7X2YUwQQx1AXNSolgtkAR\n5gt+1WzxhbFUh
ItQqlhqxgWHefLmiT5T/Ctz/P2v+zSO4efkkIzsi1iwD+ypZvM2\nlnIvB24RcSN6jzmCahLPX4CwjwIK6JsSoMVxIhpZHCguUP4LXqP8IWUZ6WgS/4zB
\n7B9E0EICl2rM1PRy+6ulv+ZOW256e8a0pijUB+hXM1msUq9L92476FAAX8va3sP7\n+Uut94+bGHmubcTLImWUPrxNT7QyrvE3FyHicfiHioeFL2oV4
cXTLZrEq2wS8R4P\nKPdSzNn5Z9e2uMEGYQaSNO+OwvVycpIhOBOqrm12wJ9ZhWKtM5UOo34/o37r5ZBI\nTYXAGbhqQDB9mWXwH+0CAwEAAaOB9jCB8z
BOBgNVHSAERzBFMEMGBWBMAQEAMDow\nOAYIKwYBBQUHAgEWLGh0dHA6Ly9hY3JhaXouaWNwYnJhc2lsLmdvdi5ici9EUENh\nY3JhaXoucGRmMEAGA1U
dHwQ5MDcwNaAzoDGGL2h0dHA6Ly9hY3JhaXouaWNwYnJh\nc2lsLmdvdi5ici9MQ1JhY3JhaXp2MTAuY3JsMB8GA1UdIwQYMBaAFHTzfv/8n1N6\n8Xzr
qz6kptoYukVjMB0GA1UdDgQWBBR0837//J9TevF866s+pKbaGLpFYzAPBgNV\nHRMBAf8EBTADAQH/MA4GA1UdDwEB/wQEAwIBBjANBgkqhkiG9w0BAQ0
FAAOCAgEA\neCNhBSuy/Ih/T+1VOtAJju85SrtoE3vET1qXASpmjQllDHG/ph7VFNRAkC+gha+B\nCbjoA5oJ/8wwl+Qdp1KGz6nXXFTLx3osU+kjm0sr
mBf9nyXHPqvFyvBeB0A7sYb7\nTmII9GKD20oCxsdkccR/oE/JuTaNnGq0GYZ2aDb5v62uLi21Y6P9UBiTxZqQ4ojW\nET6kXNjlK238jpXv17FR8Sg3V
usCvX7Q8eJkavvHHZDeWck2fSA+ycAc2JeL2Z0B\nMSxGWpH32WM9J8+6XqCJUXHiWEV0zCE8wDYiYC+047pTxQI/gB/FcU7jvylh98DJ\nkQPHd/Tp6O
g3ynlDA9n9uBbxYHVRZs9vsZ/7xTFaxRe+zk8dhgKgZ/3RrcMFB570\n2t8LFbyuUE/kQVY6rZ0QJ9qMWQ7VPLRwRhiMeU3k8WDJb/tBbOXHBqldTbWyQ
+mp\nMEDWhbrzE/IED82wAuO23Tb05cYk2xC7+Izef8fSc3XdJDuPSbcDpWukzyCDtSEH\nisLiGEtIbYRiPsF3czlQPsnIEVoTTCWxHCH1zYR6zScSv1
8Qh69qVe2J40K5jZoP\nGEOhq/oKhVJQAdvAFW5Odp7mF3Tk9nivjjsctJSxY26LFiV5GRV+07SSse4ti0aO\njO5PLg5SWjfcOtBG2rz02EIvQAmLcb0
kGBtfdj0lW/w=\n-----END CERTIFICATE-----', ]; var NS = 'http://www.portalfiscal.inf.br/nfe'; // Webservices por
autorizador — [producao, homologacao] var WS = { PR: { autorizacao:
['https://nfe.sefa.pr.gov.br/nfe/NFeAutorizacao4', 'https://homologacao.nfe.sefa.pr.gov.br/nfe/NFeAutorizacao4'],
retAutorizacao: ['https://nfe.sefa.pr.gov.br/nfe/NFeRetAutorizacao4',
'https://homologacao.nfe.sefa.pr.gov.br/nfe/NFeRetAutorizacao4'], consulta:
['https://nfe.sefa.pr.gov.br/nfe/NFeConsultaProtocolo4',
'https://homologacao.nfe.sefa.pr.gov.br/nfe/NFeConsultaProtocolo4'], status:
['https://nfe.sefa.pr.gov.br/nfe/NFeStatusServico4', 'https://homologacao.nfe.sefa.pr.gov.br/nfe/NFeStatusServico4'],
cancelamento: ['https://nfe.sefa.pr.gov.br/nfe/NFeRecepcaoEvento4',
'https://homologacao.nfe.sefa.pr.gov.br/nfe/NFeRecepcaoEvento4'], }, SP: { autorizacao:
['https://nfe.fazenda.sp.gov.br/ws/nfeautorizacao4.asmx',
'https://homologacao.nfe.fazenda.sp.gov.br/ws/nfeautorizacao4.asmx'], retAutorizacao:
['https://nfe.fazenda.sp.gov.br/ws/nferetautorizacao4.asmx',
'https://homologacao.nfe.fazenda.sp.gov.br/ws/nferetautorizacao4.asmx'], consulta:
['https://nfe.fazenda.sp.gov.br/ws/nfeconsultaprotocolo4.asmx',
'https://homologacao.nfe.fazenda.sp.gov.br/ws/nfeconsultaprotocolo4.asmx'], status:
['https://nfe.fazenda.sp.gov.br/ws/nfestatusservico4.asmx',
'https://homologacao.nfe.fazenda.sp.gov.br/ws/nfestatusservico4.asmx'], cancelamento:
['https://www.nfe.fazenda.sp.gov.br/ws/nferecepcaoevento4.asmx',
'https://homologacao.nfe.fazenda.sp.gov.br/ws/nferecepcaoevento4.asmx'], }, SVRS: { autorizacao:
['https://nfe.svrs.rs.gov.br/ws/NfeAutorizacao/NFeAutorizacao4.asmx',
'https://nfe-homologacao.svrs.rs.gov.br/ws/NfeAutorizacao/NFeAutorizacao4.asmx'], retAutorizacao:
['https://nfe.svrs.rs.gov.br/ws/NfeRetAutorizacao/NFeRetAutorizacao4.asmx',
'https://nfe-homologacao.svrs.rs.gov.br/ws/NfeRetAutorizacao/NFeRetAutorizacao4.asmx'], consulta:
['https://nfe.svrs.rs.gov.br/ws/NfeConsulta/NfeConsulta4.asmx',
'https://nfe-homologacao.svrs.rs.gov.br/ws/NfeConsulta/NfeConsulta4.asmx'], status:
['https://nfe.svrs.rs.gov.br/ws/NfeStatusServico/NfeStatusServico4.asmx',
'https://nfe-homologacao.svrs.rs.gov.br/ws/NfeStatusServico/NfeStatusServico4.asmx'], cancelamento:
['https://nfe.svrs.rs.gov.br/ws/NfeRecepcaoEvento/NfeRecepcaoEvento4.asmx',
'https://nfe-homologacao.svrs.rs.gov.br/ws/NfeRecepcaoEvento/NfeRecepcaoEvento4.asmx'], }, }; var CUF = { PR: '41',
SP: '35', RS: '43', SC: '42', MG: '31', RJ: '33', BA: '29', GO: '52', MS: '50', MT: '51' }; function uf() { return
String(process.env.NFE_UF || 'PR').toUpperCase(); } function tpAmb() { return String(process.env.SIEG_TP_AMB ||
process.env.NFE_TP_AMB || '2'); } function isHomolog() { return tpAmb() === '2'; } function autorizador() { return
WS[uf()] ? uf() : 'SVRS'; } function endpoint(tipo) { return WS[autorizador()][tipo][isHomolog() ? 1 : 0]; } function
cUF() { return CUF[uf()] || '41'; } function tag(xml, name) { var m = String(xml || '').match(new RegExp('<' + name +
'[^>]*>([\\s\\S]*?)<\\/' + name + '>', 'i')); return m ? m[1].trim() : ''; } function agent() { var cert =
carregarCertificado(); if (!cert) throw new Error('Certificado A1 nao configurado — impossivel abrir conexao mTLS com
a SEFAZ.'); // Montar lista de CAs confiáveis para validar o certificado TLS do servidor SEFAZ: // 1. CAs do proprio
certificado A1 (intermediarias ICP-Brasil do emitente) // 2. Bundle ICP_BRASIL_CA_BUNDLE embutido no codigo (AC
SOLUTI SSL EV G4 + AC Raiz v10) // Isso resolve "self-signed certificate in certificate chain" em containers Linux

// sem o bundle de CAs da ICP-Brasil instalado no OpenSSL do sistema. var extraCas = ICP_BRASIL_CA_BUNDLE.slice(); if
(cert.chainPem && cert.chainPem.length > 1) { for (var i = 1; i < cert.chainPem.length; i++) { if
```

```
(extraCas.indexOf(cert.chainPem[i]) === -1) extraCas.push(cert.chainPem[i]); } } var opts = { minVersion: 'TLSv1.2', keepAlive: true, rejectUnauthorized: process.env.NFE_TLS_INSECURE === '1' ? false : true, ca: extraCas, }; // Node 20+ com OpenSSL 3 nao suporta PFX com algoritmos legados (3DES/RC2/PBES2-AES). // Usamos PEM (key + cert + ca) extraidos previamente pelo nfe-cert.js. if (cert.privateKeyPem && cert.chainPem && cert.chainPem.length) { opts.key = cert.privateKeyPem; // chainPem ja contem [leaf, ca1, ca2, ...] — formato esperado pelo TLS opts.cert = cert.chainPem.join(''); console.log('[SEFAZ] Agente mTLS criado com PEM (key + cert chain, ' + cert.chainPem.length + ' certificados, ' + opts.ca.length + ' CAs ICP-Brasil).'); } else if (cert.privateKeyPem && cert.certPem) { opts.key = cert.privateKeyPem; opts.cert = cert.certPem; console.log('[SEFAZ] Agente mTLS criado com PEM (key + leaf cert, sem chain, ' + opts.ca.length + ' CAs ICP-Brasil).'); } else { // Fallback: tenta PFX direto (funciona em Node < 20 ou certificados modernos) opts.pfx = cert.pfx; opts.passphrase = cert.senha; console.warn('[SEFAZ] PEM nao disponivel, usando PFX direto no https.Agent (pode falhar no Node 20+ com OpenSSL 3).'); } return new https.Agent(opts); } function envelope(servico, conteudo) { return '<?xml version="1.0" encoding="utf-8"?>' + '<soap12:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" ' + 'xmlns:xsd="http://www.w3.org/2001/XMLSchema" ' + 'xmlns:soap12="http://www.w3.org/2003/05/soap-envelope">' + '<soap12:Body>' + '<nfeDadosMsg xmlns="http://www.portalfiscal.inf.br/nfe/wsdl/' + servico + '">' + conteudo + '</nfeDadosMsg>' + '</soap12:Body></soap12:Envelope>'; } async function soapPost(url, servico, conteudo, timeout) { var body = envelope(servico, conteudo); var resp = await axios.post(url, body, { httpsAgent: agent(), timeout: timeout || 60000, headers: { 'Content-Type': 'application/soap+xml; charset=utf-8', 'SOAPAction': 'http://www.portalfiscal.inf.br/nfe/wsdl/' + servico, 'User-Agent': 'AJL-Odoo-Middleware/1.0', }, validateStatus: function (s) { return s < 600; }, transformResponse: [function (d) { return d; }], }); return { status: resp.status, xml: String(resp.data || '') }; } /** Status do servico da SEFAZ — util para testar o certificado e a conexao. */ async function statusServico() { var cons = '<consStatServ xmlns="' + NS + '" versao="4.00">' + '<tpAmb>' + tpAmb() + '</tpAmb><cUF>' + cUF() + '</cUF><xServ>STATUS</xServ></consStatServ>'; var r = await soapPost(endpoint('status'), 'NFeStatusServico4', cons, 30000); return { httpStatus: r.status, ambiente: tpAmb() === '2' ? 'homologacao' : 'producao', uf: uf(), autorizador: autorizador(), endpoint: endpoint('status'), cStat: tag(r.xml, 'cStat'), xMotivo: tag(r.xml, 'xMotivo'), online: tag(r.xml, 'cStat') === '107', }; } function sanitizeNFe(xmlAssinado) {

return minify(xmlAssinado).replace(/<\?xml[^>]*\?>/g, ''); } /** Consulta o recibo do lote (assincrono). */ async function consultarRecibo(nRec) { var cons = '<consReciNFe xmlns="' + NS + '" versao="4.00">' + '<tpAmb>' + tpAmb() + '</tpAmb><nRec>' + nRec + '</nRec></consReciNFe>'; var r = await soapPost(endpoint('retAutorizacao'), 'NFeRetAutorizacao4', cons, 45000); return r.xml; } /** Consulta a situacao da NF-e pela chave (usado como ultimo recurso). */ async function consultarChave(chave) { var cons = '<consSitNFe xmlns="' + NS + '" versao="4.00">' + '<tpAmb>' + tpAmb() + '</tpAmb><xServ>CONSULTAR</xServ><chNFe>' + chave + '</chNFe></consSitNFe>'; var r = await soapPost(endpoint('consulta'), 'NFeConsultaProtocolo4', cons, 45000); return r.xml; } function extrairProtNFe(xml) { var m = String(xml || '').match(/<protNFe[\s\S]*?<\/protNFe>/i); return m ? m[0] : ''; } function montarNfeProc(xmlNFeAssinado, protNFe) { return '<?xml version="1.0" encoding="UTF-8"?>' + '<nfeProc xmlns="' + NS + '" versao="4.00">' + sanitizeNFe(xmlNFeAssinado) + protNFe + '</nfeProc>'; } function sleep(ms) { return new Promise(function (r) { setTimeout(r, ms); }); } /** * Assina e autoriza a NF-e na SEFAZ. * @param {string} xmlNFe XML <NFe> gerado (sem assinatura) */ async function autorizarNFe(xmlNFe) { var assinado = assinarNFe(xmlNFe); var chave = assinado.chave; var idLote = String(Date.now()).slice(-15); var enviNFe = '<enviNFe xmlns="' + NS + '" versao="4.00">' + '<idLote>' + idLote + '</idLote>' + '<indSinc>1</indSinc>' + sanitizeNFe(assinado.xml) + '</enviNFe>'; console.log('[SEFAZ] Enviando NF-e ' + chave + ' | UF=' + uf() + ' | amb=' + tpAmb() + ' | ' + endpoint('autorizacao')); var r; try { r = await soapPost(endpoint('autorizacao'), 'NFeAutorizacao4', enviNFe, 90000); } catch (err) { return { autorizada: false, chave: chave, xmlAssinado: assinado.xml, cStat: '', xMotivo: 'Falha de comunicacao com a SEFAZ: ' + err.message, erro: err.message, endpoint: endpoint('autorizacao'), }; } var respXml = r.xml; console.log('[SEFAZ] Resposta HTTP ' + r.status + ' (' + respXml.length + ' chars): ' + respXml.slice(0, 1200)); var cStatLote = tag(respXml, 'cStat'); var xMotivoLote = tag(respXml, 'xMotivo'); var protNFe = extrairProtNFe(respXml); // Lote recebido em processamento -> consultar recibo if (!protNFe && (cStatLote === '103' || cStatLote === '105')) { var nRec = tag(respXml, 'nRec'); for (var i = 0; i < 8 && nRec; i++) { await sleep(2500); var retXml = await consultarRecibo(nRec); console.log('[SEFAZ] Consulta recibo ' + nRec + ' tentativa ' + (i + 1) + ': ' + tag(retXml, 'cStat') + ' ' + tag(retXml, 'xMotivo')); protNFe = extrairProtNFe(retXml); if (protNFe) { respXml = retXml; break; } if (tag(retXml, 'cStat') !== '105') { respXml = retXml; break; } }

} var cStat = protNFe ? tag(protNFe, 'cStat') : cStatLote; var xMotivo = protNFe ? tag(protNFe, 'xMotivo') : xMotivoLote; var nProt = protNFe ? tag(protNFe, 'nProt') : ''; var autorizada = cStat === '100' || cStat === '150'; var out = { autorizada: autorizada, chave: chave, protocolo: nProt, cStat: cStat, xMotivo: xMotivo, ambiente: tpAmb() === '2' ? 'homologacao' : 'producao', uf: uf(), endpoint: endpoint('autorizacao'), httpStatus: r.status, xmlAssinado: assinado.xml, respostaSefaz: respXml.slice(0, 6000), certificado: assinado.certInfo, }; if (autorizada) { out.nfeProc = montarNfeProc(assinado.xml, protNFe); console.log('[SEFAZ] NF-e AUTORIZADA! chave=' + chave + ' protocolo=' + nProt); } else { out.erro = 'SEFAZ rejeitou: ' + (cStat || 's/cStat') + ' - ' + (xMotivo || 'sem motivo informado'); console.error('[SEFAZ] ' + out.erro); } return out; } module.exports = { autorizarNFe, statusServico, consultarChave, consultarRecibo, montarNfeProc, endpoint, uf, tpAmb, cUF, soapPost, };
```

6. Orquestracao da Emissao (sieg-api.js)

O sieg-api.js e o orquestrador que decide como emitir a NF-e. Ele suporta dois modos: modo proprio (default) onde o XML e assinado com o certificado A1 e enviado direto a SEFAZ, e modo SIEG onde o XML ja assinado e autorizado e importado no cofre da SIEG. Apos a autorizacao, o DANFE e gerado localmente (PDFKit) ou via SIEG, conforme a configuracao. O sistema tambem suporta emissao de NFS-e via SIEG (DPS) e importacao opcional do XML autorizado no cofre SIEG.

services/sieg-api.js

Orquestracao da emissao fiscal. decide entre emissao propria (SEFAZ direta) ou via SIEG. Gera DANFE local ou via SIEG. Suporta NF-e e NFS-e. Importa XML no cofre SIEG se configurado.

sieg-api.js (/home/z/my-project/API-Odoo/services/sieg-api.js)

```
/** * services/sieg-api.js — Emissao fiscal * ================================== * NF-e (mod 55): emissao PROPRIA — o XML e assinado com o certificado A1 da * AJL e autorizado direto na SEFAZ (services/sefaz-client.js). O DANFE em PDF * eh gerado localmente (services/danfe-pdf.js). A SIEG deixa de ser necessaria * para emitir; continua opcional como cofre de XML (SIEG_IMPORT_XML=1). * * NFS-e: continua via SIEG (prefeitura/DPS). * * Variaveis de ambiente: * NFE_EMISSAO_MODO proprio (default) | sieg * NFE_UF UF do emitente (default PR) * SIEG_TP_AMB 1=producao, 2=homologacao * NFE_DANFE_PROVIDER local (default) | sieg * SIEG_IMPORT_XML 1 = apos autorizar, importa o nfeProc no cofre SIEG */ const axios = require('axios'); const { getAuthHeaders } = require('./sieg-auth'); const { gerarXmlNFe } = require('./sieg-nfe-xml'); const { gerarXmlDPS } = require('./sieg-nfse-xml'); const { autorizarNFe } = require('./sefaz-client'); const { gerarDanfePdf } = require('./danfe-pdf'); const SIEG_BASE = 'https://api.sieg.com'; function modoEmissao() { return String(process.env.NFE_EMISSAO_MODO || 'proprio').toLowerCase(); } /** Importa um XML ja autorizado (nfeProc) no cofre da SIEG — opcional. */ async function importarXmlSieg(xmlAutorizado) { const xmlMin = String(xmlAutorizado).replace(/>\s+</g, '><'); const headers = await getAuthHeaders(); const resp = await axios.post(SIEG_BASE + '/api/v1/send-xml', { Xml: Buffer.from(xmlMin, 'utf-8').toString('base64'), }, { headers, timeout: 60000, validateStatus: function (s) { return s < 500; } }); console.log('[SIEG-API] Import no cofre HTTP ' + resp.status + ': ' + JSON.stringify(resp.data).slice(0, 500)); return { httpStatus: resp.status, resposta: resp.data }; } /** * Emissao propria: gera XML -> assina com A1 -> autoriza na SEFAZ. * Retorna o mesmo formato de resultado consumido por sieg-odoo-emit. */ async function emitirNFePropria(dadosOdoo) { const xml = gerarXmlNFe(dadosOdoo); console.log('[NFE] Emissao propria — XML gerado (' + xml.length + ' chars)'); let r; try { r = await autorizarNFe(xml); } catch (err) { console.error('[NFE] Falha antes do envio a SEFAZ: ' + err.message); return { sucesso: false, httpStatus: err.code === 'CERT_AUSENTE' || err.code === 'CERT_EXPIRADO' ? 422 : 500, xmlEnviado: xml, resposta: { xMotivo: err.message }, data: null, erro: err.message, statusCode: err.code || null, }; } const resultado = { sucesso: !!r.autorizada, httpStatus: 200, xmlEnviado: r.nfeProc || r.xmlAssinado || xml, xmlAssinado: r.xmlAssinado, nfeProc: r.nfeProc || null, resposta: { chNFe: r.chave, nProt: r.protocolo, cStat: r.cStat, xMotivo: r.xMotivo, ambiente: r.ambiente, uf: r.uf, endpoint: r.endpoint, respostaSefaz: r.respostaSefaz,

}, data: r.nfeProc || null, erro: r.autorizada ? null : (r.erro || r.xMotivo || 'NF-e nao autorizada'), statusCode: r.cStat || null, }; // Cofre SIEG (opcional): guarda o XML autorizado if (resultado.sucesso && String(process.env.SIEG_IMPORT_XML || '') === '1') { try { resultado.cofreSieg = await importarXmlSieg(r.nfeProc); } catch (e) { console.error('[SIEG-API] Falha ao importar no cofre (nota continua autorizada): ' + e.message); resultado.cofreSieg = { erro: e.message }; } } return resultado; } /** * Caminho legado: importa no SIEG um XML de NF-e ja assinado/autorizado. * So eh usado com NFE_EMISSAO_MODO=sieg. */ async function enviarNFeViaSieg(dadosOdoo) { const xml = gerarXmlNFe(dadosOdoo); const xmlMin = xml.replace(/>\s+</g, '><'); if (xmlMin.indexOf('<Signature') === -1 || xmlMin.indexOf('<protNFe') === -1) { return { sucesso: false, httpStatus: 422, xmlEnviado: xml, resposta: null, data: null, erro: 'O endpoint SIEG /send-xml aceita apenas XML de NF-e ja assinado e autorizado (nfeProc). Use NFE_EMISSAO_MODO=proprio para emitir com o certificado A1.', statusCode: 422, }; } const headers = await getAuthHeaders(); const resp = await axios.post(SIEG_BASE + '/api/v1/send-xml', { Xml: Buffer.from(xmlMin, 'utf-8').toString('base64'), }, { headers, timeout: 60000, validateStatus: function (s) { return s < 500; } }); const result = resp.data || {}; return { sucesso: result.IsSuccess === true, httpStatus: resp.status, xmlEnviado: xml, resposta: result, data: result.Data || null, erro: result.ErrorMessage || result.Message || null, statusCode: result.StatusCode || null, }; } async function enviarNFe(dadosOdoo) { return modoEmissao() === 'sieg' ? enviarNFeViaSieg(dadosOdoo) : emitirNFePropria(dadosOdoo); } /** * Emitir NFS-e via SIEG (DPS) */ async function emitirNFSe(dadosOdoo) { const xml = gerarXmlDPS(dadosOdoo); console.log('[SIEG-API] Emitindo NFS-e DPS (' + xml.length + ' chars)'); const headers = await getAuthHeaders(); let resp; try { resp = await axios.post(SIEG_BASE + '/api/v1/send-xml', { Xml: Buffer.from(xml, 'utf-8').toString('base64'), }, { headers, timeout: 60000, validateStatus: function (s) { return s < 500; } }); } catch (err) { console.error('[SIEG-API] Erro de conexao SIEG:', err.message); throw err; } const result = resp.data; const httpStatus = resp.status;

console.log('[SIEG-API] Resposta send-xml NFS-e HTTP ' + httpStatus + ':', JSON.stringify(result).slice(0, 2000)); if (httpStatus >= 400 && httpStatus < 500) { console.error('[SIEG-API] *** ERRO SIEG NFS-e HTTP ' + httpStatus + ' *** ' + (result.ErrorMessage || result.Message || '')); } return { sucesso: !!(result.IsSuccess === true), httpStatus: httpStatus, xmlEnviado: xml, resposta: result, data: result.Data || null, erro: result.ErrorMessage || result.Message || null, statusCode: result.StatusCode || null, }; } /** * DANFE (PDF) — local por padrao; SIEG apenas se NFE_DANFE_PROVIDER=sieg */ async function gerarDanfe(xmlAutorizado) { if (String(process.env.NFE_DANFE_PROVIDER || 'local').toLowerCase() === 'sieg') { const headers = await getAuthHeaders(); const resp = await axios.post(SIEG_BASE + '/api/v1/gerarDanfeViaXml', { Xml: Buffer.from(xmlAutorizado, 'utf-8').toString('base64'), }, { headers, timeout: 30000 }); return resp.data; } const pdf = await gerarDanfePdf(xmlAutorizado); console.log('[DANFE] PDF gerado localmente (' + pdf.length + ' bytes)'); return pdf.toString('base64'); } /** * DANFSE (PDF) via SIEG */ async function gerarDanfse(xmlAutorizado) { const headers = await getAuthHeaders(); const resp = await axios.post(SIEG_BASE + '/api/v1/gerarDanfseViaXml', { Xml: Buffer.from(xmlAutorizado, 'utf-8').toString('base64'), }, { headers, timeout: 30000 }); return resp.data; } /** * Emitir nota fiscal (auto-detect NF-e ou NFS-e) */ async function emitirNota(dadosOdoo) { let tipo = dadosOdoo.tipo; if (!tipo) { tipo = (dadosOdoo.service && dadosOdoo.lines && dadosOdoo.lines.every(function (l) { return l.detailed_type === 'service'; })) ? 'nfse' : 'nfe'; } let resultado; if (tipo === 'nfse') { resultado = await emitirNFSe(dadosOdoo); } else { resultado = await enviarNFe(dadosOdoo); } resultado.tipo = tipo; // Se autorizado, gerar PDF if (resultado.sucesso && resultado.data) { try { let xmlAutorizado = resultado.data; if (typeof xmlAutorizado === 'string' && xmlAutorizado.length < 500 && xmlAutorizado.indexOf('<') === -1) { xmlAutorizado = Buffer.from(xmlAutorizado, 'base64').toString('utf-8'); } resultado.pdfBase64 = tipo === 'nfse' ? await gerarDanfse(xmlAutorizado) : await gerarDanfe(xmlAutorizado); resultado.pdfGerado = true; } catch (errPdf) { console.error('[SIEG-API] Erro ao gerar PDF (nota ainda pode estar autorizada):', errPdf.message); resultado.pdfGerado = false; resultado.pdfErro = errPdf.message; } } return resultado;

} module.exports = { enviarNFe, emitirNFSe, emitirNFePropria, importarXmlSieg, gerarDanfe, gerarDanfse, emitirNota, };
```

7. Processamento de Faturas Pendentes

O sieg-odoo-emit.js e o motor de integracao com o Odoo. Ele faz polling periodico das faturas com status "pendente" no campo x_studio_nfe_status (NF-e) ou x_studio_nfse_status (NFS-e), extrai todos os dados necessarios via XML-RPC (empresa com campos l10n_br opcionais, parceiro com fallback de codigo IBGE, linhas com NCM/CST/CFOP/impostos, produtos com campos x_studio_), monta o payload de emissao, chama a API e atualiza o Odoo com o resultado. O sistema e tolerante a campos ausentes no Odoo SaaS (l10n_br) e faz fallback para lookup de cidades por nome/UF.

services/sieg-odoo-emit.js

Processamento de emissoes pendentes via polling Odoo XML-RPC. Le faturas, empresa, parceiro, linhas, produtos. Monta payload, chama sieg-api.js, atualiza Odoo, anexa XML+PDF no chatter. Tolerante a campos ausentes no Odoo 19 SaaS.

sieg-odoo-emit.js (/home/z/my-project/API-Odoo/services/sieg-odoo-emit.js)

```
/** * services/sieg-odoo-emit.js v2.0.0 * ============================================ * Processa emissoes pendentes de NF-e/NFS-e via polling Odoo XML-RPC. * * CAMPOS USADOS (existentes no Odoo do cliente): * account.move: * x_studio_nfe_status (Selection: vazio, pendente, processando, autorizada, erro) * x_studio_nfe_chave (Char) * x_studio_nfe_protocolo (Char) * x_studio_nfse_numero (Char) * x_studio_nfse_status (Selection: vazio, pendente, processando, autorizada, erro) * res.company: * x_studio_sieg_ultimo_nfe (Integer) * x_studio_sieg_ultimo_nfse (Integer) * x_studio_sieg_serie_nfe (Char, default 100) * x_studio_sieg_serie_nfse (Char, default 1) * product.product: * x_studio_c_trib_nac (Char) — Codigo tributo nacional (NFS-e) * x_studio_c_nbs (Char) — Codigo NBS (NFS-e) * x_studio_aliquota_iss (Char) — Aliquota ISS (NFS-e) * x_studio_ibge_code (Char) — Codigo IBGE municipio * * Chatter: apos emissao, XML + PDF (DANFE/DANFSE) aparecem como anexos. */ var xmlrpc = require('xmlrpc'); var { emitirNota } = require('./sieg-api'); var config = require('../config'); // ============================================================ // XML-RPC Helpers // ============================================================ function createClient(url) { var base = url.replace(/\/+$/, ''); var host = base.replace('https://', '').replace('http://', ''); var port = base.startsWith('https') ? 443 : 80; var isSecure = base.startsWith('https'); var createFn = isSecure ? xmlrpc.createSecureClient : xmlrpc.createClient; return { common: createFn({ host: host, path: '/xmlrpc/2/common', port: port }), models: createFn({ host: host, path: '/xmlrpc/2/object', port: port }), }; } function authenticate(client, db, login, password) { return new Promise(function(resolve, reject) { client.common.methodCall('authenticate', [db, login, password, {}], function(err, uid) { if (err) reject(new Error('Auth Odoo falhou: ' + (err.message || JSON.stringify(err)))); else if (uid === false || uid === null) reject(new Error('Credenciais Odoo invalidas')); else resolve(uid); }); }); } function executeKw(client, db, uid, password, model, method, args, kwargs) { return new Promise(function(resolve, reject) { var params = [db, uid, password, model, method, args || []]; if (kwargs) params.push(kwargs); client.models.methodCall('execute_kw', params, function(err, result) { if (err) reject(new Error(model + '.' + method + ': ' + (err.message || JSON.stringify(err)))); else resolve(result); }); }); } // ============================================================ // Main: Process Pending Emissions // ============================================================ async function processPendingEmissions() { var odoo = config.odoo; if (!odoo || !odoo.enabled || !odoo.url) { console.log('[SIEG-EMIT] Odoo nao configurado'); return { processed: 0, reason: 'odoo_not_configured' }; } var client = createClient(odoo.url); var uid = await authenticate(client, odoo.db, odoo.user, odoo.password); var db = odoo.db; var pwd = odoo.password;

console.log('[SIEG-EMIT] Buscando faturas pendentes...'); // Buscar NF-e pendentes var nfeIds = await executeKw(client, db, uid, pwd, 'account.move', 'search', [[ ['move_type', '=', 'out_invoice'], ['x_studio_nfe_status', '=', 'pendente'], ]], { order: 'id asc', limit: 5 }); // Buscar NFS-e pendentes var nfseIds = await executeKw(client, db, uid, pwd, 'account.move', 'search', [[ ['move_type', '=', 'out_invoice'], ['x_studio_nfse_status', '=', 'pendente'], ]], { order: 'id asc', limit: 5 }); var allPending = []; for (var i = 0; i < nfeIds.length; i++) { allPending.push({ id: nfeIds[i], tipo: 'nfe' }); } for (var j = 0; j < nfseIds.length; j++) { allPending.push({ id: nfseIds[j], tipo: 'nfse' }); } if (!allPending.length) { console.log('[SIEG-EMIT] Nenhuma fatura pendente'); return { processed: 0 }; } console.log('[SIEG-EMIT] ' + allPending.length + ' fatura(s) pendente(s)'); var results = []; for (var k = 0; k < allPending.length; k++) { var item = allPending[k]; try { var r = await processOne(client, db, uid, pwd, item.id, item.tipo); results.push(r); } catch (err) { console.error('[SIEG-EMIT] ERRO fatura ' + item.id + ':', err.message); await safeUpdateError(client, db, uid, pwd, item.id, item.tipo, err.message); results.push({ move_id: item.id, sucesso: false, tipo: item.tipo, erro: err.message }); } } var ok = results.filter(function(r) { return r.sucesso; }).length; console.log('[SIEG-EMIT] Concluido: ' + ok + '/' + results.length + ' sucesso'); return { processed: results.length, sucesso: ok, detalhes: results }; } // ============================================================ // Process Single Invoice // ============================================================ async function processOne(client, db, uid, pwd, moveId, tipo) { console.log('[SIEG-EMIT] --- Fatura ID: ' + moveId + ' | Tipo: ' + tipo + ' ---'); // 1. Read account.move (sem invoice_line_ids — pode ser computed nao-stored no Odoo 19) var moves = await executeKw(client, db, uid, pwd, 'account.move', 'read', [[moveId], [ 'name', 'partner_id', 'company_id', 'invoice_date', 'date', 'amount_total', 'narration', 'x_studio_nfe_status', 'x_studio_nfse_status', 'payment_state', ]]); if (!moves || !moves.length) throw new Error('Fatura ' + moveId + ' nao encontrada'); var move = moves[0]; console.log('[SIEG-EMIT] Fatura: ' + move.name + ' | R$ ' + (move.amount_total || 0)); // 2. Read company var companyId = tupId(move.company_id); var company = await readCompany(client, db, uid, pwd, companyId); console.log('[SIEG-EMIT] [EMITENTE] CNPJ=' + company.cnpj_cpf + ' IE=' + company.inscr_est + ' cMun=' + company.city_ibge_code + ' xMun=' + company.city + ' UF=' + company.state + ' xLgr=' + company.street + ' nro=' + company.number); // 3. Read partner var partnerId = tupId(move.partner_id); if (!partnerId) throw new Error('Fatura sem parceiro'); var partner = await readPartner(client, db, uid, pwd, partnerId); console.log('[SIEG-EMIT] [DESTINATARIO] CNPJ=' + partner.cnpj_cpf + '
```

```
xNome=' + partner.xNome + ' cMun=' + partner.city_ibge_code + ' xMun=' + partner.city + ' UF=' + partner.state + ' xLgr=' + partner.street + ' nro=' + partner.number); // 4. Read invoice lines diretamente de account.move.line (evita computed field) var allLineIds = await executeKw(client, db, uid, pwd, 'account.move.line', 'search', [[ ['move_id', '=', moveId], ]], { order: 'id asc' }); console.log('[SIEG-EMIT] Linhas encontradas na fatura: ' + allLineIds.length);

var rawLines = await executeKw(client, db, uid, pwd, 'account.move.line', 'read', [allLineIds, [ 'display_type', 'product_id', 'name', 'quantity', 'price_unit', 'price_subtotal', 'tax_ids', 'discount', ]]); // Debug: log todas as linhas for (var li = 0; li < rawLines.length; li++) { var l = rawLines[li]; var pName = 'N/A'; if (l.product_id) pName = Array.isArray(l.product_id) ? l.product_id[1] : String(l.product_id); console.log('[SIEG-EMIT] Line ' + li + ': display_type=' + JSON.stringify(l.display_type) + ' product=' + pName + ' qty=' + l.quantity + ' price=' + l.price_unit); } // Filtro principal: linhas com produto associado (mais confiavel que display_type) var invoiceLines = rawLines.filter(function(l) { return l.product_id; }); // Fallback: se nenhuma tem product_id, tenta por display_type if (!invoiceLines.length) { invoiceLines = rawLines.filter(function(l) { var dt = l.display_type; return !dt || dt === 'product' || dt === 'service'; }); } if (!invoiceLines.length) throw new Error('Fatura sem linhas de produto/servico (total: ' + rawLines.length + ', com product: 0)'); console.log('[SIEG-EMIT] Linhas de produto: ' + invoiceLines.length); // 5. Get next NF number from company var serie, numField, nextNum; if (tipo === 'nfe') { serie = String(company.x_studio_sieg_serie_nfe || '100'); numField = 'x_studio_sieg_ultimo_nfe'; } else { serie = String(company.x_studio_sieg_serie_nfse || '1'); numField = 'x_studio_sieg_ultimo_nfse'; } nextNum = (parseInt(company[numField]) || 0) + 1; console.log('[SIEG-EMIT] Serie: ' + serie + ' | nNF: ' + nextNum); // 6. Build line data with taxes + product x_studio_ fields var linesData = []; for (var i = 0; i < invoiceLines.length; i++) { var ld = await buildLineData(client, db, uid, pwd, invoiceLines[i]); linesData.push(ld); } // 7. Build emission payload // Nota: usar amount_untaxed (sem impostos) para NF-e com CSOSN 103 // pois vNF deve ser igual a soma dos componentes (vProd + vICMS + ...) // e para CSOSN 103 o ICMS e 0, entao vNF = vProd var vProdSum = 0; for (var li = 0; li < linesData.length; li++) { vProdSum += parseFloat(linesData[li].price_subtotal) || 0; } // Pagamentos: NF-e 4.00 — vPag deve ser igual a vNF (total SEM IBS/CBS). // IBS/CBS sao tributos "por fora" e nao entram no bloco <pag>. // Se vPag > vNF sem vTroco declarado -> cStat 866. var pagamentos = [{ tPag: '15', // PIX vPag: String(vProdSum.toFixed(2)), }]; var emitData = { company: company, partner: partner, order: { name: move.name, number: String(nextNum), date_order: move.invoice_date || move.date, amount_total: vProdSum, // vNF (sem IBS/CBS) — o XML calcula vNFTot internamente note: move.narration || '', }, lines: linesData, pagamentos: pagamentos, config: { serie: serie, tpAmb: process.env.SIEG_TP_AMB || config.sieg.tpAmb || '2', natOp: 'Venda de Mercadoria', mod: tipo === 'nfe' ? '55' : '01', }, tipo: tipo, };

// NFS-e: build service block from product x_studio_ fields if (tipo === 'nfse') { emitData.service = buildServiceBlock(linesData, move); } // 8. Set status to 'processando' var processVals = {}; if (tipo === 'nfe') processVals.x_studio_nfe_status = 'processando'; else processVals.x_studio_nfse_status = 'processando'; await executeKw(client, db, uid, pwd, 'account.move', 'write', [[moveId], processVals]); // 9. Call SIEG API console.log('[SIEG-EMIT] Enviando ao SIEG...'); var resultado; try { resultado = await emitirNota(emitData); } catch (siegErr) { // Captura erros HTTP (ex: 409 com detalhes no body) var siegDetail = siegErr.response && siegErr.response.data; var errMsg = 'SIEG HTTP ' + (siegErr.response ? siegErr.response.status : 'erro') + ': '; if (siegDetail) { // Tentar extrair ErrorMessage do formato SIEG (PascalCase) var siegMsg = siegDetail.ErrorMessage || siegDetail.Message || siegDetail.message || ''; if (siegMsg) { errMsg += siegMsg; } else { errMsg += typeof siegDetail === 'string' ? siegDetail : JSON.stringify(siegDetail).slice(0, 800); } } else { errMsg += siegErr.message; } console.error('[SIEG-EMIT] Erro SIEG:', errMsg); throw new Error(errMsg); } console.log('[SIEG-EMIT] SIEG retornou - sucesso: ' + resultado.sucesso + (resultado.httpStatus ? ' (HTTP ' + resultado.httpStatus + ')' : '')); // Se SIEG retornou erro (IsSuccess=false), extrair mensagem detalhada if (!resultado.sucesso && resultado.erro) { console.error('[SIEG-EMIT] SIEG rejeitou o XML: ' + resultado.erro); } // 10. Parse result var info = parseResult(resultado, tipo); var xmlRetornoText = ''; var rawXmlAuth = resultado.resposta && (resultado.resposta.Xml || resultado.resposta.xml || resultado.resposta.xmlBase64); if (rawXmlAuth) { xmlRetornoText = (typeof rawXmlAuth === 'string' && rawXmlAuth.length > 200) ? rawXmlAuth : Buffer.from(rawXmlAuth, 'base64').toString('utf-8'); } // 11. Update account.move with results var updateVals = {}; if (info.sucesso) { if (tipo === 'nfe') { updateVals.x_studio_nfe_status = 'autorizada'; updateVals.x_studio_nfe_chave = info.chave; updateVals.x_studio_nfe_protocolo = info.protocolo; } else { updateVals.x_studio_nfse_status = 'autorizada'; updateVals.x_studio_nfse_numero = String(nextNum); } // Increment NF number (only on authorization) try { var companyUpdate = {}; companyUpdate[numField] = nextNum; await executeKw(client, db, uid, pwd, 'res.company', 'write', [[companyId], companyUpdate]); console.log('[SIEG-EMIT] Sequencia ' + numField + ' = ' + nextNum); } catch (seqErr) { console.error('[SIEG-EMIT] Erro ao atualizar sequencia:', seqErr.message); } } else { // Falha: NUNCA gravar texto livre num campo Selection (o Odoo descarta e a // fatura some da fila). Grava um valor valido ('erro' ou 'pendente') e // mantem o motivo detalhado no chatter. var errStatus = statusOnError(); if (tipo === 'nfe') { updateVals.x_studio_nfe_status = errStatus; } else {

updateVals.x_studio_nfse_status = errStatus; } console.warn('[SIEG-EMIT] Emissao NAO autorizada -> status "' + errStatus + '" (motivo: ' + (info.cStat ? info.cStat + ' - ' : '') + (info.motivo || 'n/d') + ')'); } await writeStatusSafe(client, db, uid, pwd, moveId, tipo, updateVals); console.log('[SIEG-EMIT] Fatura atualizada'); // 12. Post XML + PDF to Chatter await postChatterResult(client, db, uid, pwd, moveId, move.name, tipo, info, resultado, xmlRetornoText); return { move_id: moveId, sucesso: info.sucesso, tipo: tipo, chave: info.chave, numero: nextNum, serie: serie, }; } // ============================================================ // Post XML + PDF to Odoo Chatter // ============================================================ async function postChatterResult(client, db, uid, pwd, moveId, moveName, tipo, info, resultado, xmlRetornoText) { var attachIds = []; var safeName = (moveName || 'Fatura').replace(/[^a-zA-Z0-9\-_]/g, '_'); var nfLabel = tipo === 'nfe' ? 'NF-e' : 'NFS-e'; var xmlContent = xmlRetornoText || resultado.xmlEnviado || ''; var pdfBase64 = resultado.pdfBase64 || ''; // Attach XML if (xmlContent) { try { var xmlB64 = Buffer.from(xmlContent, 'utf-8').toString('base64'); var xmlAttach = await executeKw(client, db, uid, pwd, 'ir.attachment', 'create', [{ name: nfLabel + '_' + safeName + '.xml', datas: xmlB64, res_model: 'account.move', res_id: moveId, mimetype: 'application/xml', }]); attachIds.push(xmlAttach); console.log('[SIEG-EMIT] XML anexado (ID: ' + xmlAttach + ')'); } catch (e) { console.error('[SIEG-EMIT] Erro anexar XML:', e.message); } } // Attach PDF (DANFE / DANFSE) if (pdfBase64) { try { var pdfLabel = tipo === 'nfe' ? 'DANFE' : 'DANFSE'; var pdfAttach = await executeKw(client, db, uid, pwd, 'ir.attachment', 'create', [{ name: pdfLabel + '_' + safeName + '.pdf', datas: pdfBase64, res_model: 'account.move', res_id: moveId, mimetype: 'application/pdf', }]); attachIds.push(pdfAttach); console.log('[SIEG-EMIT] PDF anexado (ID: ' + pdfAttach + ')'); } catch (e) { console.error('[SIEG-EMIT] Erro anexar PDF:', e.message); } } // Post chatter message var body = ''; if (info.sucesso) { body = '<b>' + nfLabel + ' Emitida com Sucesso!</b><br/>'; if (tipo === 'nfe') { body += 'Chave: ' + info.chave + '<br/>'; body += 'Protocolo: ' + info.protocolo + '<br/>'; } else { body += 'Numero: ' + info.numero + '<br/>'; } body += 'Status: ' + info.cStat + ' - ' + info.motivo; body += '<br/><br/><i>Arquivos anexados: XML e ' +
```

```
(tipo === 'nfe' ? 'DANFE' : 'DANFSE') + ' (PDF)</i>'; } else { body = '<b>Erro na Emissao de ' + nfLabel + '</b><br/>'; body += 'Status: ' + info.cStat + ' - ' + info.motivo; } try { var msgVals = { model: 'account.move',

res_id: moveId, body: body, message_type: 'comment', }; if (attachIds.length > 0) { msgVals.attachment_ids = [[6, 0, attachIds]]; } await executeKw(client, db, uid, pwd, 'mail.message', 'create', [msgVals]); console.log('[SIEG-EMIT] Mensagem postada no chatter'); } catch (e) { console.error('[SIEG-EMIT] Erro postar chatter:', e.message); } } // ============================================================ // Read Company // ============================================================ async function readCompany(client, db, uid, pwd, companyId) { var recs = await executeKw(client, db, uid, pwd, 'res.company', 'read', [[companyId], [ 'name', 'partner_id', 'street', 'city', 'state_id', 'zip', 'phone', 'email', 'vat', 'x_studio_sieg_ultimo_nfe', 'x_studio_sieg_ultimo_nfse', 'x_studio_sieg_serie_nfe', 'x_studio_sieg_serie_nfse', ]]); if (!recs || !recs.length) throw new Error('Empresa ' + companyId + ' nao encontrada'); var c = recs[0]; var pId = tupId(c.partner_id); var p = {}; if (pId) { try { p = await readPartnerSafe(client, db, uid, pwd, pId); } catch (e) { console.warn('[SIEG-EMIT] Erro ao ler partner da empresa:', e.message); } } var stateCode = '', stateIbge = ''; var stateId = tupId(c.state_id || p.state_id); if (stateId) { try { var sts = await executeKw(client, db, uid, pwd, 'res.country.state', 'read', [[stateId], ['code', 'ibge_code']]); if (sts && sts[0]) { stateCode = sts[0].code || ''; stateIbge = sts[0].ibge_code || ''; } } catch (e) {} } var cityIbge = ''; var cityRef = p.l10n_br_city_id || p.city_id; if (cityRef && Array.isArray(cityRef)) cityIbge = await readCityIbge(client, db, uid, pwd, cityRef[0]); return { cnpj_cpf: p.cnpj_cpf || c.vat || '22603750000190', legal_name: p.legal_name || c.name || 'AJL FERRO E ACO LTDA', name: c.name || 'AJL', inscr_est: p.inscr_est || '9069585890', street: c.street || p.street || '', number: p.number || 'S/N', street2: p.street2 || p.district || '', city: c.city || p.city || 'Curitiba', state: stateCode || 'PR', zip: c.zip || p.zip || '', city_ibge_code: cityIbge || '4106902', state_ibge: stateIbge || '41', crt: '3', phone: c.phone || p.phone || '', email: c.email || p.email || '', x_studio_sieg_ultimo_nfe: c.x_studio_sieg_ultimo_nfe || 0, x_studio_sieg_ultimo_nfse: c.x_studio_sieg_ultimo_nfse || 0, x_studio_sieg_serie_nfe: c.x_studio_sieg_serie_nfe || '100', x_studio_sieg_serie_nfse: c.x_studio_sieg_serie_nfse || '1', }; } // ============================================================ // Read Partner (safe for Odoo 19 SaaS — l10n_br fields may not exist) // ============================================================ var PARTNER_SAFE_FIELDS = [ 'name', 'vat', 'street', 'street2', 'city', 'state_id', 'zip', 'phone', 'email', 'is_company', 'city_id', 'country_id', ]; var PARTNER_BR_FIELDS = ['cnpj_cpf', 'inscr_est', 'legal_name', 'number', 'l10n_br_city_id', 'district']; /** * Leitura segura de res.partner: campos core + l10n_br opcionais. * No Odoo 19 SaaS, campos como cnpj_cpf, inscr_est, legal_name, number,

* l10n_br_city_id, district podem nao existir. Fazemos duas leituras: * 1. Campos core (sempre existem) * 2. Campos l10n_br (try/catch, ignora se falhar) */ async function readPartnerSafe(client, db, uid, pwd, partnerId, extraFields) { var fields = PARTNER_SAFE_FIELDS.slice(); if (extraFields) fields = fields.concat(extraFields); var recs = await executeKw(client, db, uid, pwd, 'res.partner', 'read', [[partnerId], fields]); if (!recs || !recs.length) throw new Error('Parceiro ' + partnerId + ' nao encontrado'); var p = recs[0]; // Try Brazilian fields separately try { var brRecs = await executeKw(client, db, uid, pwd, 'res.partner', 'read', [[partnerId], PARTNER_BR_FIELDS]); if (brRecs && brRecs[0]) { for (var k in brRecs[0]) { if (brRecs[0][k] !== undefined) p[k] = brRecs[0][k]; } } } catch (e) { console.log('[SIEG-EMIT] Campos l10n_br nao disponiveis, usando vat como CNPJ/CPF'); } return p; } async function readPartner(client, db, uid, pwd, partnerId) { var p = await readPartnerSafe(client, db, uid, pwd, partnerId); var stateCode = ''; var stId = tupId(p.state_id); if (stId) { try { var sts = await executeKw(client, db, uid, pwd, 'res.country.state', 'read', [[stId], ['code']]); if (sts && sts[0]) stateCode = sts[0].code || ''; } catch (e) {} } var cityIbge = ''; var cityRef = p.l10n_br_city_id || p.city_id; if (cityRef && Array.isArray(cityRef)) cityIbge = await readCityIbge(client, db, uid, pwd, cityRef[0]); // Fallback 1: buscar res.city por nome + estado if (!cityIbge && p.city && stateCode) { cityIbge = await searchCityIbge(client, db, uid, pwd, p.city, stId); } // Fallback 2: tabela embutida de cidades principais if (!cityIbge && p.city && stateCode) { cityIbge = lookupIbgeFallback(p.city, stateCode); } if (!cityIbge) { console.warn('[SIEG-EMIT] cMun vazio para parceiro ' + partnerId + ' (' + (p.name || '?') + '): cidade=' + (p.city || '?') + ' UF=' + (stateCode || '?')); } return { cnpj_cpf: p.cnpj_cpf || p.vat || '', legal_name: p.legal_name || p.name || '', xNome: p.name || '', inscr_est: p.inscr_est || '', street: p.street || '', number: p.number || 'S/N', street2: p.street2 || p.district || '', city: p.city || '', state: stateCode, zip: p.zip || '', city_ibge_code: cityIbge, phone: p.phone || '', email: p.email || '', is_consumer: !p.is_company, district: p.district || p.street2 || '', }; } // ============================================================ // NCM Extraction Helper // ============================================================ /** * Extrai codigo NCM (8 digitos) de um valor many2one do Odoo. * Aceita: [id, "7308.90.90"], [id, "7308.90.90 - Descricao"], string pura, numero */ function extractNcmFromRef(ref) { if (!ref) return ''; var str = ''; if (Array.isArray(ref)) {

str = String(ref[1] || ref[0] || ''); } else if (typeof ref === 'string') { str = ref; } else if (typeof ref === 'number') { return String(ref).length === 8 ? String(ref) : ''; } else { str = String(ref); } // Tentar regex NNNN.NN.NN var m = str.match(/(\d{4})\.(\d{2})\.(\d{2})/); if (m) return m[1] + m[2] + m[3]; // Fallback: so digitos var digits = str.replace(/\D/g, ''); return digits.length === 8 ? digits : ''; } // ============================================================ // Build Line Data (with product x_studio_ fields for NFS-e) // ============================================================ async function buildLineData(client, db, uid, pwd, line) { var productId = tupId(line.product_id); var productName = Array.isArray(line.product_id) ? line.product_id[1] : (line.name || 'Item'); var defaultCode = '', barcode = '', ncm = '', detailedType = 'product', uomName = 'UN'; var prodStudio = {}; // x_studio_ fields from product if (productId) { try { // Campos core do produto (sempre existem) — SEM detailed_type (pode nao existir no SaaS) var prods = await executeKw(client, db, uid, pwd, 'product.product', 'read', [[productId], [ 'default_code', 'barcode', 'name', 'uom_id', ]]); if (prods && prods[0]) { var pr = prods[0]; defaultCode = pr.default_code || ''; barcode = pr.barcode || ''; productName = pr.name || productName; if (pr.uom_id && Array.isArray(pr.uom_id)) uomName = pr.uom_id[1] || 'UN'; } // Campo detailed_type (pode nao existir no Odoo 19 SaaS) try { var prodsDt = await executeKw(client, db, uid, pwd, 'product.product', 'read', [[productId], ['detailed_type']]); if (prodsDt && prodsDt[0] && prodsDt[0].detailed_type) detailedType = prodsDt[0].detailed_type; } catch (eDt) { console.log('[SIEG-EMIT] detailed_type nao disponivel, usando product'); } // NCM do produto // Campo correto no Odoo 19 SaaS com l10n_br: l10n_br_ncm_code_id (many2one -> l10n_br.ncm.code) // Este campo esta em product.template (nao product.product) // Estrategia: // T1: Ler product_tmpl_id + l10n_br_ncm_code_id do template // T2: Tentar ncm_id em product.product (l10n_br_fiscal — pode nao existir) // T3: fields_get para descobrir campo NCM alternativo // === T1: Ler l10n_br_ncm_code_id do product.template === try { console.log('[SIEG-EMIT-NCM] [T1] Lendo product_tmpl_id + l10n_br_ncm_code_id...'); var prodsTmpl = await executeKw(client, db, uid, pwd, 'product.product', 'read', [[productId], ['product_tmpl_id']]); if (prodsTmpl && prodsTmpl[0] && prodsTmpl[0].product_tmpl_id) { var tmplId = tupId(prodsTmpl[0].product_tmpl_id); console.log('[SIEG-EMIT-NCM] [T1] product_tmpl_id: ' + tmplId); var tmplNcm = await executeKw(client, db, uid, pwd, 'product.template', 'read', [[tmplId], ['l10n_br_ncm_code_id']]); console.log('[SIEG-EMIT-NCM] [T1] l10n_br_ncm_code_id bruto: ' + JSON.stringify(tmplNcm && tmplNcm[0] ? tmplNcm[0].l10n_br_ncm_code_id : 'N/A')); if
```

```
(tmplNcm && tmplNcm[0] && tmplNcm[0].l10n_br_ncm_code_id) { ncm = extractNcmFromRef(tmplNcm[0].l10n_br_ncm_code_id); if (ncm) console.log('[SIEG-EMIT-NCM] [T1] *** NCM ENCONTRADO: ' + ncm + ' ***'); } else { console.warn('[SIEG-EMIT-NCM] [T1] l10n_br_ncm_code_id vazio no template ' + tmplId); } } else { console.warn('[SIEG-EMIT-NCM] [T1] product_tmpl_id vazio para produto ' + productId); } } catch (eT1) { console.error('[SIEG-EMIT-NCM] [T1] FALHOU: ' + eT1.message); } // === T2: Tentar ncm_id em product.product (l10n_br_fiscal) === if (!ncm) { try {

console.log('[SIEG-EMIT-NCM] [T2] Tentando ncm_id em product.product...'); var prodsNcm = await executeKw(client, db, uid, pwd, 'product.product', 'read', [[productId], ['ncm_id']]); if (prodsNcm && prodsNcm[0] && prodsNcm[0].ncm_id) { ncm = extractNcmFromRef(prodsNcm[0].ncm_id); if (ncm) console.log('[SIEG-EMIT-NCM] [T2] NCM via ncm_id: ' + ncm); } } catch (eT2) { console.log('[SIEG-EMIT-NCM] [T2] ncm_id nao disponivel: ' + eT2.message); } } // === T3: Descobrir campos NCM via fields_get === if (!ncm) { try { console.log('[SIEG-EMIT-NCM] [T3] Buscando campos NCM via fields_get...'); var fieldsInfo = await executeKw(client, db, uid, pwd, 'product.template', 'fields_get', [ ['l10n_br_ncm_code_id', 'ncm_id', 'l10n_br_ncm_id', 'ncm_code_id'], ['type', 'relation', 'string'] ]); console.log('[SIEG-EMIT-NCM] [T3] fields_get: ' + JSON.stringify(fieldsInfo)); var fieldNames = Object.keys(fieldsInfo || {}); for (var fi = 0; fi < fieldNames.length; fi++) { var fname = fieldNames[fi]; if (fieldsInfo[fname].type === 'many2one') { try { var fv = await executeKw(client, db, uid, pwd, 'product.template', 'read', [[tmplId], [fname]]); if (fv && fv[0] && fv[0][fname]) { var extracted = extractNcmFromRef(fv[0][fname]); if (extracted) { ncm = extracted; console.log('[SIEG-EMIT-NCM] [T3] NCM via ' + fname + ': ' + ncm); break; } } } catch (ef3) { console.log('[SIEG-EMIT-NCM] [T3] ' + fname + ' falhou: ' + ef3.message); } } } } catch (eT3) { console.error('[SIEG-EMIT-NCM] [T3] FALHOU: ' + eT3.message); } } // x_studio fields documentados neste Odoo (NFS-e e customizacoes) try { var prods2 = await executeKw(client, db, uid, pwd, 'product.product', 'read', [[productId], [ 'x_studio_c_trib_nac', 'x_studio_c_nbs', 'x_studio_aliquota_iss', 'x_studio_ibge_code', ]]); if (prods2 && prods2[0]) { var pr2 = prods2[0]; prodStudio = { c_trib_nac: pr2.x_studio_c_trib_nac || '', c_nbs: pr2.x_studio_c_nbs || '', aliquota_iss: pr2.x_studio_aliquota_iss || '', ibge_code: pr2.x_studio_ibge_code || '', }; } } catch (e2) { console.log('[SIEG-EMIT] Campos x_studio do produto nao disponiveis: ' + e2.message); } // Fallback final: NCM do env var (se configurado) if (!ncm && process.env.SIEG_DEFAULT_NCM) { ncm = String(process.env.SIEG_DEFAULT_NCM).replace(/\D/g, ''); console.log('[SIEG-EMIT-NCM] NCM via SIEG_DEFAULT_NCM env: ' + ncm); } // ALERTA se NCM ainda vazio if (!ncm) { console.error('[SIEG-EMIT-NCM] *** NCM VAZIO para produto ' + productId + ' (' + productName + ') — XML sera invalido ***'); } } catch (e) { console.warn('[SIEG-EMIT] Erro ao ler produto ' + productId + ':', e.message); } } // Tax extraction var tax = await extractTaxes(client, db, uid, pwd, line); var lineData = { cProd: defaultCode, barcode: barcode, product_name: productName, xProd: productName, ncm: ncm, cfop: '5102', uom: uomName,

qty: line.quantity || 0, price_unit: line.price_unit || 0, price_subtotal: line.price_subtotal || (line.quantity * line.price_unit), detailed_type: detailedType, // Tributação Lucro Real (padrão) — sobrescrito se Odoo tiver impostos configurados csosn: tax.csosn || '', cst_icms: tax.cst_icms || '00', mod_bc: tax.mod_bc || '3', orig: '0', vbc_icms: String(tax.vbc || 0), vicms: String(tax.vicms || 0), picms: String(tax.picms || 0), cst_pis: tax.cst_pis || '01', vbc_pis: String(tax.vbc_pis || 0), ppis: String(tax.ppis || 0), vpis: String(tax.vpis || 0), cst_cofins: tax.cst_cofins || '01', vbc_cofins: String(tax.vbc_cofins || 0), pcofins: String(tax.pcofins || 0), vcofins: String(tax.vcofins || 0), // NFS-e fields from product x_studio_c_trib_nac: prodStudio.c_trib_nac, x_studio_c_nbs: prodStudio.c_nbs, x_studio_aliquota_iss: prodStudio.aliquota_iss, x_studio_ibge_code: prodStudio.ibge_code, }; // === LOG DETALHADO POR CAMPO (para debug de XML invalido) === console.log('[SIEG-EMIT-LINE] === Dados da linha (product ' + productId + ') ==='); console.log('[SIEG-EMIT-LINE] cProd: ' + JSON.stringify(lineData.cProd) + (lineData.cProd ? '' : ' *** VAZIO ***')); console.log('[SIEG-EMIT-LINE] xProd: ' + JSON.stringify(lineData.xProd)); console.log('[SIEG-EMIT-LINE] NCM: ' + JSON.stringify(lineData.ncm) + (lineData.ncm ? '' : ' *** VAZIO ***')); console.log('[SIEG-EMIT-LINE] CFOP: ' + JSON.stringify(lineData.cfop)); console.log('[SIEG-EMIT-LINE] uCom: ' + JSON.stringify(lineData.uom)); console.log('[SIEG-EMIT-LINE] qCom: ' + lineData.qty); console.log('[SIEG-EMIT-LINE] vUnCom: ' + lineData.price_unit); console.log('[SIEG-EMIT-LINE] vProd: ' + lineData.price_subtotal); console.log('[SIEG-EMIT-LINE] CSOSN: ' + JSON.stringify(lineData.csosn)); console.log('[SIEG-EMIT-LINE] CST_ICMS: ' + JSON.stringify(lineData.cst_icms)); console.log('[SIEG-EMIT-LINE] modBC: ' + JSON.stringify(lineData.mod_bc)); console.log('[SIEG-EMIT-LINE] vBC_ICMS: ' + lineData.vbc_icms); console.log('[SIEG-EMIT-LINE] vICMS: ' + lineData.vicms); console.log('[SIEG-EMIT-LINE] pICMS: ' + lineData.picms); console.log('[SIEG-EMIT-LINE] CST_PIS: ' + JSON.stringify(lineData.cst_pis)); console.log('[SIEG-EMIT-LINE] vBC_PIS: ' + lineData.vbc_pis); console.log('[SIEG-EMIT-LINE] pPIS: ' + lineData.ppis); console.log('[SIEG-EMIT-LINE] vPIS: ' + lineData.vpis); console.log('[SIEG-EMIT-LINE] CST_COFINS: ' + JSON.stringify(lineData.cst_cofins)); console.log('[SIEG-EMIT-LINE] vBC_COFINS: ' + lineData.vbc_cofins); console.log('[SIEG-EMIT-LINE] pCOFINS: ' + lineData.pcofins); console.log('[SIEG-EMIT-LINE] vCOFINS: ' + lineData.vcofins); return lineData; } // ============================================================ // Build Service Block for NFS-e (reads from product x_studio_ fields) // ============================================================ function buildServiceBlock(linesData, move) { // Use first service line's x_studio_ fields var cTribNac = ''; var cNBS = '999999999'; var pAliq = '2.00'; var cIntContrib = ''; var xDescServ = move.narration || 'Servico prestado conforme contrato'; for (var i = 0; i < linesData.length; i++) { var l = linesData[i]; if (l.x_studio_c_trib_nac) cTribNac = l.x_studio_c_trib_nac; if (l.x_studio_c_nbs) cNBS = l.x_studio_c_nbs; if (l.x_studio_aliquota_iss) pAliq = String(l.x_studio_aliquota_iss); if (l.x_studio_ibge_code) cIntContrib = l.x_studio_ibge_code; } if (!cTribNac) cTribNac = '140101'; return { cTribNac: cTribNac, xDescServ: xDescServ, cNBS: cNBS, cIntContrib: cIntContrib,

valor: move.amount_total, pAliq: pAliq, tpRetISSQN: '1', pTotTribFed: '13.45', pTotTribEst: '0.00', pTotTribMun: '3.29', }; } // ============================================================ // Tax Extraction // ============================================================ async function extractTaxes(client, db, uid, pwd, line) { // Padrão: Lucro Real (CRT 3) — CST 00, ICMS 18% PR interno // PIS/COFINS não-cumulativo com base no valor do produto var base = parseFloat(line.price_subtotal) || 0; var result = { csosn: '', cst_icms: '00', mod_bc: '3', vbc: base, vicms: base * 0.18, picms: 18.00, cst_pis: '01', vbc_pis: base, ppis: 1.65, vpis: base * 0.0165, cst_cofins: '01', vbc_cofins: base, pcofins: 7.60, vcofins: base * 0.076, }; var taxIds = (line.tax_ids || []).map(function(t) { return Array.isArray(t) ? t[0] : t; }).filter(Boolean); if (!taxIds.length) return result; try { var taxes = await executeKw(client, db, uid, pwd, 'account.tax', 'read', [taxIds, [ 'name', 'amount', 'amount_type', 'description', ]]); if (!taxes || !taxes.length) return result; var totalIcms = 0, totalPis = 0, totalCofins = 0; var foundCsosn = ''; for (var i = 0; i < taxes.length; i++) { var tax = taxes[i]; var n = ((tax.name || tax.description || '').toUpperCase()); var amt = parseFloat(tax.amount) || 0; if (n.indexOf('ICMS') >= 0 || n.indexOf('CSOSN') >= 0) { totalIcms += amt; var m = n.match(/(10[0-9]|20[0-9]|300|400|500|900)/); if (m) foundCsosn = m[1]; } else if (n.indexOf('PIS') >= 0) { totalPis += amt; } else if (n.indexOf('COFINS') >= 0) { totalCofins += amt; } } // Sobrescreve defaults apenas se o Odoo tiver impostos reais configurados (> 0) if (foundCsosn) { result.csosn = foundCsosn; result.cst_icms = ''; } if (totalIcms > 0) { result.picms = totalIcms; result.vicms = base * (totalIcms /
```

```
100); } if (totalPis > 0) { result.ppis = totalPis; result.vpis = base * (totalPis / 100); } if (totalCofins > 0) { result.pcofins = totalCofins; result.vcofins = base * (totalCofins / 100); } } catch (e) { console.warn('[SIEG-EMIT] Erro ao extrair impostos:', e.message); } return result; } //
============================================================

// Helpers // ============================================================ function parseResult(resultado, tipo) { var resp = resultado.resposta || {}; var chave = '', protocolo = '', cStat = '', motivo = '', numero = '', sucesso = false; // Se SIEG retornou HTTP 4xx, usar ErrorMessage diretamente como motivo var siegErro = resultado.erro || ''; if (resultado.httpStatus && resultado.httpStatus >= 400 && resultado.httpStatus < 500) { // Erro de validacao do XML — extrair mensagem do SIEG motivo = siegErro || 'Erro HTTP ' + resultado.httpStatus; if (tipo === 'nfe') { cStat = String(resultado.statusCode || resp.cStat || resp.status || ''); } return { sucesso: false, chave: chave, protocolo: protocolo, cStat: cStat, motivo: motivo, numero: numero }; } if (tipo === 'nfe') { chave = resp.chNFe || resp.chave || resp.ChaveXml || ''; protocolo = resp.nProt || resp.protocolo || ''; cStat = String(resp.cStat || resp.status || ''); motivo = resp.xMotivo || resp.motivo || (siegErro || 'Processado'); sucesso = (cStat === '100' || cStat === '104' || cStat === '150'); } else { chave = resp.Chave || resp.chave || ''; protocolo = resp.Protocolo || resp.protocolo || ''; numero = resp.NumeroNfse || resp.nDFSe || resp.numero || ''; cStat = String(resp.CodigoVerificacao || resp.codigo || ''); motivo = resp.Motivo || resp.motivo || siegErro || (resultado.sucesso ? 'Autorizada' : 'Erro na emissao'); sucesso = resultado.sucesso; } return { sucesso: sucesso, chave: chave, protocolo: protocolo, cStat: cStat, motivo: motivo, numero: numero }; } async function readCityIbge(client, db, uid, pwd, cityId) { try { var c = await executeKw(client, db, uid, pwd, 'res.city', 'read', [[cityId], ['ibge_code']]); if (c && c[0] && c[0].ibge_code) return String(c[0].ibge_code); } catch (e) {} try { var c2 = await executeKw(client, db, uid, pwd, 'l10n_br.city', 'read', [[cityId], ['ibge_code']]); if (c2 && c2[0] && c2[0].ibge_code) return String(c2[0].ibge_code); } catch (e2) {} return ''; } /** * Busca IBGE de cidade pelo nome e estado_id no Odoo */ async function searchCityIbge(client, db, uid, pwd, cityName, stateId) { if (!cityName || !stateId) return ''; try { var domain = [['name', 'ilike', cityName]]; if (stateId) domain.push(['state_id', '=', stateId]); var ids = await executeKw(client, db, uid, pwd, 'res.city', 'search', [domain], { limit: 5 }); if (ids && ids.length > 0) { // Ler ibge_code de todos os resultados encontrados var cities = await executeKw(client, db, uid, pwd, 'res.city', 'read', [ids, ['name', 'ibge_code']]); if (cities) { for (var i = 0; i < cities.length; i++) { if (cities[i].ibge_code) { console.log('[SIEG-EMIT] IBGE encontrado por busca: ' + cities[i].name + ' = ' + cities[i].ibge_code); return String(cities[i].ibge_code); } } } } catch (e) { console.log('[SIEG-EMIT] Busca res.city falhou: ' + e.message); } return ''; } /** * Tabela de fallback para cidades principais do BR * Chave: 'CIDADE/UF' (normalizado) -> codigo IBGE 7 digitos */ var IBGE_FALLBACK = { 'CURITIBA/PR': '4106902',

'SAO PAULO/SP': '3550308', 'RIO DE JANEIRO/RJ': '3304557', 'BELO HORIZONTE/MG': '3106200', 'PORTO ALEGRE/RS': '4314902', 'SALVADOR/BA': '2927408', 'BRASILIA/DF': '5300108', 'FORTALEZA/CE': '2304400', 'RECIFE/PE': '2611606', 'CURITIBA/PR': '4106902', 'GUARULHOS/SP': '3518800', 'CAMPINAS/SP': '3509502', 'SAO BERNARDO DO CAMPO/SP': '3548708', 'SANTOS/SP': '3548500', 'RIBEIRAO PRETO/SP': '3543402', 'UBERLANDIA/MG': '3170206', 'LONDRINA/PR': '4113700', 'MARINGA/PR': '4115200', 'PONTA GROSSA/PR': '4119905', 'FOZ DO IGUACU/PR': '4108304', 'CAMPO MOURAO/PR': '4104803', 'CASCAVEL/PR': '4104808', 'JOINVILLE/SC': '4209102', 'FLORIANOPOLIS/SC': '4205407', 'BALNEARIO CAMBORIU/SC': '4202008', 'ITU/SP': '3523909', 'JUNDIAI/SP': '3525904', 'SOROCABA/SP': '3552205', 'SAO JOSE DOS CAMPOS/SP': '3549904', 'SANTO ANDRE/SP': '3548807', 'SAO JOSE DO RIO PRETO/SP': '3549805', 'MANAUS/AM': '1302603', 'BELEM/PA': '1501402', 'GOIANIA/GO': '5208707', 'VITORIA/ES': '3205309', 'VOLTA REDONDA/RJ': '3306305', 'NILOPOLIS/RJ': '3303203', 'MESQUITA/RJ': '3302858', 'DUQUE DE CAXIAS/RJ': '3301702', 'NOVA IGUACU/RJ': '3303500', 'SAO GONCALO/RJ': '3304904', 'MAUA/SP': '3529401', 'DIADEMA/SP': '3513801', 'OSASCO/SP': '3534401', 'CARAPICUIBA/SP': '3509502', 'MOGI DAS CRUZES/SP': '3530607', 'SUZANO/SP': '3552503', 'TABOAO DA SERRA/SP': '3552809', }; function lookupIbgeFallback(cityName, stateCode) { if (!cityName || !stateCode) return ''; var key = (cityName.toUpperCase().trim() + '/' + stateCode.toUpperCase().trim()); var code = IBGE_FALLBACK[key]; if (code) { console.log('[SIEG-EMIT] IBGE via tabela fallback: ' + key + ' = ' + code); } return code || ''; } function tupId(val) { if (Array.isArray(val)) return val[0] || 0; if (typeof val === 'number') return val; return parseInt(val) || 0; } /** * Valor gravado no campo Selection quando a emissao falha. * NFE_STATUS_ON_ERROR=pendente -> a fatura volta para a fila e e retentada. * Padrao 'erro' -> exige reprocessamento manual. */ function statusOnError() { var v = String(process.env.NFE_STATUS_ON_ERROR || 'erro').trim().toLowerCase(); return v === 'pendente' ? 'pendente' : 'erro'; } /** * Grava o status com fallback: se o Odoo recusar o valor da Selection, * tenta 'pendente' para a fatura nunca ficar fora da fila silenciosamente. */ async function writeStatusSafe(client, db, uid, pwd, moveId, tipo, vals) {

var field = tipo === 'nfe' ? 'x_studio_nfe_status' : 'x_studio_nfse_status'; try { await executeKw(client, db, uid, pwd, 'account.move', 'write', [[moveId], vals]); } catch (e) { console.error('[SIEG-EMIT] Falha ao gravar status (' + vals[field] + '): ' + e.message); var fb = {}; fb[field] = 'pendente'; try { await executeKw(client, db, uid, pwd, 'account.move', 'write', [[moveId], fb]); console.warn('[SIEG-EMIT] Status revertido para "pendente" (retentativa possivel)'); } catch (e2) { console.error('[SIEG-EMIT] Falha no fallback de status:', e2.message); } } } async function safeUpdateError(client, db, uid, pwd, moveId, tipo, errMsg) { try { var vals = {}; var st = statusOnError(); if (tipo === 'nfe') vals.x_studio_nfe_status = st; else vals.x_studio_nfse_status = st; await writeStatusSafe(client, db, uid, pwd, moveId, tipo, vals); await executeKw(client, db, uid, pwd, 'mail.message', 'create', [{ model: 'account.move', res_id: moveId, body: '<b>Erro na Emissao de ' + (tipo === 'nfe' ? 'NF-e' : 'NFS-e') + '</b><br/>' + errMsg.substring(0, 500), message_type: 'comment', }]); } catch (e) { console.error('[SIEG-EMIT] Falha ao registrar erro:', e.message); } } // ============================================================ // Process Pending Cancellations (polling para status 'cancelando') // ============================================================ /** * Busca faturas com x_studio_nfe_status = 'cancelando' e envia o * evento de cancelamento à SEFAZ PR, depois atualiza o Odoo. * Chamado pelo runSiegPoll no server.js junto com processPendingEmissions. */ async function processPendingCancellations() { var odoo = config.odoo; if (!odoo || !odoo.enabled || !odoo.url) { return { processed: 0, reason: 'odoo_not_configured' }; } var client = createClient(odoo.url); var uid = await authenticate(client, odoo.db, odoo.user, odoo.password); var db = odoo.db; var pwd = odoo.password; var ids = await executeKw(client, db, uid, pwd, 'account.move', 'search', [[ ['move_type', '=', 'out_invoice'], ['x_studio_nfe_status', '=', 'cancelando'], ]], { order: 'id asc', limit: 5 }); if (!ids.length) return { processed: 0 }; console.log('[SIEG-CANCEL-POLL] ' + ids.length + ' fatura(s) aguardando cancelamento'); var { cancelarNFe } = require('./nfe-cancelamento'); var results = []; for (var i = 0; i < ids.length; i++) { var moveId = ids[i]; try { var r = await cancelOneInvoice(client, db, uid, pwd, moveId, cancelarNFe); results.push(r); } catch (err) { console.error('[SIEG-CANCEL-POLL] ERRO fatura ' + moveId + ':', err.message); try { await executeKw(client, db, uid, pwd, 'account.move', 'write', [[moveId], { x_studio_nfe_status: 'erro' }]); await executeKw(client, db, uid, pwd, 'mail.message', 'create', [{ model: 'account.move', res_id: moveId, body: '<b>Erro no Cancelamento</b><br/>' + err.message.substring(0, 500), message_type: 'comment', }]); } catch (e2) { console.error('[SIEG-CANCEL-POLL] Falha ao gravar erro:', e2.message); } results.push({ move_id: moveId, sucesso: false, erro: err.message }); } }
```

```
var ok = results.filter(function(r) { return r.sucesso; }).length; console.log('[SIEG-CANCEL-POLL] Concluido: ' + ok + '/' + results.length + ' cancelada(s)'); return { processed: results.length, sucesso: ok }; } async function cancelOneInvoice(client, db, uid, pwd, moveId, cancelarNFe) { var moves = await executeKw(client, db, uid, pwd, 'account.move', 'read', [[moveId], ['name', 'x_studio_nfe_chave', 'x_studio_nfe_protocolo', 'x_studio_nfe_status', 'company_id']]); if (!moves || !moves.length) throw new Error('Fatura ' + moveId + ' nao encontrada'); var move = moves[0]; var chave = (move.x_studio_nfe_chave || '').replace(/\D/g, ''); var protocolo = String(move.x_studio_nfe_protocolo || '').trim(); var justificativa = 'Cancelamento solicitado pelo emitente via Odoo'; if (justificativa.length < 15) justificativa = 'Cancelamento solicitado pelo emitente via Odoo'; console.log('[SIEG-CANCEL-POLL] Fatura ' + move.name + ' | chave=' + chave.slice(0, 15) + '...'); if (!chave || chave.length !== 44) throw new Error('Chave de acesso invalida (' + chave.length + ' digitos). Nota precisa estar autorizada.'); if (!protocolo) throw new Error('Protocolo de autorizacao ausente na fatura ' + move.name); var companyId = Array.isArray(move.company_id) ? move.company_id[0] : move.company_id; var companies = await executeKw(client, db, uid, pwd, 'res.company', 'read', [[companyId], ['vat']]); var cnpj = ((companies && companies[0] && companies[0].vat) || '').replace(/\D/g, ''); if (!cnpj) throw new Error('CNPJ do emitente nao encontrado na empresa'); // Marcar como processando para nao processar duas vezes em paralelo await executeKw(client, db, uid, pwd, 'account.move', 'write', [[moveId], { x_studio_nfe_status: 'processando' }]); var resultado = await cancelarNFe({ chave, protocolo, cnpj, justificativa }); var statusNovo = resultado.sucesso ? 'cancelada' : 'erro'; await executeKw(client, db, uid, pwd, 'account.move', 'write', [[moveId], { x_studio_nfe_status: statusNovo, x_studio_nfe_protocolo: resultado.nProt || protocolo, }]); if (resultado.sucesso) { try { await executeKw(client, db, uid, pwd, 'account.move', 'button_draft', [[moveId]]); await executeKw(client, db, uid, pwd, 'account.move', 'button_cancel', [[moveId]]); console.log('[SIEG-CANCEL-POLL] Fatura ' + move.name + ' revertida no Odoo'); } catch (e) { console.warn('[SIEG-CANCEL-POLL] Nao foi possivel reverter a fatura: ' + e.message); } } var msgCorpo = resultado.sucesso ? '<b>NF-e Cancelada na SEFAZ</b><br/>' + 'Protocolo: ' + (resultado.nProt || protocolo) + '<br/>' + 'Data: ' + (resultado.dhRecbto || '') + '<br/>' + 'Justificativa: ' + justificativa : '<b>Cancelamento rejeitado pela SEFAZ</b><br/>' + 'Status: ' + resultado.cStat + ' - ' + resultado.xMotivo + '<br/>' + 'Justificativa enviada: ' + justificativa; try { await executeKw(client, db, uid, pwd, 'mail.message', 'create', [{ model: 'account.move', res_id: moveId, body: msgCorpo, message_type: 'comment', }]); } catch (e) { console.warn('[SIEG-CANCEL-POLL] Erro ao postar chatter:', e.message); } return { move_id: moveId, sucesso: resultado.sucesso, fatura: move.name, cStat: resultado.cStat }; } module.exports = { processPendingEmissions, processPendingCancellations };
```

8. Cancelamento de NF-e

O cancelamento de NF-e implementa o evento de cancelamento conforme a NT2011.002 da SEFAZ. O fluxo monta o XML do evento evCancNFe (tpEvento 110111) com a justificativa, assina com o certificado A1, envia via SOAP para NFeRecepcaoEvento4 e verifica o cStat 135 (evento registrado) ou 155 (cancelamento homologado). Apos o cancelamento na SEFAZ, o sistema reverte a fatura para rascunho no Odoo (button_draft + button_cancel) e posta o resultado no chatter.

8.1 services/nfe-cancelamento.js

Cancelamento de NF-e na SEFAZ (NT2011.002). Monta evCancNFe, assina com A1, envia via SOAP para NFeRecepcaoEvento4. Verifica cStat 135/155.

nfe-cancelamento.js (/home/z/my-project/API-Odoo/services/nfe-cancelamento.js)

```
/** * services/nfe-cancelamento.js — Cancelamento de NF-e na SEFAZ (NT2011.002) *
================================================================================ * Fluxo: * 1. Monta o evento de cancelamento (evCancNFe) com justificativa * 2. Assina com o certificado A1 da empresa (XMLDSig RSA-SHA1) * 3. Envia via SOAP para NFeRecepcaoEvento4 * 4. Retorna cStat 135 (cancelada) ou erro * * Endpoint SEFAZ PR: * Prod: https://nfe.sefa.pr.gov.br/nfe/NFeRecepcaoEvento4 * Hom: https://homologacao.nfe.sefa.pr.gov.br/nfe/NFeRecepcaoEvento4 */ 'use strict'; var { SignedXml } = require('xml-crypto'); var { carregarCertificado } = require('./nfe-cert'); var { soapPost, endpoint, uf, tpAmb, cUF } = require('./sefaz-client'); var { minify } = require('./nfe-signer'); var C14N = 'http://www.w3.org/TR/2001/REC-xml-c14n-20010315'; var ENVELOPED = 'http://www.w3.org/2000/09/xmldsig#enveloped-signature'; var SHA1 = 'http://www.w3.org/2000/09/xmldsig#sha1'; var RSA_SHA1 = 'http://www.w3.org/2000/09/xmldsig#rsa-sha1'; /** Formata data-hora no padrão SEFAZ: YYYY-MM-DDTHH:MM:SS-03:00 * O servidor pode estar em UTC (Render). Converte UTC -> BRT (UTC-3) explicitamente. */ function dhEvento() { // Pega o timestamp UTC e subtrai 3h para obter horário de Brasília var d = new Date(Date.now() - 3 * 60 * 60 * 1000); var pad = function(n) { return String(n).padStart(2, '0'); }; return d.getUTCFullYear() + '-' + pad(d.getUTCMonth()+1) + '-' + pad(d.getUTCDate()) + 'T' + pad(d.getUTCHours()) + ':' + pad(d.getUTCMinutes()) + ':' + pad(d.getUTCSeconds()) + '-03:00'; } /** Extrai tag do XML de resposta */ function tag(xml, name) { var m = xml && xml.match(new RegExp('<' + name + '[^>]*>([\\s\\S]*?)</' + name + '>')); return m ? m[1] : ''; } /** * Cancela uma NF-e autorizada na SEFAZ. * @param {object} opts * @param {string} opts.chave - Chave de acesso de 44 dígitos * @param {string} opts.protocolo - Número do protocolo de autorização (nProt) * @param {string} opts.cnpj - CNPJ do emitente (14 dígitos, sem pontuação) * @param {string} opts.justificativa - Justificativa (15-255 chars) * @returns {Promise<{sucesso:boolean, cStat:string, xMotivo:string, nProt:string, dhRecbto:string}>} */ async function cancelarNFe(opts) { var chave = String(opts.chave || '').replace(/\D/g, '').slice(0, 44); var protocolo = String(opts.protocolo || ''); var cnpj = String(opts.cnpj || '').replace(/\D/g, '').slice(0, 14); var justificativa = String(opts.justificativa || 'Cancelamento solicitado pelo emitente').trim(); if (chave.length !== 44) throw new Error('Chave de acesso inválida (' + chave.length + ' dígitos, esperado 44)'); if (!protocolo) throw new Error('Número do protocolo de autorização é obrigatório'); if (!cnpj || cnpj.length !== 14) throw new Error('CNPJ do emitente inválido'); if (justificativa.length < 15) throw new Error('Justificativa mínima: 15 caracteres'); if
```

```
(justificativa.length > 255) justificativa = justificativa.slice(0, 255); var cert = carregarCertificado(); if (!cert) throw new Error('Certificado digital A1 não configurado'); if (cert.info && cert.info.expirado) throw new Error('Certificado expirado em ' + cert.info.validoAte); var nSeqEvento = '1'; var dhEv = dhEvento(); var idEvento = 'ID110111' + chave + nSeqEvento.padStart(2, '0'); // -------- Montar XML do evento -------- var xmlEvento = minify( '<evento versao="1.00" xmlns="http://www.portalfiscal.inf.br/nfe">' + '<infEvento Id="' + idEvento + '">' + '<cOrgao>' + cUF() + '</cOrgao>' + '<tpAmb>' + tpAmb() + '</tpAmb>' + '<CNPJ>' + cnpj + '</CNPJ>' + '<chNFe>' + chave + '</chNFe>' + '<dhEvento>' + dhEv + '</dhEvento>' +

'<tpEvento>110111</tpEvento>' + '<nSeqEvento>' + nSeqEvento + '</nSeqEvento>' + '<verEvento>1.00</verEvento>' + '<detEvento versao="1.00">' + '<descEvento>Cancelamento</descEvento>' + '<nProt>' + protocolo + '</nProt>' + '<xJust>' + justificativa + '</xJust>' + '</detEvento>' + '</infEvento>' + '</evento>' ); // -------- Assinar o evento -------- var sig = new SignedXml({ privateKey: cert.privateKeyPem, publicCert: cert.certPem, signatureAlgorithm: RSA_SHA1, canonicalizationAlgorithm: C14N, }); sig.addReference({ xpath: "//*[local-name(.)='infEvento']", transforms: [ENVELOPED, C14N], digestAlgorithm: SHA1, uri: '#' + idEvento, }); sig.computeSignature(xmlEvento, { location: { reference: "//*[local-name(.)='infEvento']", action: 'after' }, }); var xmlAssinado = sig.getSignedXml(); // -------- Montar lote de eventos -------- var idLote = Date.now().toString().slice(-15); var xmlLote = minify( '<envEvento versao="1.00" xmlns="http://www.portalfiscal.inf.br/nfe">' + '<idLote>' + idLote + '</idLote>' + xmlAssinado + '</envEvento>' ); // -------- URL do endpoint -------- var baseUrl = endpoint('cancelamento'); console.log('[NFE-CANCEL] Enviando cancelamento chave=' + chave + ' prot=' + protocolo + ' -> ' + baseUrl); // -------- Enviar via SOAP -------- var respRaw = await soapPost(baseUrl, 'NFeRecepcaoEvento4', xmlLote, 30000); var respXml = typeof respRaw === 'string' ? respRaw : (respRaw && respRaw.xml ? String(respRaw.xml) : ''); console.log('[NFE-CANCEL] HTTP status=' + (respRaw && respRaw.status) + ' xml_len=' + respXml.length); // cStat do lote (128 = processado) — o resultado real fica dentro de <retEvento> var cStatLote = tag(respXml, 'cStat'); var xMotivoLote = tag(respXml, 'xMotivo'); console.log('[NFE-CANCEL] cStat lote: ' + cStatLote + ' - ' + xMotivoLote); // Extrair bloco <retEvento> para pegar o cStat individual do evento var retEventoMatch = respXml.match(/<retEvento[\s\S]*?<\/retEvento>/); var retEventoXml = retEventoMatch ? retEventoMatch[0] : respXml; var cStat = tag(retEventoXml, 'cStat'); var xMotivo = tag(retEventoXml, 'xMotivo'); var nProt = tag(retEventoXml, 'nProt'); var dhRecbto = tag(retEventoXml, 'dhRecbto'); // cStat 135 = Evento registrado e vinculado à NF-e (cancelamento confirmado) // cStat 155 = Cancelamento homologado (também aceito) var sucesso = cStat === '135' || cStat === '155'; console.log('[NFE-CANCEL] Resultado evento: cStat=' + cStat + ' - ' + xMotivo + (nProt ? ' nProt=' + nProt : '')); return { sucesso, cStat, xMotivo, nProt, dhRecbto }; } module.exports = { cancelarNFe };
```

8.2 services/nfe-cancelar-odoo.js

Cancelamento de NF-e integrado ao Odoo. Le fatura (chave + protocolo), envia cancelamento a SEFAZ, atualiza status, reverte fatura para rascunho e posta no chatter.

nfe-cancelar-odoo.js (/home/z/my-project/API-Odoo/services/nfe-cancelar-odoo.js)

```
/** * services/nfe-cancelar-odoo.js — Cancelamento de NF-e integrado ao Odoo *
================================================================================ * 1. Lê a fatura no Odoo para obter chave + protocolo de autorização * 2. Envia o evento de cancelamento à SEFAZ PR * 3. Atualiza o status da fatura no Odoo para "cancelada" * 4. Reverte a fatura para rascunho (button_draft) e cancela (button_cancel) * 5. Posta mensagem no chatter com resultado */ 'use strict'; var xmlrpc = require('xmlrpc'); var config = require('../config'); var { cancelarNFe } = require('./nfe-cancelamento'); // ── XML-RPC helpers (mesmo padrão do sieg-odoo-emit.js) ──────────────────── function createClient(url) { var base = url.replace(/\/+$/, ''); var host = base.replace('https://', '').replace('http://', ''); var port = base.startsWith('https') ? 443 : 80; var isSecure = base.startsWith('https'); var fn = isSecure ? xmlrpc.createSecureClient : xmlrpc.createClient; return { common: fn({ host, path: '/xmlrpc/2/common', port }), models: fn({ host, path: '/xmlrpc/2/object', port }), }; } function authenticate(client, db, login, password) { return new Promise(function(resolve, reject) { client.common.methodCall('authenticate', [db, login, password, {}], function(err, uid) { if (err) return reject(new Error('Auth Odoo: ' + (err.message || err))); if (!uid) return reject(new Error('Credenciais Odoo inválidas')); resolve(uid); }); }); } function executeKw(client, db, uid, password, model, method, args, kwargs) { return new Promise(function(resolve, reject) { var params = [db, uid, password, model, method, args || []]; if (kwargs) params.push(kwargs); client.models.methodCall('execute_kw', params, function(err, result) { if (err) return reject(new Error(model + '.' + method + ': ' + (err.faultString || err.message || err))); resolve(result); }); }); } // ── Principal ──────────────────────────────────────────────────────────── async function cancelarNFeOdoo({ moveId, justificativa }) { var odooUrl = config.odoo.url; var db = config.odoo.db; var login = config.odoo.login; var pwd = config.odoo.password; var client = createClient(odooUrl); var uid = await authenticate(client, db, login, pwd); // 1. Ler a fatura var moves = await executeKw(client, db, uid, pwd, 'account.move', 'read', [[Number(moveId)], ['name', 'state', 'x_studio_nfe_chave', 'x_studio_nfe_protocolo', 'x_studio_nfe_status', 'company_id']]); if (!moves || !moves.length) throw new Error('Fatura não encontrada: move_id=' + moveId); var move = moves[0]; var chave = (move.x_studio_nfe_chave || '').replace(/\D/g, ''); var protocolo = String(move.x_studio_nfe_protocolo || '').trim(); var status = move.x_studio_nfe_status || ''; console.log('[NFE-CANCEL] Fatura ' + move.name + ' | chave=' + chave.slice(0,15) + '... | protocolo=' + protocolo + ' | status=' + status); if (!chave || chave.length !== 44) throw new Error('Chave de acesso inválida ou ausente na fatura (' + chave.length + ' dígitos). A nota precisa estar autorizada.'); if (!protocolo) throw new Error('Protocolo de autorização ausente. A nota precisa estar autorizada (cStat 100).'); if (status !== 'autorizada') throw new Error('Fatura não está autorizada (status atual: "' + status + '"). Só é possível cancelar notas autorizadas.'); // 2. CNPJ do emitente (via company) var companyId = Array.isArray(move.company_id) ? move.company_id[0] : move.company_id; var companies = await executeKw(client, db, uid, pwd, 'res.company', 'read', [[companyId], ['vat']]); var cnpj = (companies && companies[0] && companies[0].vat || '').replace(/\D/g, '');

if (!cnpj) throw new Error('CNPJ do emitente não encontrado na empresa do Odoo'); // 3. Enviar cancelamento à SEFAZ var resultado = await cancelarNFe({ chave, protocolo, cnpj, justificativa }); // 4. Atualizar status no Odoo independente do resultado var statusNovo = resultado.sucesso ? 'cancelada' : 'erro'; var msgSefaz = resultado.cStat + ' - ' + resultado.xMotivo; await executeKw(client, db, uid, pwd, 'account.move', 'write', [[Number(moveId)], { x_studio_nfe_status: statusNovo, x_studio_nfe_protocolo: resultado.nProt || move.x_studio_nfe_protocolo, }]); // 5.
```

```
Reverter fatura para rascunho + cancelar (apenas se SEFAZ confirmou) if (resultado.sucesso) { try { // Odoo 17+/18+: button_draft reverte a fatura para rascunho await executeKw(client, db, uid, pwd, 'account.move', 'button_draft', [[Number(moveId)]]); await executeKw(client, db, uid, pwd, 'account.move', 'button_cancel', [[Number(moveId)]]); console.log('[NFE-CANCEL] Fatura ' + move.name + ' revertida para rascunho e cancelada no Odoo'); } catch (e) { // Pode falhar se a fatura já foi paga — apenas logar console.warn('[NFE-CANCEL] Nao foi possivel reverter a fatura no Odoo: ' + e.message); } } // 6. Postar mensagem no chatter var msgCorpo = resultado.sucesso ? '<p> <strong>NF-e Cancelada na SEFAZ</strong><br/>' + 'Protocolo: ' + (resultado.nProt || protocolo) + '<br/>' + 'Data: ' + (resultado.dhRecbto || '') + '<br/>' + 'Justificativa: ' + justificativa + '</p>' : '<p> <strong>Cancelamento rejeitado pela SEFAZ</strong><br/>' + 'Status: ' + msgSefaz + '<br/>' + 'Justificativa enviada: ' + justificativa + '</p>'; try { await executeKw(client, db, uid, pwd, 'mail.message', 'create', [{ model: 'account.move', res_id: Number(moveId), message_type: 'comment', body: msgCorpo, subtype_id: 1, }]); } catch (e) { console.warn('[NFE-CANCEL] Nao foi possivel postar no chatter: ' + e.message); } console.log('[NFE-CANCEL] Concluido: sucesso=' + resultado.sucesso + ' cStat=' + resultado.cStat); return { sucesso: resultado.sucesso, cStat: resultado.cStat, xMotivo: resultado.xMotivo, nProt: resultado.nProt, dhRecbto: resultado.dhRecbto, fatura: move.name, odoo_status: statusNovo, }; } module.exports = { cancelarNFeOdoo };
```

9. DANFE - PDF

O DANFE (Documento Auxiliar da Nota Fiscal Eletronica) e gerado localmente usando PDFKit e bwip-js (codigo de barras Code128C da chave de acesso). O layout segue o padrao simplificado em A4 retrato: canhoto, cabecalho com emitente e chave de acesso com barcode, dados do destinatario, calculo de impostos, dados do transportador, tabela de itens e dados adicionais. Em ambiente de homologacao, inclui marca dagua "SEM VALOR FISCAL".

services/danfe-pdf.js

Geracao local do DANFE em PDF (PDFKit + bwip-js). Layout A4 retrato com canhoto, emitente, destinatario, impostos, transportador, itens e dados adicionais. Barcode Code128C da chave.

danfe-pdf.js (/home/z/my-project/API-Odoo/services/danfe-pdf.js)

```
/** * services/danfe-pdf.js — Geracao local do DANFE (PDF) a partir do nfeProc * ========== * Sem dependencia da SIEG: usa pdfkit + bwip-js (codigo de barras Code128C * da chave de acesso). Layout retrato A4 no padrao do DANFE simplificado: * canhoto, identificacao, emitente/destinatario, impostos, itens e dados * adicionais. */ var PDFDocument = require('pdfkit'); var bwipjs = require('bwip-js'); function tag(xml, name) { var m = String(xml || '').match(new RegExp('<' + name + '[^>]*>([\\s\\S]*?)<\\/' + name + '>', 'i')); return m ? m[1].trim() : ''; } function tagIn(xml, path) { var cur = xml; var parts = path.split('.'); for (var i = 0; i < parts.length; i++) cur = tag(cur, parts[i]); return cur; } function bloco(xml, name) { var m = String(xml || '').match(new RegExp('<' + name + '[\\s>][\\s\\S]*?<\\/' + name + '>', 'i')); return m ? m[0] : ''; } function blocos(xml, name) { var re = new RegExp('<' + name + '[\\s>][\\s\\S]*?<\\/' + name + '>', 'gi'); return String(xml || '').match(re) || []; } function moeda(v) { var n = parseFloat(v || 0) || 0; return n.toLocaleString('pt-BR', { minimumFractionDigits: 2, maximumFractionDigits: 2 }); } function qtd(v) { var n = parseFloat(v || 0) || 0; return n.toLocaleString('pt-BR', { minimumFractionDigits: 2, maximumFractionDigits: 4 }); } function cnpjFmt(v) { var s = String(v || '').replace(/\D/g, ''); if (s.length === 14) return s.replace(/^(\d{2})(\d{3})(\d{3})(\d{4})(\d{2})$/, '$1.$2.$3/$4-$5'); if (s.length === 11) return s.replace(/^(\d{3})(\d{3})(\d{3})(\d{2})$/, '$1.$2.$3-$4'); return s; } function chaveFmt(c) { return String(c || '').replace(/(\d{4})(?=\d)/g, '$1 ').trim(); } function dataFmt(iso) { if (!iso) return ''; var d = new Date(iso); if (isNaN(d.getTime())) return String(iso).slice(0, 10); var p = function (n) { return String(n).padStart(2, '0'); }; return p(d.getDate()) + '/' + p(d.getMonth() + 1) + '/' + d.getFullYear() + ' ' + p(d.getHours()) + ':' + p(d.getMinutes()); } async function barcode(chave) { return bwipjs.toBuffer({ bcid: 'code128', text: String(chave), scale: 3, height: 12, includetext: false, paddingwidth: 0, paddingheight: 0, }); } /** * Gera o PDF do DANFE. * @param {string} nfeProcXml XML autorizado (nfeProc) ou NFe + protNFe * @returns {Promise<Buffer>} */ async function gerarDanfePdf(nfeProcXml) { var xml = String(nfeProcXml || ''); var infNFe = bloco(xml, 'infNFe'); if (!infNFe) throw new Error('XML sem <infNFe> — nao eh possivel gerar o DANFE.'); var ide = bloco(infNFe, 'ide'); var emit = bloco(infNFe, 'emit'); var dest = bloco(infNFe, 'dest'); var total = bloco(infNFe, 'ICMSTot');

var transp = bloco(infNFe, 'transp'); var infAdic = bloco(infNFe, 'infAdic'); var prot = bloco(xml, 'infProt'); var chave = (infNFe.match(/Id="NFe(\d{44})"/) || [])[1] || tag(prot, 'chNFe'); var tpAmb = tag(ide, 'tpAmb'); var homolog = tpAmb === '2'; var doc = new PDFDocument({ size: 'A4', margin: 24 }); var chunks = []; doc.on('data', function (c) { chunks.push(c); }); var done = new Promise(function (resolve) { doc.on('end', function () { resolve(Buffer.concat(chunks)); }); }); var L = doc.page.margins.left; var W = doc.page.width - doc.page.margins.left - doc.page.margins.right; function box(x, y, w, h) { doc.lineWidth(0.6).rect(x, y, w, h).stroke(); } function label(x, y, t) { doc.fontSize(5.5).font('Helvetica').fillColor('#444').text(t, x + 3, y + 2, { width: 200 }); doc.fillColor('#000'); } function value(x, y, w, t, size, bold) { doc.fontSize(size || 8).font(bold ? 'Helvetica-Bold' : 'Helvetica') .text(String(t == null ? '' : t), x + 3, y + 9, { width: w - 6, ellipsis: true, lineBreak: false }); } function campo(x, y, w, h, lab, val, size, bold) { box(x, y, w, h); label(x, y, lab); value(x, y, w, val, size, bold); } var y = doc.page.margins.top; // Marca d'agua de homologacao if (homolog) { doc.save().rotate(-40, { origin: [300, 400] }).fontSize(38).fillColor('#e5e5e5') .text('SEM VALOR FISCAL - HOMOLOGACAO', 20, 380).restore().fillColor('#000'); } // --- Canhoto --- box(L, y, W - 110, 26); doc.fontSize(5.5).text('RECEBEMOS DE ' + tag(emit, 'xNome') + ' OS PRODUTOS CONSTANTES DA NOTA FISCAL INDICADA AO LADO', L + 3, y + 3, { width: W - 120 }); doc.fontSize(5.5).text('DATA DE RECEBIMENTO', L + 3, y + 15);
```

```
doc.text('IDENTIFICACAO E ASSINATURA DO RECEBEDOR', L + 130, y + 15); box(L + W - 110, y, 110, 26);
doc.fontSize(7).font('Helvetica-Bold') .text('NF-e N. ' + tag(ide, 'nNF') + '\nSERIE ' + tag(ide, 'serie'), L + W -
105, y + 6); y += 32; // --- Cabecalho: emitente + chave --- var hHead = 78; box(L, y, W, hHead);
doc.fontSize(9).font('Helvetica-Bold').text(tag(emit, 'xNome'), L + 5, y + 6, { width: 200 }); var eEnd = bloco(emit,
'enderEmit'); doc.fontSize(6.5).font('Helvetica').text( tag(eEnd, 'xLgr') + ', ' + tag(eEnd, 'nro') + ' ' + tag(eEnd,
'xBairro') + '\n' + tag(eEnd, 'xMun') + ' - ' + tag(eEnd, 'UF') + ' CEP ' + tag(eEnd, 'CEP') + '\n' + 'CNPJ ' +
cnpjFmt(tag(emit, 'CNPJ')) + ' IE ' + tag(emit, 'IE') + '\n' + 'Fone ' + (tag(eEnd, 'fone') || '-'), L + 5, y + 22, {
width: 200 } ); doc.fontSize(13).font('Helvetica-Bold').text('DANFE', L + 215, y + 6, { width: 90, align: 'center'
}); doc.fontSize(5.5).font('Helvetica').text('Documento Auxiliar da Nota Fiscal Eletronica', L + 210, y + 22, {
width: 100, align: 'center' }); doc.fontSize(7).font('Helvetica-Bold').text((tag(ide, 'tpNF') === '1' ? '1 - SAIDA' :
'0 - ENTRADA'), L + 210, y + 36, { width: 100, align: 'center' }); doc.fontSize(8).text('N. ' + tag(ide, 'nNF') +
'\nSERIE ' + tag(ide, 'serie'), L + 210, y + 48, { width: 100, align: 'center' }); // Codigo de barras try { var png =
await barcode(chave); doc.image(png, L + 320, y + 6, { width: W - 330, height: 30 }); } catch (e) {
doc.fontSize(6).text('(codigo de barras indisponivel)', L + 320, y + 16); } box(L + 315, y + 40, W - 315, 18); label(L
+ 315, y + 40, 'CHAVE DE ACESSO'); doc.fontSize(7).font('Helvetica-Bold').text(chaveFmt(chave), L + 318, y + 49, {
width: W - 322 }); box(L + 315, y + 58, W - 315, 20); doc.fontSize(6).font('Helvetica').text( 'Consulta de
autenticidade no portal nacional da NF-e www.nfe.fazenda.gov.br/portal ou no site da SEFAZ autorizadora', L + 318, y
+ 62, { width: W - 322 } ); y += hHead + 2; // --- Protocolo / natureza --- campo(L, y, W * 0.55, 20, 'NATUREZA DA
OPERACAO', tag(ide, 'natOp'), 7); campo(L + W * 0.55, y, W * 0.45, 20, 'PROTOCOLO DE AUTORIZACAO DE USO',

(tag(prot, 'nProt') || '-') + ' ' + dataFmt(tag(prot, 'dhRecbto')), 7); y += 22; campo(L, y, W / 3, 20, 'INSCRICAO
ESTADUAL', tag(emit, 'IE'), 7); campo(L + W / 3, y, W / 3, 20, 'INSC. ESTADUAL DO SUBST. TRIB.', tag(emit, 'IEST') ||
'', 7); campo(L + 2 * W / 3, y, W / 3, 20, 'CNPJ', cnpjFmt(tag(emit, 'CNPJ')), 7); y += 24; // --- Destinatario ---
doc.fontSize(6).font('Helvetica-Bold').text('DESTINATARIO / REMETENTE', L, y); y += 8; var dEnd = bloco(dest,
'enderDest'); campo(L, y, W * 0.55, 20, 'NOME / RAZAO SOCIAL', tag(dest, 'xNome'), 7); campo(L + W * 0.55, y, W *
0.25, 20, 'CNPJ / CPF', cnpjFmt(tag(dest, 'CNPJ') || tag(dest, 'CPF')), 7); campo(L + W * 0.80, y, W * 0.20, 20, 'DATA
DA EMISSAO', dataFmt(tag(ide, 'dhEmi')), 7); y += 20; campo(L, y, W * 0.45, 20, 'ENDERECO', tag(dEnd, 'xLgr') + ', ' +
tag(dEnd, 'nro'), 7); campo(L + W * 0.45, y, W * 0.25, 20, 'BAIRRO', tag(dEnd, 'xBairro'), 7); campo(L + W * 0.70, y,
W * 0.15, 20, 'CEP', tag(dEnd, 'CEP'), 7); campo(L + W * 0.85, y, W * 0.15, 20, 'DATA SAIDA/ENTRADA', dataFmt(tag(ide,
'dhSaiEnt')) || '-', 7); y += 20; campo(L, y, W * 0.45, 20, 'MUNICIPIO', tag(dEnd, 'xMun'), 7); campo(L + W * 0.45, y,
W * 0.10, 20, 'UF', tag(dEnd, 'UF'), 7); campo(L + W * 0.55, y, W * 0.15, 20, 'FONE', tag(dEnd, 'fone'), 7); campo(L +
W * 0.70, y, W * 0.30, 20, 'INSCRICAO ESTADUAL', tag(dest, 'IE') || 'ISENTO', 7); y += 24; // --- Totais ---
doc.fontSize(6).font('Helvetica-Bold').text('CALCULO DO IMPOSTO', L, y); y += 8; var c5 = W / 5; campo(L, y, c5, 20,
'BASE DE CALCULO DO ICMS', moeda(tag(total, 'vBC')), 7); campo(L + c5, y, c5, 20, 'VALOR DO ICMS', moeda(tag(total,
'vICMS')), 7); campo(L + 2 * c5, y, c5, 20, 'BASE DE CALCULO ICMS ST', moeda(tag(total, 'vBCST')), 7); campo(L + 3 *
c5, y, c5, 20, 'VALOR DO ICMS ST', moeda(tag(total, 'vST')), 7); campo(L + 4 * c5, y, c5, 20, 'VALOR TOTAL DOS
PRODUTOS', moeda(tag(total, 'vProd')), 7, true); y += 20; campo(L, y, c5, 20, 'VALOR DO FRETE', moeda(tag(total,
'vFrete')), 7); campo(L + c5, y, c5, 20, 'VALOR DO SEGURO', moeda(tag(total, 'vSeg')), 7); campo(L + 2 * c5, y, c5,
20, 'DESCONTO', moeda(tag(total, 'vDesc')), 7); campo(L + 3 * c5, y, c5, 20, 'OUTRAS DESPESAS', moeda(tag(total,
'vOutro')), 7); campo(L + 4 * c5, y, c5, 20, 'VALOR TOTAL DA NOTA', moeda(tag(total, 'vNF')), 8, true); y += 24; //
--- Transportador --- var vol = bloco(transp, 'vol'); doc.fontSize(6).font('Helvetica-Bold').text('TRANSPORTADOR /
VOLUMES TRANSPORTADOS', L, y); y += 8; var transporta = bloco(transp, 'transporta'); campo(L, y, W * 0.5, 20, 'NOME /
RAZAO SOCIAL', tag(transporta, 'xNome') || '-', 7); campo(L + W * 0.5, y, W * 0.2, 20, 'FRETE POR CONTA', tag(transp,
'modFrete') === '0' ? '0-EMITENTE' : (tag(transp, 'modFrete') === '9' ? '9-SEM FRETE' : tag(transp, 'modFrete')), 7);
campo(L + W * 0.7, y, W * 0.3, 20, 'CNPJ / CPF', cnpjFmt(tag(transporta, 'CNPJ') || tag(transporta, 'CPF')) || '-',
7); y += 20; campo(L, y, W * 0.2, 20, 'QUANTIDADE', tag(vol, 'qVol') || '-', 7); campo(L + W * 0.2, y, W * 0.2, 20,
'ESPECIE', tag(vol, 'esp') || '-', 7); campo(L + W * 0.4, y, W * 0.2, 20, 'MARCA', tag(vol, 'marca') || '-', 7);
campo(L + W * 0.6, y, W * 0.2, 20, 'PESO BRUTO', tag(vol, 'pesoB') || '-', 7); campo(L + W * 0.8, y, W * 0.2, 20,
'PESO LIQUIDO', tag(vol, 'pesoL') || '-', 7); y += 24; // --- Itens ---
doc.fontSize(6).font('Helvetica-Bold').text('DADOS DOS PRODUTOS / SERVICOS', L, y); y += 8; var cols = [ { t: 'COD',
w: 0.08 }, { t: 'DESCRICAO', w: 0.30 }, { t: 'NCM', w: 0.07 }, { t: 'CST', w: 0.05 }, { t: 'CFOP', w: 0.05 }, { t:
'UN', w: 0.04 }, { t: 'QTD', w: 0.08 }, { t: 'V.UNIT', w: 0.10 }, { t: 'V.TOTAL', w: 0.10 }, { t: 'BC ICMS', w: 0.07
}, { t: 'V.ICMS', w: 0.06 }, ]; var x = L; box(L, y, W, 12); cols.forEach(function (c) {
doc.fontSize(5.5).font('Helvetica-Bold').text(c.t, x + 2, y + 4, { width: W * c.w - 3, lineBreak: false }); x += W *
c.w; }); y += 12; var dets = blocos(infNFe, 'det'); dets.forEach(function (det) { if (y > doc.page.height - 130) {
doc.addPage(); y = doc.page.margins.top; } var prod = bloco(det, 'prod'); var icms = bloco(det, 'ICMS'); var h = 12;
box(L, y, W, h);

var vals = [ tag(prod, 'cProd'), tag(prod, 'xProd'), tag(prod, 'NCM'), tag(icms, 'CST') || tag(icms, 'CSOSN'),
tag(prod, 'CFOP'), tag(prod, 'uCom'), qtd(tag(prod, 'qCom')), moeda(tag(prod, 'vUnCom')), moeda(tag(prod, 'vProd')),
moeda(tag(icms, 'vBC')), moeda(tag(icms, 'vICMS')), ]; var xx = L; cols.forEach(function (c, i) { var align = i >= 6
? 'right' : 'left'; doc.fontSize(5.8).font('Helvetica') .text(String(vals[i] || ''), xx + 2, y + 4, { width: W * c.w
- 4, lineBreak: false, ellipsis: true, align: align }); xx += W * c.w; }); y += h; }); y += 6; // --- Dados adicionais
--- if (y > doc.page.height - 90) { doc.addPage(); y = doc.page.margins.top; }
doc.fontSize(6).font('Helvetica-Bold').text('DADOS ADICIONAIS', L, y); y += 8; var infoAd = (homolog ? 'AMBIENTE DE
HOMOLOGACAO - SEM VALOR FISCAL. ' : '') + (tag(infAdic, 'infCpl') || '') + ' ' + (tag(infAdic, 'infAdFisco') || '');
box(L, y, W, 60); doc.fontSize(6.5).font('Helvetica').text(infoAd.trim() || '-', L + 4, y + 4, { width: W - 8,
height: 52 }); y += 66; doc.fontSize(5.5).fillColor('#666') .text('DANFE gerado pelo middleware AJL (emissao propria
com certificado A1) em ' + dataFmt(new Date().toISOString()), L, y); doc.end(); return done; } module.exports = {
gerarDanfePdf };
```

10. Rotas da API

As rotas da API de NF-e estao organizadas em dois arquivos de rota Express. O routes/nfe-cert.js concentra os endpoints de gerenciamento do certificado digital e geracao de DANFE, incluindo um painel HTML para upload do certificado via navegador. O routes/sieg.js concentra os endpoints de

emissao (auto-detect NF-e/NFS-e, emissao forcada, processamento de pendentes, cancelamento) e o callback OAuth da SIEG.

10.1 routes/nfe-cert.js

Rotas: POST/GET/DELETE /api/v1/nfe/certificado, GET /api/v1/nfe/sefaz/status, POST /api/v1/nfe/danfe, GET /api/v1/nfe/painel (HTML de upload de certificado).

nfe-cert.js (/home/z/my-project/API-Odoo/routes/nfe-cert.js)

```
/** * routes/nfe-cert.js — Certificado A1 e emissao propria na SEFAZ * * POST /api/v1/nfe/certificado — upload do .pfx (base64 + senha) * GET /api/v1/nfe/certificado — status do certificado carregado * DELETE /api/v1/nfe/certificado — remove o certificado * GET /api/v1/nfe/sefaz/status — status do servico da SEFAZ (testa mTLS) * POST /api/v1/nfe/danfe — gera o DANFE PDF localmente a partir do XML */ var express = require('express'); var router = express.Router(); var { apiKeyAuth } = require('../middleware/auth'); var { salvarCertificado, statusCertificado, removerCertificado } = require('../services/nfe-cert'); var { statusServico } = require('../services/sefaz-client'); var { gerarDanfePdf } = require('../services/danfe-pdf'); // Upload do certificado A1 (protegido por API key) router.post('/certificado', apiKeyAuth, express.json({ limit: '10mb' }), function (req, res) { try { var body = req.body || {}; var b64 = body.pfxBase64 || body.pfx_base64 || body.certificado || body.arquivo; var senha = body.senha || body.password || body.pfxSenha; if (!b64) return res.status(400).json({ erro: 'Envie o arquivo .pfx em base64 no campo "pfxBase64".' }); if (!senha) return res.status(400).json({ erro: 'Campo "senha" obrigatorio.' }); var buf = Buffer.from(String(b64).replace(/^data:[^,]+,/, ''), 'base64'); var info = salvarCertificado(buf, senha); res.json({ sucesso: true, mensagem: 'Certificado A1 armazenado com sucesso.', certificado: info }); } catch (err) { console.error('[NFE-CERT] Erro no upload: ' + err.message); res.status(400).json({ sucesso: false, erro: err.message }); } }); router.get('/certificado', apiKeyAuth, function (req, res) { res.json(statusCertificado()); }); router.delete('/certificado', apiKeyAuth, function (req, res) { removerCertificado(); res.json({ sucesso: true, mensagem: 'Certificado removido.' }); }); // Testa conexao mTLS com a SEFAZ usando o certificado carregado router.get('/sefaz/status', apiKeyAuth, async function (req, res) { try { res.json(await statusServico()); } catch (err) { res.status(500).json({ erro: err.message }); } }); // Gera o DANFE PDF localmente (sem SIEG) router.post('/danfe', apiKeyAuth, express.json({ limit: '10mb' }), async function (req, res) { try { var xml = req.body && (req.body.xml || (req.body.xmlBase64 ? Buffer.from(req.body.xmlBase64, 'base64').toString('utf8') : '')); if (!xml) return res.status(400).json({ erro: 'Informe "xml" (nfeProc autorizado) ou "xmlBase64".' }); var pdf = await gerarDanfePdf(xml); if (req.query.base64 === '1') return res.json({ sucesso: true, pdfBase64: pdf.toString('base64') }); res.setHeader('Content-Type', 'application/pdf'); res.setHeader('Content-Disposition', 'inline; filename="danfe.pdf"'); res.send(pdf); } catch (err) { res.status(400).json({ erro: err.message }); } }); // Painel HTML simples para enviar o certificado sem usar terminal router.get('/painel', function (req, res) { res.setHeader('Content-Type', 'text/html; charset=utf-8'); res.send(PAINEL_HTML); }); var PAINEL_HTML = `<!doctype html> <html lang="pt-BR"><head><meta charset="utf-8"> <meta name="viewport" content="width=device-width,initial-scale=1"> <title>Certificado A1 - NF-e | Middleware AJL</title> <style> :root{color-scheme:light} body{font-family:system-ui,-apple-system,Segoe UI,Roboto,sans-serif;background:#f4f6f8;margin:0;padding:24px;color:#1c2430}

.card{max-width:620px;margin:0 auto 18px;background:#fff;border:1px solid #dde3ea;border-radius:12px;padding:22px} h1{font-size:20px;margin:0 0 4px} h2{font-size:16px;margin:0 0 14px} p.sub{color:#5b6875;margin:0 0 18px;font-size:14px} label{display:block;font-size:13px;font-weight:600;margin:14px 0 6px} input{width:100%;box-sizing:border-box;padding:10px;border:1px solid #cbd4de;border-radius:8px;font-size:14px} button{margin-top:18px;width:100%;padding:12px;border:0;border-radius:8px;background:#12508f;color:#fff;font-size:15px;font-weight:600;cursor:pointer} button.sec{background:#5b6875;margin-top:10px} button:disabled{opacity:.6;cursor:progress} pre{background:#0f172a;color:#d7e3f4;padding:14px;border-radius:8px;font-size:12px;white-space:pre-wrap;word-break:break-word;max-height:320px;overflow:auto} .ok{color:#0a7a41;font-weight:600}.err{color:#b3261e;font-weight:600} </style></head><body> <div class="card"> <h1>Certificado Digital A1 - NF-e</h1> <p class="sub">Envie o arquivo <b>.pfx</b> da AJL e a senha. O arquivo vai direto para o servidor por HTTPS e a senha fica criptografada em disco.</p> <label>API Key (a mesma que o Odoo usa)</label> <input id="key" type="password" placeholder="x-api-key" autocomplete="off"> <label>Arquivo do certificado (.pfx ou .p12)</label> <input id="pfx" type="file" accept=".pfx,.p12"> <label>Senha do certificado</label> <input id="senha" type="password" placeholder="senha do .pfx" autocomplete="off"> <button id="btnEnviar">Enviar certificado</button> <button id="btnStatus" class="sec">Ver certificado atual</button> <button id="btnSefaz" class="sec">Testar conexao com a SEFAZ</button> </div> <div class="card"><h2>Resultado</h2><pre id="out">Aguardando...</pre></div> <script> var out=document.getElementById('out'); function key(){var k=document.getElementById('key').value.trim();if(!k){show('Informe a API Key.',true);}return k;} function show(t,erro){out.textContent=typeof t==='string'?t:JSON.stringify(t,null,2);out.className=erro?'err':'';} async function call(m,p,body){ var r=await fetch(p,{method:m,headers:{'x-api-key':key(),'Content-Type':'application/json'}, body:body?JSON.stringify(body):undefined}); var j=await r.json().catch(function(){return{erro:'resposta invalida'}}); show('HTTP '+r.status+'\\n'+JSON.stringify(j,null,2), !r.ok); return j; } document.getElementById('btnStatus').onclick=function(){if(key())call('GET','/api/v1/nfe/certificado');}; document.getElementById('btnSefaz').onclick=function(){if(key()){show('Consultando a SEFAZ...');call('GET','/api/v1/nfe/sefaz/status');}}; document.getElementById('btnEnviar').onclick=function(){ var f=document.getElementById('pfx').files[0], s=document.getElementById('senha').value; if(!key())return; if(!f)return show('Selecione o arquivo .pfx.',true); if(!s)return show('Informe a senha.',true); var b=this; b.disabled=true; show('Enviando...'); var fr=new FileReader(); fr.onload=function(){ var b64=String(fr.result).split(',')[1]; call('POST','/api/v1/nfe/certificado',{pfxBase64:b64,senha:s}).finally(function(){b.disabled=false;}); }; fr.readAsDataURL(f); }; </script></body></html>`; module.exports = router;
```

10.2 routes/sieg.js

Rotas: GET /api/v1/sieg/status, GET /api/v1/sieg/oauth-url, POST /api/v1/sieg/emitir, POST /api/v1/sieg/emitir-nfe, POST /api/v1/sieg/emitir-nfse, POST /api/v1/sieg/danfe, POST /api/v1/sieg/process-pending, POST /api/v1/sieg/cancelar, GET /callback/sieg.

sieg.js (/home/z/my-project/API-Odoo/routes/sieg.js)

```
/** * routes/sieg.js — Rotas de emissao fiscal SIEG * * GET /api/v1/sieg/status — Status dos tokens SIEG * GET /api/v1/sieg/oauth-url — Gerar URL de autorizacao OAuth * POST /api/v1/sieg/emitir — Emitir NF-e ou NFS-e (auto-detect) * POST /api/v1/sieg/emitir-nfe — Emitir NF-e (produtos) * POST /api/v1/sieg/emitir-nfse — Emitir NFS-e (servicos) * POST /api/v1/sieg/danfe — Gerar DANFE a partir de XML * POST /api/v1/sieg/process-pending — Poll: processar emissoes pendentes * GET /callback/sieg — OAuth callback SIEG (recebe token temp) */ const express = require('express'); const router = express.Router(); const { apiKeyAuth } = require('../middleware/auth'); const { getTokenState, getOAuthAuthorizeUrl, exchangeTempToken, setOAuthToken, } = require('../services/sieg-auth'); const { emitirNota, enviarNFe, emitirNFSe, gerarDanfe, gerarDanfse } = require('../services/sieg-api'); const { processPendingEmissions } = require('../services/sieg-odoo-emit'); const { cancelarNFeOdoo } = require('../services/nfe-cancelar-odoo'); // === OAuth Callback === // A SIEG redireciona aqui apos o usuario autorizar o acesso. // O SIEG pode retornar o token temporario em varios query params. router.get('/callback/sieg', async (req, res) => { try { // Tentar varios nomes de parametro que o SIEG pode usar var temporaryToken = req.query.temporaryToken || req.query.tempToken || req.query.code || req.query.token || req.query.accessToken || req.query.AccessToken; var state = req.query.state; var redirectUri = req.protocol + '://' + req.get('host') + '/callback/sieg'; if (!temporaryToken) { console.log('[SIEG-CALLBACK] Callback recebido. Query:', JSON.stringify(req.query)); return res.status(400).json({ erro: 'Token temporario nao fornecido. Query params recebidos: ' + JSON.stringify(req.query) }); } console.log('[SIEG-CALLBACK] Token temporario recebido (' + temporaryToken.length + ' chars), trocando por definitivo... (state=' + state + ')'); var tokens = await exchangeTempToken(temporaryToken, state, redirectUri); res.send( '<html><body style="font-family:sans-serif;text-align:center;padding:50px;background:#f0f4f8">' + '<div style="max-width:500px;margin:0 auto;background:white;padding:40px;border-radius:12px;box-shadow:0 2px 10px rgba(0,0,0,0.1)">' + '<h2 style="color:#16a34a">SIEG Integrado com sucesso!</h2>' + '<p>Token OAuth definitivo obtido.</p>' + '<p style="color:#666;font-size:14px">Validade: 30 dias. Voce pode fechar esta aba.</p>' + '<p style="color:#999;font-size:12px">State: ' + (state || 'N/A') + '</p>' + '</div></body></html>' ); } catch (err) { console.error('[SIEG-CALLBACK] Erro:', err.message); res.status(500).send( '<html><body style="font-family:sans-serif;text-align:center;padding:50px">' + '<h2 style="color:#dc2626">Erro na autenticacao SIEG</h2>' + '<p>' + err.message + '</p>' + '<p style="color:#999">Verifique os logs do servidor para detalhes.</p>' + '</body></html>' ); } }); // === Status dos tokens === router.get('/status', apiKeyAuth, async (req, res) => { try { const state = getTokenState(); res.json({ sieg_auth: state }); } catch (err) { res.status(500).json({ erro: err.message }); } }); // === Gerar URL de autorizacao OAuth === // Retorna a URL que o usuario deve visitar para autorizar o acesso router.get('/oauth-url', apiKeyAuth, async (req, res) => {

try { var accessLevel = req.query.accessLevel || 'write'; var state = req.query.state || null; var result = getOAuthAuthorizeUrl(state, accessLevel); res.json({ authorize_url: result.url, state: result.state, instrucoes: 'Abra esta URL no navegador, faca login na SIEG e autorize o acesso. O token sera enviado para o callback configurado.' }); } catch (err) { res.status(500).json({ erro: err.message }); } }); // === Emitir NF (auto-detect NF-e ou NFS-e) === router.post('/emitir', apiKeyAuth, async (req, res) => { const t0 = Date.now(); try { const dados = req.body; if (!dados || !dados.company || !dados.partner || !dados.lines || !dados.lines.length) { return res.status(400).json({ erro: 'Dados obrigatorios: company, partner, lines' }); } console.log('[SIEG] Emitir NF pedido ' + (dados.order && dados.order.name || '?') + ' - ' + dados.lines.length + ' linhas'); const resultado = await emitirNota(dados); const duracao = Date.now() - t0; console.log('[SIEG] NF ' + resultado.tipo + ' processada em ' + duracao + 'ms - sucesso: ' + resultado.sucesso); res.json({ sucesso: resultado.sucesso, tipo: resultado.tipo, duracao_ms: duracao, resposta_sieg: resultado.resposta, erro: resultado.erro, pdf_gerado: resultado.pdfGerado, pdf_base64: resultado.pdfGerado ? resultado.pdfBase64 : undefined, }); } catch (err) { const duracao = Date.now() - t0; console.error('[SIEG] Erro ao emitir NF (' + duracao + 'ms):', err.message); res.status(500).json({ sucesso: false, duracao_ms: duracao, erro: err.message, detalhes: err.response && err.response.data || null, }); } }); // === Emitir NF-e (forcar produto) === router.post('/emitir-nfe', apiKeyAuth, async (req, res) => { try { req.body.tipo = 'nfe'; const resultado = await emitirNota(req.body); res.json({ sucesso: resultado.sucesso, tipo: 'nfe', resposta_sieg: resultado.resposta, erro: resultado.erro }); } catch (err) { res.status(500).json({ sucesso: false, erro: err.message }); } }); // === Emitir NFS-e (forcar servico) === router.post('/emitir-nfse', apiKeyAuth, async (req, res) => { try { req.body.tipo = 'nfse'; const resultado = await emitirNota(req.body); res.json({ sucesso: resultado.sucesso, tipo: 'nfse', resposta_sieg: resultado.resposta, erro: resultado.erro }); } catch (err) { res.status(500).json({ sucesso: false, erro: err.message }); } }); // === Gerar DANFE (PDF) a partir de XML === router.post('/danfe', apiKeyAuth, async (req, res) => { try { const xml = req.body.xml; if (!xml) return res.status(400).json({ erro: 'XML obrigatorio' }); const pdf = await gerarDanfe(xml); res.json({ sucesso: true, pdf_base64: pdf }); } catch (err) { res.status(500).json({ sucesso: false, erro: err.message });

} }); // === PROCESSAR EMISSOES PENDENTES (Polling / Cron) === router.post('/process-pending', apiKeyAuth, async (req, res) => { console.log('[SIEG] process-pending chamado'); try { const resultado = await processPendingEmissions(); res.json({ sucesso: true, processadas: resultado.processed, autorizadas: resultado.sucesso || 0, detalhes: resultado.detalhes || [], }); } catch (err) { console.error('[SIEG] Erro process-pending:', err.message); res.status(500).json({ sucesso: false, erro: err.message }); } }); // === WEBHOOK do Odoo === router.post('/webhook', apiKeyAuth, async (req, res) => { console.log('[SIEG] Webhook recebido do Odoo:', JSON.stringify(req.body).substring(0, 300)); try { const resultado = await processPendingEmissions(); res.json({ sucesso: true, processadas: resultado.processed, autorizadas: resultado.sucesso || 0, }); } catch (err) { console.error('[SIEG] Erro webhook:', err.message); res.status(500).json({ sucesso: false, erro: err.message }); } }); // === CANCELAR NF-e === // Recebe: { move_id, justificativa } // Cancela na SEFAZ + reverte fatura no Odoo router.post('/cancelar', apiKeyAuth, async (req, res) => { var moveId = req.body.move_id; var justificativa = req.body.justificativa || 'Cancelamento solicitado pelo emitente'; if (!moveId) return res.status(400).json({ sucesso: false, erro: 'move_id obrigatorio' }); console.log('[NFE-CANCEL] Solicitacao de cancelamento move_id=' + moveId + ' just=' + justificativa.slice(0, 50)); try { var resultado = await cancelarNFeOdoo({ moveId, justificativa }); res.json(resultado); } catch (err) { console.error('[NFE-CANCEL] Erro:', err.message); res.status(500).json({ sucesso: false, erro: err.message }); } }); module.exports = router;
```

11. Script Odoo - Botao Cancelar NF-e

O script criar-botao-cancelar-nfe.py e um utilitario que cria via XML-RPC um Server Action no Odoo, visivel como botao no menu Acao (engrenagem) do formulario de faturas (account.move). O botao verifica se a NF-e esta com status "autorizada" antes de chamar o middleware para o cancelamento, e exibe o resultado (protocolo ou erro) via UserError para o usuario.

criar-botao-cancelar-nfe.py

Script Python que cria o botao "Cancelar NF-e" no Odoo via XML-RPC. O botao chama POST /api/v1/sieg/cancelar no middleware e exibe resultado ao usuario.

criar-botao-cancelar-nfe.py (/home/z/my-project/API-Odoo/odoo-scripts/criar-botao-cancelar-nfe.py)

```
#!/usr/bin/env python3 """ odoo-scripts/criar-botao-cancelar-nfe.py ======================================== Cria via XML-RPC um Server Action "Cancelar NF-e" visivel como botao no menu Acao (engrenagem) do formulario de faturas (account.move). Execute: ODOO_URL=https://nytro.odoo.com ODOO_DB=nytro \ ODOO_LOGIN=admin ODOO_PASSWORD=xxx \ MIDDLEWARE_URL=https://api-odoo.onrender.com \ MIDDLEWARE_API_KEY=sua-chave \ python3 odoo-scripts/criar-botao-cancelar-nfe.py """ import os, xmlrpc.client ODOO_URL = os.environ['ODOO_URL'].rstrip('/') ODOO_DB = os.environ['ODOO_DB'] ODOO_LOGIN = os.environ['ODOO_LOGIN'] ODOO_PASSWORD = os.environ['ODOO_PASSWORD'] MIDDLEWARE_URL = os.environ.get('MIDDLEWARE_URL', 'https://api-odoo.onrender.com') MIDDLEWARE_KEY = os.environ.get('MIDDLEWARE_API_KEY', '') common = xmlrpc.client.ServerProxy(f'{ODOO_URL}/xmlrpc/2/common') uid = common.authenticate(ODOO_DB, ODOO_LOGIN, ODOO_PASSWORD, {}) if not uid: raise SystemExit('Autenticacao falhou') print(f'Autenticado como uid={uid}') models = xmlrpc.client.ServerProxy(f'{ODOO_URL}/xmlrpc/2/object') def kw(model, method, args=None, kwargs=None): return models.execute_kw(ODOO_DB, uid, ODOO_PASSWORD, model, method, args or [], kwargs or {}) # ID do modelo account.move model_ids = kw('ir.model', 'search', [[['model', '=', 'account.move']]]) if not model_ids: raise SystemExit('Modelo account.move nao encontrado') model_id = model_ids[0] codigo = f"""import urllib.request, json from odoo.exceptions import UserError move_id = record.id move_name = record.name nfe_status = record.x_studio_nfe_status or '' if nfe_status != 'autorizada': raise UserError('A NF-e precisa estar com status "autorizada" para ser cancelada. Status atual: ' + (nfe_status or 'vazio')) justificativa = 'Cancelamento solicitado pelo emitente via Odoo' url = '{MIDDLEWARE_URL}/api/v1/sieg/cancelar' payload = json.dumps({{'move_id': move_id, 'justificativa': justificativa}}).encode('utf-8') req = urllib.request.Request(url, data=payload, headers={{ 'Content-Type': 'application/json', 'X-Api-Key': '{MIDDLEWARE_KEY}', }}, method='POST') try: with urllib.request.urlopen(req, timeout=60) as resp: resultado = json.loads(resp.read().decode('utf-8')) if resultado.get('sucesso'): raise UserError('NF-e ' + move_name + ' cancelada! Protocolo: ' + str(resultado.get('nProt', ''))) else: raise UserError('Cancelamento rejeitado: ' + str(resultado.get('cStat', '')) + ' - ' + str(resultado.get('xMotivo', 'Erro'))) except urllib.error.URLError as e: raise UserError('Erro ao contatar middleware: ' + str(e)) """ action_id = kw('ir.actions.server', 'create', [{ 'name': 'Cancelar NF-e', 'model_id': model_id, 'binding_model_id': model_id, 'binding_view_types': 'form', 'state': 'code', 'code': codigo, }]) print(f'Botao "Cancelar NF-e" criado com sucesso! (ir.actions.server ID={action_id})') print('Acesse qualquer fatura no Odoo -> menu Acao (engrenagem) -> "Cancelar NF-e"')
```

12. Documentacao de Referencia

O repositorio inclui tambem o arquivo docs-emissao-propria-nfe.md com documentacao adicional sobre a configuracao do disco persistente no Render, instrucoes para envio do certificado, teste da conexao com a SEFAZ e procedimentos de emissao. Este arquivo serve como guia rapido de referencia para a operacao do modulo de NF-e em producao.

docs-emissao-propria-nfe.md

Documentacao de referencia: fluxo, variaveis de ambiente, envio de certificado, teste SEFAZ e endpoints.

docs-emissao-propria-nfe.md (/home/z/my-project/API-Odoo/docs-emissao-propria-nfe.md)

```
# Emissao propria de NF-e com certificado A1 (SEFAZ-PR) A partir desta versao a API **emite a NF-e sozinha**: assina o XML com o certificado digital A1 da AJL e envia direto para a SEFAZ do Parana. A SIEG deixa de ser obrigatoria para emitir — continua util apenas como cofre de XMLs (opcional) e para NFS-e. ## Fluxo ```text Odoo (fatura pendente) -> sieg-nfe-xml.js monta o XML da NF-e 4.00 -> nfe-signer.js assina infNFe com o A1 (RSA-SHA1 + C14N) -> sefaz-client.js NFeAutorizacao4 (SOAP 1.2 + mTLS, indSinc=1) cStat 100 -> monta o nfeProc -> danfe-pdf.js gera o DANFE em PDF localmente -> sieg-odoo-emit.js anexa .xml e .pdf e posta no chatter ``` ## 1. Disco persistente no Render Crie um **Disk** no servico, montado em `/var/data` (1 GB basta). Sem ele o certificado precisa ser reenviado a cada deploy — ou informado pelas variaveis `NFE_CERT_PFX_BASE64` / `NFE_CERT_SENHA`. ## 2. Variaveis de ambiente | Variavel | Valor | Descricao | |---|---|---| | `NFE_EMISSAO_MODO` | `proprio` | `proprio` = assina e autoriza na SEFAZ; `sieg` = so
```

importa XML autorizado | | `NFE_UF` | `PR` | UF do emitente; define o webservice | | `SIEG_TP_AMB` | `2` | 1 = producao, 2 = homologacao | | `NFE_CERT_DIR` | `/var/data` | Onde o `.pfx` fica guardado | | `NFE_CERT_KEK` | (string forte) | Cifra a senha do certificado em disco (AES-256-GCM) | | `NFE_DANFE_PROVIDER` | `local` | `local` (padrao) ou `sieg` | | `SIEG_IMPORT_XML` | `0` | `1` importa o XML autorizado no cofre SIEG | ## 3. Enviar o certificado **Jeito facil (navegador):** abra `https://sua-api.onrender.com/api/v1/nfe/painel`, cole a API Key, escolha o `.pfx`, digite a senha e clique em **Enviar certificado**. A mesma pagina tem os botoes "Ver certificado atual" e "Testar conexao com a SEFAZ". **Pelo terminal (opcional):** ```bash node scripts/enviar-certificado.js ./AJL.pfx "SENHA_DO_PFX" \ https://sua-api.onrender.com SUA_API_KEY ``` Ou direto por HTTP: ```bash curl -X POST https://sua-api.onrender.com/api/v1/nfe/certificado \ -H "x-api-key: SUA_API_KEY" -H "Content-Type: application/json" \ -d "{\"pfxBase64\":\"$(base64 -w0 AJL.pfx)\",\"senha\":\"SENHA\"}" ``` A resposta traz titular, CNPJ, emissor e validade. A senha nunca aparece em log e fica cifrada em disco. ## 4. Testar a conexao com a SEFAZ ```bash curl -H "x-api-key: SUA_API_KEY" https://sua-api.onrender.com/api/v1/nfe/sefaz/status ``` `cStat 107` = "Servico em Operacao" -> certificado valido e mTLS funcionando. ## 5. Emitir Basta deixar a fatura com `x_studio_nfe_status = pendente` no Odoo. O polling emite, e o chatter recebe: - `NFe<chave>.xml` — o `nfeProc` autorizado (NF-e + protocolo) - `DANFE-<fatura>.pdf` — gerado localmente Se a SEFAZ rejeitar, o chatter mostra o `cStat`, o `xMotivo` exato e o trecho do XML relacionado; a fatura volta para `erro` (ou `pendente`, se `NFE_STATUS_ON_ERROR=pendente`) para nova tentativa.

Endpoints novos | Metodo | Rota | Funcao | |---|---|---| | POST | `/api/v1/nfe/certificado` | Upload do `.pfx` (base64 + senha) | | GET | `/api/v1/nfe/certificado` | Status/validade do certificado | | DELETE | `/api/v1/nfe/certificado` | Remove o certificado | | GET | `/api/v1/nfe/sefaz/status` | Status do servico da SEFAZ (testa mTLS) | | POST | `/api/v1/nfe/danfe` | Gera o DANFE PDF a partir de um XML autorizado | Todos exigem o header `x-api-key`.